User Manual

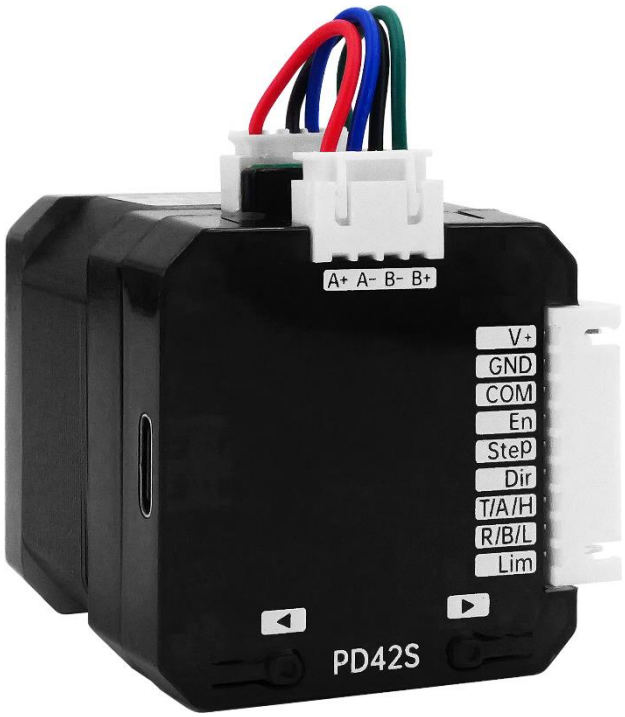
PD42S1 电机闭环驱动器用户手册

高性能步进电机 FOC 闭环驱动器

用户手册

正点原子

广州市星翼电子科技有限公司



修订历史

版本	日期	原因
V1.0	2025/10/30	第一次发布
V1.1	2025/12/23	优化算法，最高速度提升至 6000RPM，并新增双限位模式说明，以及 IO 启停引脚更换说明、新增设置速度环 PID 指令
V1.2	2026/02/26	1， 修复 0x2F 读取电机使能状态的错误描述 2， 新增 USB 通信功能（通过 USB 也可和上位机交互） 3， 增加 USB 通信连接描述

目 录

注意事项(必读).....1
1，特性参数.....2
2，快速入门.....4
2.1 硬件说明.....6
2.2 接线方法.....7
2.2.1 电机接线.....7
2.2.2 USB 通讯控制接线.....7
2.2.3 串口 TTL 通讯控制接线.....8
2.2.4 串口 RS485 通讯控制接线.....9
2.2.5 CAN 通讯控制接线.....10
2.2.6 脉冲模式控制接线.....11
2.2.7 左右限位开关接线.....12
2.2.8 IO 启停接线.....13
2.3 运动模式简介.....13
2.4 产品尺寸.....15
2.5 产品安装.....16
3，菜单说明.....19
3.1 参数显示说明.....19
3.2 菜单设置说明.....20
4，自定义协议指令使用说明.....22
4.1 指令格式说明.....25
4.2 系统类指令说明.....25
4.2.1 校准编码器指令.....25
4.2.2 复位重启指令.....26
4.2.3 恢复出厂设置指令.....26
4.2.4 参数保存指令.....27
4.3 参数类指令说明.....27
4.3.1 读取软硬件版本指令.....27
4.3.2 读取电机磁链指令.....28
4.3.3 读取相电阻和相电感指令.....30
4.3.4 读取相电流指令.....30
4.3.5 读取总线电压指令.....31
4.3.6 读取电流环 PID 参数指令.....31
4.3.7 读取速度环 PID 参数指令.....32
4.3.8 读取位置环 PID 参数指令.....32
4.3.9 读取输入累计脉冲数指令.....33
4.3.10 读取电机实时转速指令.....33
4.3.11 读取电机实时位置指令.....34
4.3.12 读取电机位置误差指令.....34
4.3.13 读取电机运行状态指令.....34
4.3.14 读取电机堵转标志指令.....35

4.3.15 读取堵转电流指令.....35
4.3.16 读取电机使能状态指令.....36
4.3.17 读取电机到位标志指令.....36
4.3.18 读取系统参数指令.....37
4.3.19 读取驱动参数指令.....38
4.3.20 设置从机地址指令.....40
4.3.21 设置分组地址指令.....40
4.3.22 设置工作模式指令.....41
4.3.23 设置位置环 PID 参数指令.....42
4.3.24 设置速度/位置环最大力矩限制指令.....43
4.3.25 设置细分指令.....43
4.3.26 设置目标电流指令.....44
4.3.27 设置串口波特率指令.....44
4.3.28 设置 CAN 波特率指令.....45
4.3.29 设置 moubus 指令.....45
4.3.30 设置堵转保护功能指令.....46
4.3.31 设置堵转电流指令.....46
4.3.32 设置发送 CAN_ID 指令.....46
4.3.33 电机正反转 DIR 控制指令.....47
4.3.34 设置 EN 引脚有效电平指令.....47
4.3.35 指令回响指令.....48
4.3.36 设置按键锁定指令.....48
4.3.37 设置自动熄屏指令.....49
4.3.38 设置 IO 启动指令.....49
4.3.39 设置速度环 PID 参数指令.....50
4.4 运动类指令说明.....51
4.4.1 力矩模式控制指令.....51
4.4.2 速度模式控制指令.....51
4.4.3 绝对位置模式控制指令.....52
4.4.4 相对位置模式控制指令.....52
4.4.5 脉冲模式控制指令.....53
4.4.6 脉宽位置模式控制指令.....54
4.4.7 脉宽力矩模式控制指令.....55
4.4.8 脉宽速度模式控制指令.....56
4.4.9 将当前的位置角度清零指令.....57
4.4.10 解除堵转保护指令.....57
4.4.11 电机使能控制指令.....57
4.4.12 清除状态（堵转、刹车，失能）指令.....58
4.4.13 立即停止（刹车）指令.....58
4.4.14 开环速度模式控制指令.....58
4.4.15 开环绝对位置模式控制指令.....59
4.4.16 开环相对位置模式控制指令.....60
4.4.17 开环脉冲模式控制指令.....60
4.4.18 IO 启停模式控制指令.....61

4.5 回零类指令说明.....62
4.5.1 设置左限位原点指令.....62
4.5.2 设置有无限位回零指令.....62
4.5.3 触发回零指令.....63
4.5.4 强制中断并退出回零操作指令.....63
4.5.5 读取回零参数指令.....64
4.5.6 修改原点回零超时时间指令.....65
4.5.7 读取回零状态指令.....65
4.5.8 设置上电自动回零指令.....66
4.5.9 设置右限位原点指令.....66
4.5.10 设置左右限位开关状态.....67
5，Modbus-RTU 指令使用说明.....68
5.1 指令格式说明.....71
5.2 系统类指令说明.....72
5.2.1 校准编码器指令.....72
5.2.2 复位重启指令.....72
5.2.3 恢复出厂设置指令.....73
5.2.4 参数保存指令.....73
5.3 参数类指令说明.....73
5.3.1 读取软硬件版本指令.....73
5.3.2 读取电机磁链指令.....74
5.3.3 读取相电阻和相电感指令.....74
5.3.4 读取相电流指令.....75
5.3.5 读取总线电压指令.....75
5.3.6 读取电流环 PID 参数指令.....75
5.3.7 读取速度环 PID 参数指令.....76
5.3.8 读取位置环 PID 参数指令.....76
5.3.9 读取输入累计脉冲数指令.....77
5.3.10 读取电机实时转速指令.....77
5.3.11 读取电机实时位置指令.....78
4.3.12 读取电机位置误差指令.....78
5.3.13 读取电机运行状态指令.....79
5.3.14 读取电机堵转标志指令.....79
5.3.15 读取堵转电流指令.....80
5.3.16 读取电机使能状态指令.....80
5.3.17 读取电机到位标志指令.....80
5.3.18 读取系统参数指令.....81
5.3.19 读取驱动参数指令.....82
5.3.20 设置从机地址指令.....84
5.3.21 设置工作模式指令.....84
5.3.22 设置位置环 PID 参数指令.....85
5.3.23 设置速度/位置环最大力矩限制指令.....85
5.3.24 设置细分指令.....86
5.3.25 设置目标电流指令.....86

5.3.26 设置串口波特率指令.....86
5.3.27 设置 moubus 指令.....87
5.3.28 设置堵转保护功能指令.....87
5.3.29 设置堵转电流指令.....88
5.3.30 电机正反转 DIR 控制指令.....88
5.3.31 设置 EN 引脚有效电平指令.....89
5.3.32 设置按键锁定指令.....89
5.3.33 设置自动熄屏指令.....90
5.3.34 设置 IO 启动电平.....90
5.3.35 设置速度环 PID 参数指令.....90
5.4 运动类指令说明.....92
5.4.1 力矩模式控制指令.....92
5.4.2 速度模式控制指令.....92
5.4.3 绝对位置模式控制指令.....93
5.4.4 相对位置模式控制指令.....94
5.4.5 脉冲模式控制指令.....94
5.4.6 脉宽位置模式控制指令.....95
5.4.7 脉宽力矩模式控制指令.....96
5.4.8 脉宽速度模式控制指令.....97
5.4.9 将当前的位置角度清零指令.....98
5.4.10 解除堵转保护指令.....98
5.4.11 电机使能控制指令.....98
5.4.12 清除状态（堵转、刹车，失能）指令.....99
5.4.13 立即停止（刹车）指令.....99
5.4.14 开环速度模式控制指令.....100
5.4.15 开环绝对位置模式控制指令.....100
5.4.16 开环相对位置模式控制指令.....101
5.4.17 开环脉冲模式控制指令.....102
5.4.18 IO 启停模式控制指令.....102
5.5 回零类指令说明.....103
5.5.1 设置左限位原点位置指令.....103
5.5.2 设置有无限位回零指令.....103
5.5.3 触发回零指令.....104
5.5.4 强制中断并退出回零操作指令.....105
5.5.5 读取回零参数指令.....105
5.5.6 修改原点回零超时时间指令.....106
5.5.7 读取回零状态指令.....106
5.5.8 设置上电自动回零指令.....107
5.5.9 设置右限位原点位置指令.....107
5.5.10 设置左右限位开关状态.....108
6，上位机使用说明.....109
6.1 USB/TTL/RS485 通信上位机使用说明.....109
6.1.1 ATK_PDXX.exe.....109
固件升级操作流程.....113

6.1.2 ATK_XCOM.exe.....114
6.2 CAN 通信上位机使用说明.....115
6.2.1 Cangaroo.exe.....115
7，保修条款.....117
8，其他.....118

感谢您购买此产品，为了让您更充分的了解产品特性和操作方法，我们建议您在使用前先仔细阅读本手册。

注意事项(必读)

1， **电源输入电压：12V–32V，建议与电机匹配。使用本店 42 步进电机时，推荐电压 24V、电流 3A！**
2， **每个电机的参数都有差别，所以更换电机时，在安装完成并重新上电后，务必执行一次“恢复出厂设置”，可读取电机最新的参数。**
3， **请勿带电拔插电源线、信号线和 Type-C 线等，以免造成电气损坏！（属于不恰当使用行为，自负后果，已有损坏案例，切忌！切忌！切忌！）**
4， **电机校准时，请不要带负载！**
5， 最高转速 6000 RPM（每个电机参数不同，最高转速可能不同）。
6， 本驱动不一定适用所有的 42 电机(不同电机 PID 参数略有区别)。
7， 建议搭配本店 42 磁柱步进电机。
8， 在本店购买 42 步进电机和驱动板，会安装并校准好整套发货。
9， 支持串口 TTL / RS485 / CAN 通讯接口，请根据自己的需求选择对应版本。
10，根据场景分 PLC 和光耦版本，PLC 版本适合个人 DIY 使用，光耦版本适合工业使用。
11，当电机在运行过程中出现明显抖动时，建议先执行一次手动校准操作。
用于重新进行一次校准和识别电机关键参数：磁链、相电感、相电阻，以确保驱动器与电机的匹配精度与运行稳定性。

常见问题：

序号	问题	解决方法
1	欠压	供电电压不足， V+和 GND 需要 12-32V 进行供电
2	过压	供电电压超过安全范围，V+ 和 GND 输入电压不得高于 32V
3	未校准	编码器未校准，需要点击“校准”按钮完成编码器校准
4	异常	包括电机失能、回零失败、脉宽模式下脉宽长度不符等异常情况
5	堵转	电机在运行过程中被阻挡，转子无法正常转动
6	过载	负载过大，导致电机运行电流或转矩超过额定范围
7	超速	电机转速超过系统设定的最大安全速度

1，特性参数

PD42S1 步进电机 FOC 闭环驱动器是广州市星翼电子科技有限公司（正点原子）最新推出的一款高性能步进电机 FOC 闭环驱动器，适用于 3D 打印机，写字机，雕刻机，小型机械臂，PLC 控制和电子竞赛等场景。该产品主要有以下特点：

- 采用先进的 FOC 矢量控制算法，力矩，速度，位置三环控制
- 最大工作电流 3000mA，最高转速 6000RPM（承载能力强），最低转速 0.1RPM
- 电机高速性能稳定，运行平稳，超低震动，运行超静音
- 支持曲线加减速，电机启动和停止更平稳
- 板载工业级高精度线磁编码器，板载精密电流传感器
- 板载两个按键，板载 0.42 寸 54*96 高分辨率 LCD 屏幕，可实时显示系统信息
- 支持串口 TTL/RS485/CAN 通讯控制、支持脉冲控制、脉宽控制、回零控制等
- 支持正点原子自定义协议、支持 Modbus-RTU 协议
- 支持最多 255 个从机地址，可实现多机通讯
- 支持分组地址，电机设置为相同的分组地址即可实现多机同步通讯和控制
- 支持广播地址，所有总线上的从机可实现同步通讯和控制
- 支持内置串口 TTL/RS485/CAN 总线模块和接口，根据需求选不同版本
- 支持绝对位置模式、相对位置模式、速度模式、力矩模式、脉冲模式、脉宽模式、IO 启停模式、开环控制（速度 0~400RPM）等
- 支持脉冲模式下，最高输入脉冲频率 160KHz+，最大转速 6000RPM
- 支持单圈回零、就近回零和多圈回零，支持无限位或有限位回零
- 支持上电自动回零设置、支持设置零点，支持强制中断并退出回零操作等
- 支持堵转保护功能，支持电机立即停止功能、支持当前位置清零
- 支持 1～256 任意细分，支持细分插补功能，电机运行更加平滑和静音
- 支持读取电机实时速度、角度位置、总线电压、相电流、相电阻、相电感、磁链等
- 支持读取电流环/速度环/位置环 PID 参数、输入累计脉冲数、到位标志、运行状态
- 支持编码器自校准、自动熄屏、按键锁定、一键恢复出厂、一键复位重启等功能
- 可支持共阳/共阴信号输入，可支持 PLC 的 0-24V 信号输入，可选光耦隔离输入
- 支持设置位置环 PID 参数、支持设置速度/位置环最大力矩限制
- 支持参数保存指令，就算掉电，配置参数依然保存
- 提供支持正点原子自定义协议和 Modbus-RTU 协议的上位机等
- 提供 STM32 等使用例程，例程注释非常清晰

PD42S1 步进电机 FOC 闭环驱动器详细规格参数如表 1.1 所示：

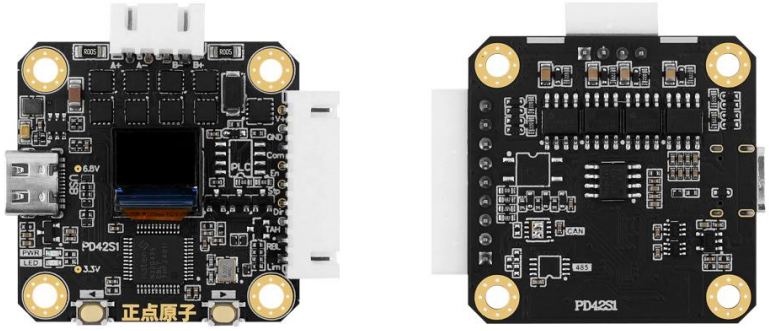
产品参数	
产品型号	ATK-PD42S1
主控芯片	国产高性能 32 位 ARM 微处理器
驱动电路	8 个高品质 MOS 管组成双 H 桥
传感器	工业级高精度线磁编码器、精密电流传感器
工作电压	12～32V
工作电流	0～3000mA
脉冲信号输入	0V～24V (PLC 版支持共阳/共阴输入) ，0V～5V（光耦版本仅支持共阳）
脉冲信号频率	最高 160KHz+
闭环反馈频率	力矩环 20KHz
	速度环 20KHz
	位置环 20KHz
最高转速	6000RPM，承载能力强
细分支持	1～256 任意细分，支持细分插补功能
控制精度	小于 0.08°
通信方式	支持串口 TTL/RS485/CAN 接口，可根据需求选择版本
通信协议	正点原子自定义协议、Modbus-RTU(CAN 版本是不支持的)
控制运动方式	脉冲控制、支持串口 TTL/RS485/CAN 总线控制
运动模式	绝对位置模式、相对位置模式、速度模式、力矩模式、脉冲模式、脉宽位置模式、脉宽力矩模式、脉宽速度模式、回零模式、IO 启停模式、开环模式控制
静音/震动	低速超静音、超低震动，高速运行平稳，不抖动，可急停
保护措施	支持堵转保护、欠压、过压警告、过载保护、电源防反接等功能

表 1.1 PD42S1 步进电机闭环驱动器详细规格参数

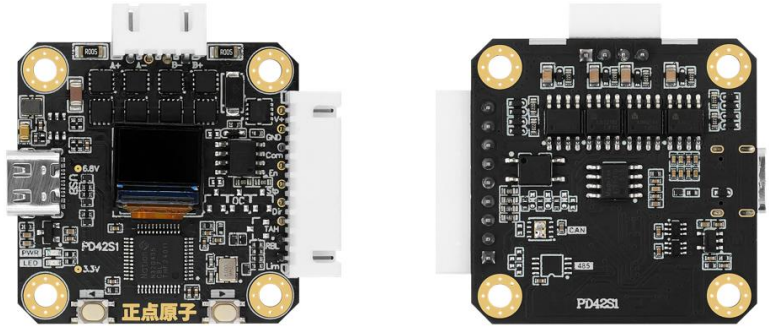
2，快速入门

PD42S1 步进电机 FOC 闭环驱动器根据通讯接口（TTL、RS485、CAN）和应用场景（PLC、光耦），分为 6 个版本，即：TTL-PLC 版本、TTL-光耦版本、RS485-PLC 版本、RS485-光耦版本、CAN-PLC 版本、CAN-光耦版本。PLC 版本适合个人 DIY 使用，光耦版本的 EN、STP、DIR 引脚都有光耦进行隔离保护适合工业使用。6 个版本的硬件实物图如下：

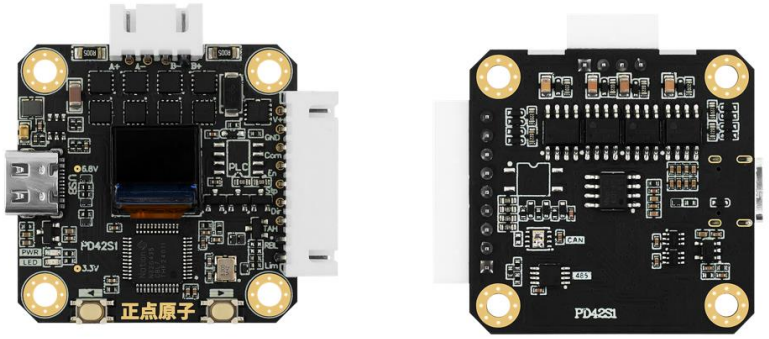
TTL-PLC版本



TTL-光耦版本

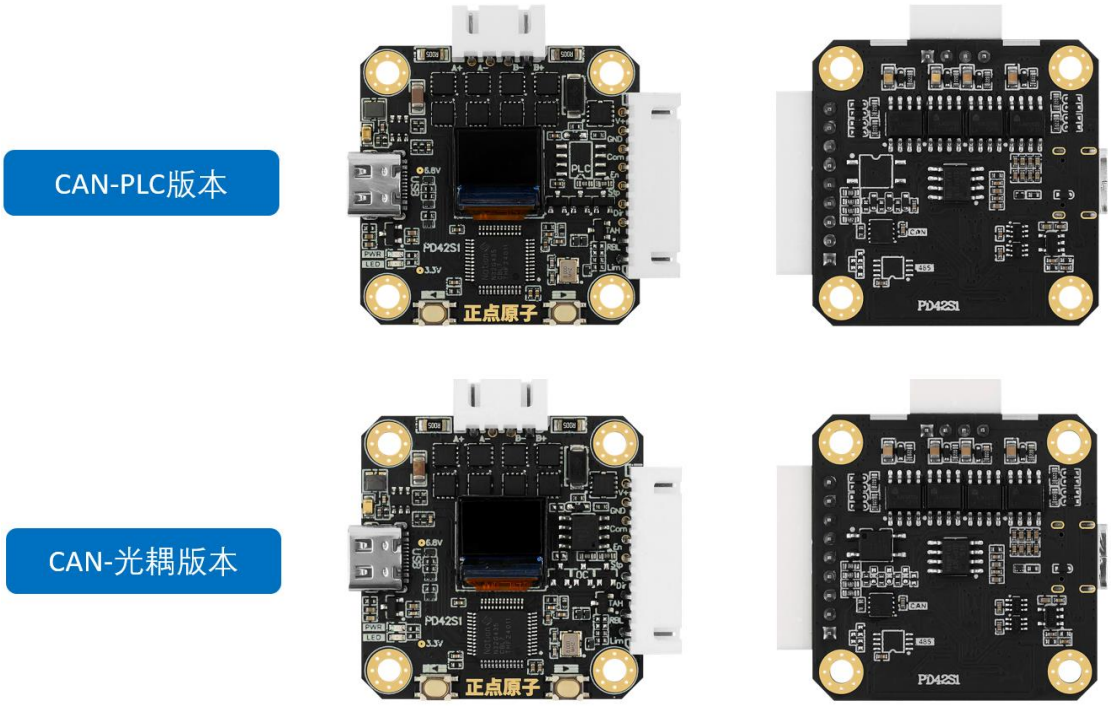


485-PLC版本



485-光耦版本





这么多版本分不清？我们发货时会在座子上贴上贴纸标明，比如：TTL-PLC 版本的贴纸（在对应格子打勾），如图 2.1 所示：

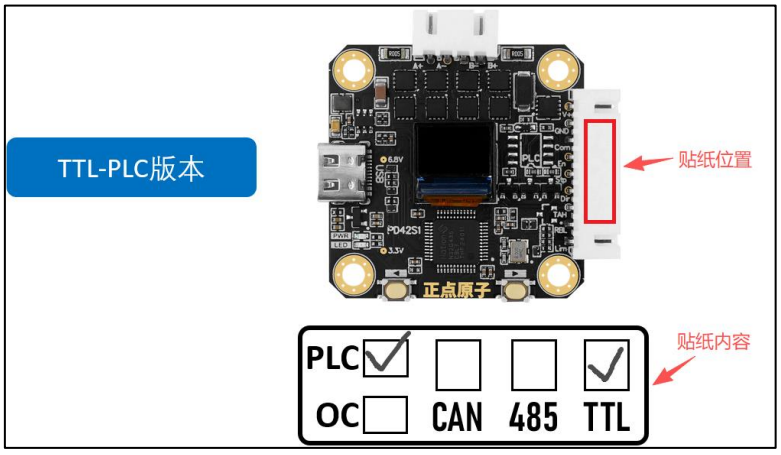


图 2.1 版本贴纸

贴纸掉了怎么区分？其实我们还可以通过观察驱动板正反面的芯片焊接情况，来判断是什么版本的，方法如图 2.2 所示：

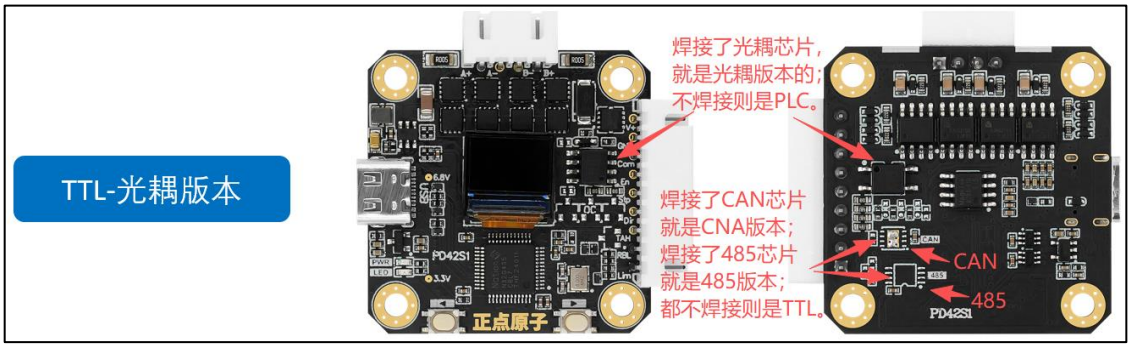


图 2.2 版本判断方法

本例中，两个光耦芯片焊接了，CAN 和 485 芯片都没有焊接，所以就是 TTL-光耦版本。

2.1 硬件说明

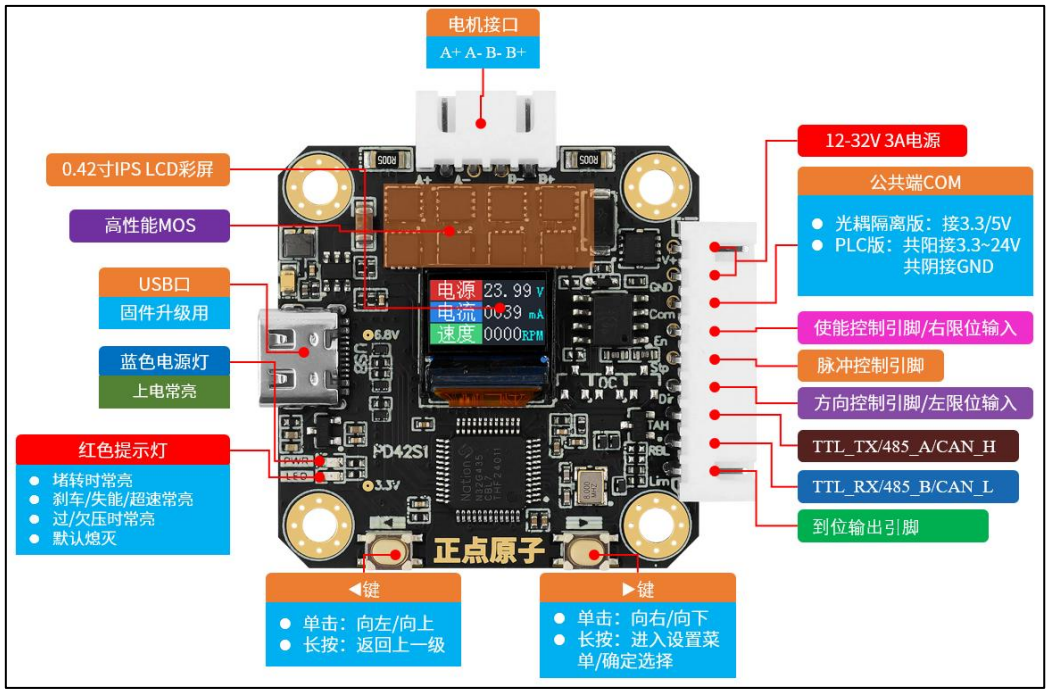


图 2.1.1 PD42S1 硬件整体了解

通过图 2.1.1，我们可以比较清楚的了解 PD42S1 硬件功能情况。下面我们再补充一下①驱动器通讯和控制接口的说明。①接口从上到下分别是：

- V+：电源正极，输入电压 12～32V，最大输入电流 3A
- GND：电源负极
- COM：公共端输入引脚，不同版本有较大区别（请注意！）：
  1) PLC 版本：支持共阳 / 共阴输入，输入电压范围 0V～24V。
  2) 光耦版本：仅支持共阳输入，输入电压范围 3.3V～5V。
- EN（使能/右限位输入）：
  1) 使能功能：输入 0V ~ COM 电压，控制电机启停（脉冲模式下用于使能电机运行）。
  2) 限位功能：可连接右限位开关，用于回零与防止电机越界，保证运动安全，注意低电平触发。
- STP：脉冲信号输入引脚，0V～COM 所接的电压（脉冲模式，脉冲输入）
- DIR（方向/左限位/IO 启停）：
  1) 方向控制：输入 0V～COM 电压，控制电机正反转方向（脉冲模式）。
  2) 左限位输入
  可与 EN 接入的右限位开关配合使用，实现左右双限位保护，注意低电平触发。适用于回零定位与安全边界限制，防止电机超行程。
  3) IO 启停模式
  可由外部 MCU 的一个 GPIO 引脚实现“启停控制”。优势：只需一个 IO，通过高/低电平即可启动或停止电机，接线更简单，控制更快速。
- TX/485_A/CAN_H：不同版本对应不同的通信引脚
- RX/485_B/CAN_L：不同版本对应不同的通信引脚
- Lim：到位输出功能引脚（输入/输出电压范围：0~3.3V）

到位输出：当电机位置到达目标位置时，该引脚输出低电平；未到达目标位置时，输出高电平。

注意：如果（EN/STP/DIR）信号的高电平为 3.3V，COM 必须接 3.3V。
如果（EN/STP/DIR）信号的高电平为 5.0V，COM 必须接 5.0V。
以此类推……

2.2 接线方法

在接线之前，请先确认自己的闭环步进电机驱动器是哪个版本的，然后找到对应的接线方法进行连接。

2.2.1 电机接线

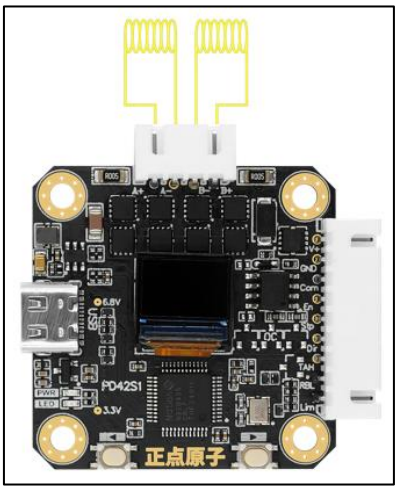


图 2.2.1.1 电机接线

注意：电机接口的线序，从左到右分别是：A+ A- B- B+。A+ A- 接电机的一相线，B+ B- 接电机的另一相线。

2.2.2 USB 通讯控制接线

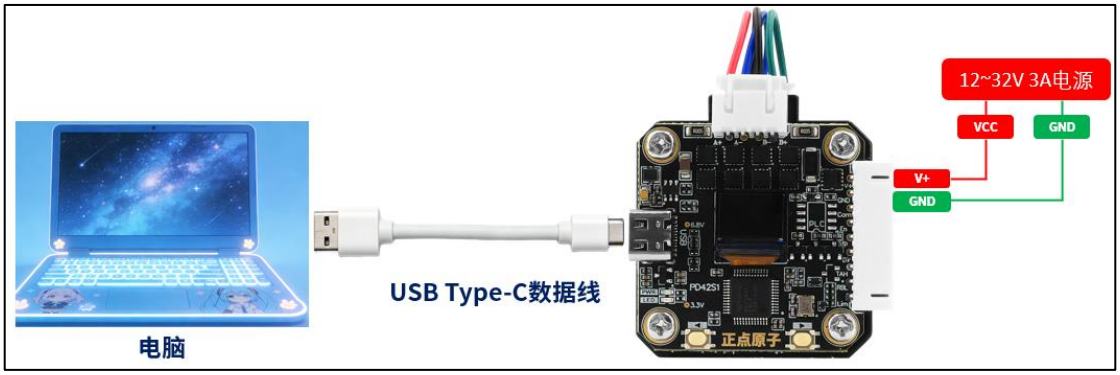


图 2.2.2.1 USB 通讯控制接线示意图

如上图所示，将设备通过 **USB Type-C 数据线** 连接至电脑 USB 接口，即可建立 USB 通讯连接，实现与上位机软件的数据交互与控制。

具体连接方式如下：

- 使用 USB Type-C 数据线，一端连接电脑 USB 接口；
- 另一端连接控制板上的 USB 接口；

- 外部电源通过 V+ 和 GND 端子接入（12V～32V，建议电流≥3A），为系统正常运行提供电力支持。完成以上连接后，设备即可通过 USB 与电脑进行通信。

注意：本产品所有版本（TTL/RS485/CAN）均支持 USB 通讯控制功能，均可通过 USB 接口连接上位机软件进行控制和调试，无需额外转换设备，使用方便快捷。**需要注意的是，USB 通信方式仅支持一对一连接模式，即一台设备对应一台电脑进行通信，不支持多设备并联通信。**

2.2.3 串口 TTL 通讯控制接线

串口 TTL 单机通讯控制接线示意图：

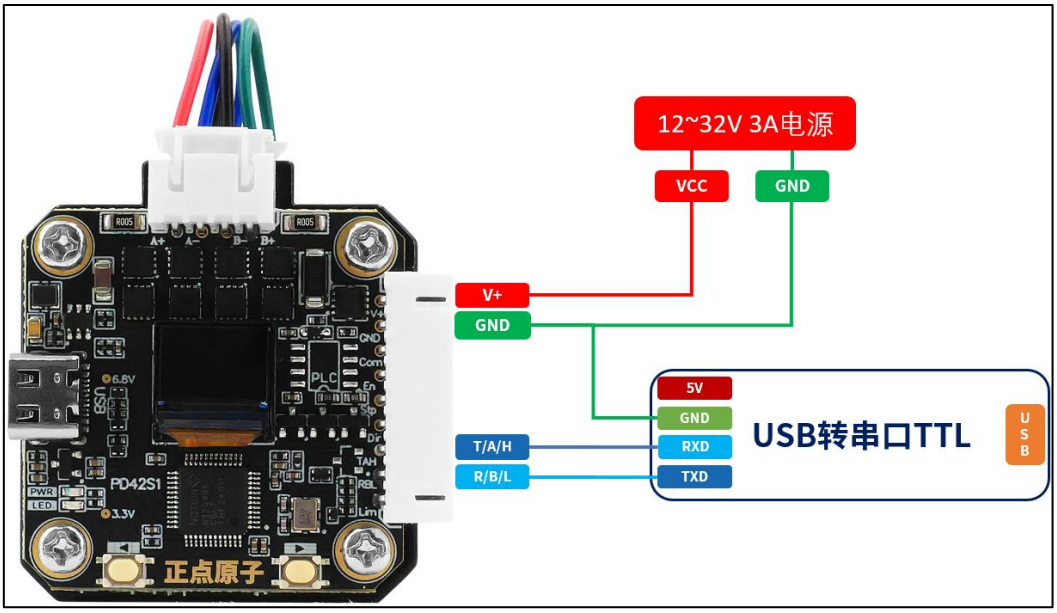


图 2.2.3.1 串口 TTL 通讯控制接线示意图

注意：PDxxS1 驱动器要和 USB 转串口 TTL 设备共地，以及 RXD 和 TXD 是交叉连接的。

下面展示 PDxxS1 驱动器与 USB 转串口 TTL 设备实物连接示意图，以正点原子 ATK-MO340P 模块为例。

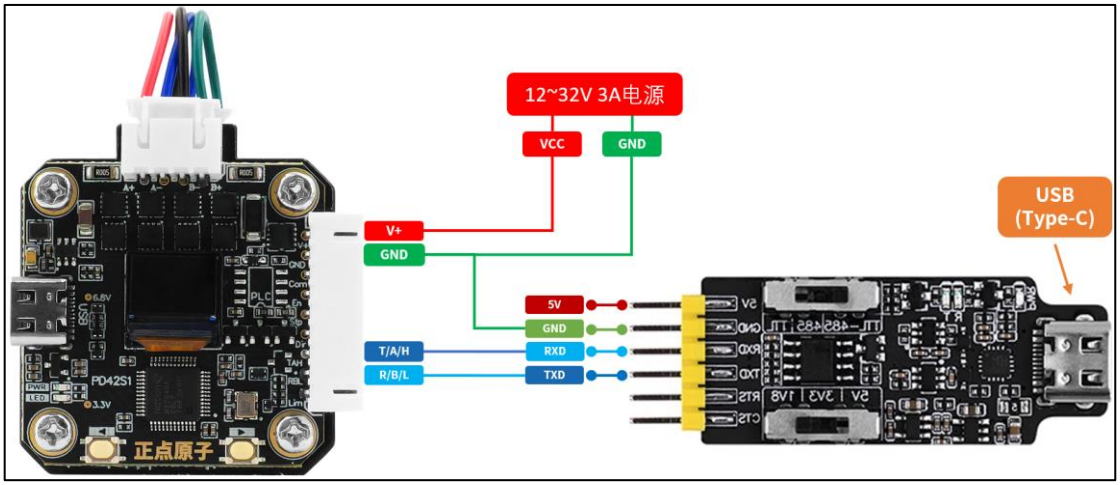


图 2.2.3.2 PDxxS1 驱动器与 ATK-MO340P 模块接线示意图

串口 TTL 多机通讯控制接线示意图：

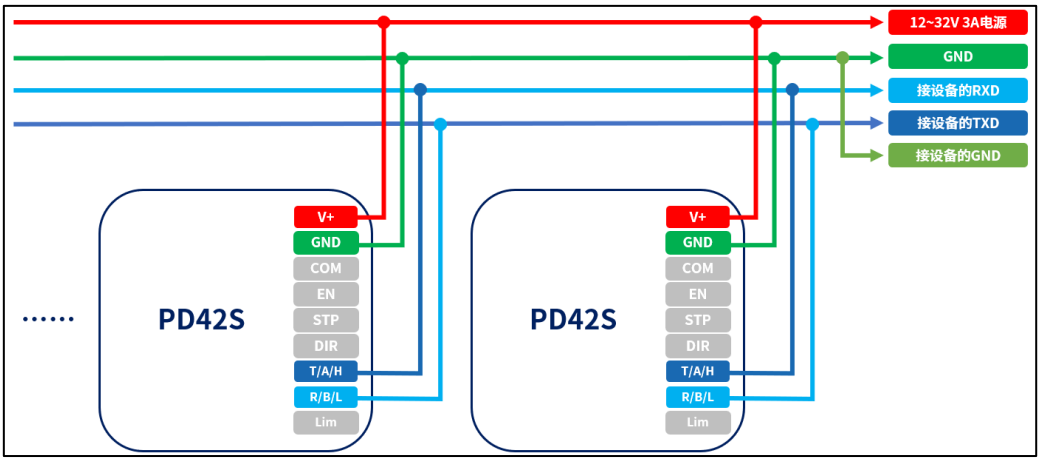


图 2.2.3.3 串口 TTL 多机通讯控制接线示意图

需要注意的是：在使用串口 TTL 进行多机通信时，当多个设备同时连接到同一串口总线上，通常只能保证发送功能正常，而接收数据可能出现异常或不稳定现象。这是由于 TTL 串口为点对点通信方式，不具备总线仲裁和冲突避免机制，多个设备并联时容易产生信号干扰和数据冲突。

因此，在需要实现多设备通信，且要求通信过程稳定、支持双向正常收发的应用场景中，建议选用具备总线结构和抗干扰能力的通信方式，例如 **RS485** 或 **CAN 总线**，以确保系统的可靠性和通信稳定性。

2.2.4 串口 RS485 通讯控制接线

串口 RS485 单机通讯控制接线示意图：

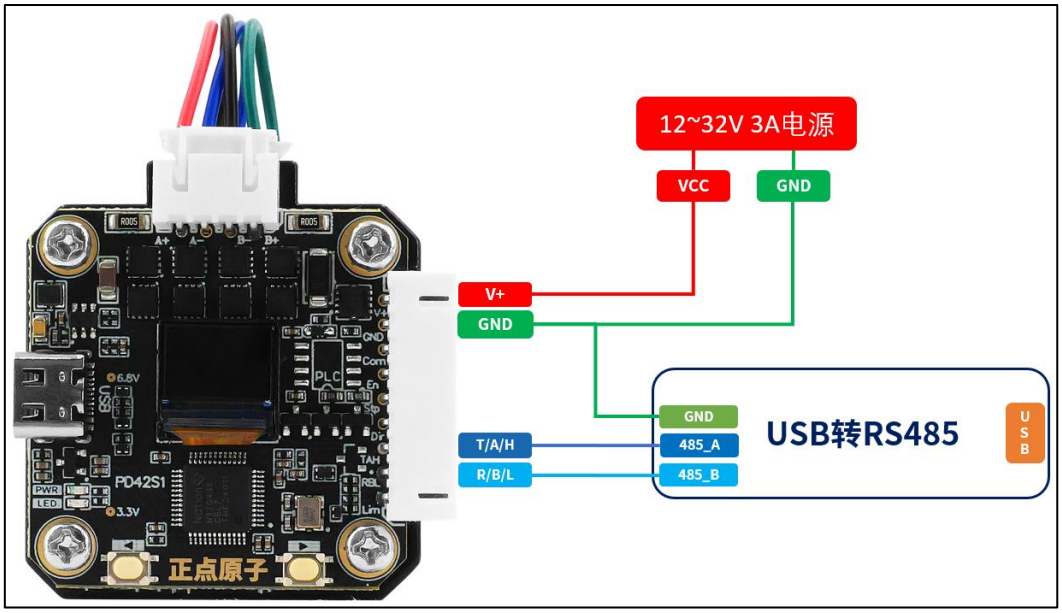


图 2.2.4.1 串口 RS485 通讯控制接线示意图

注意：PDxxS1 驱动器要和 USB 转 RS485 设备共地，两设备的 485_A 相互连接，485_B 也相互连接。

下面展示 PDxxS1 驱动器与 USB 转串口 RS485 设备实物连接示意图，以正点原子 ATK-MO340P 模块为例。

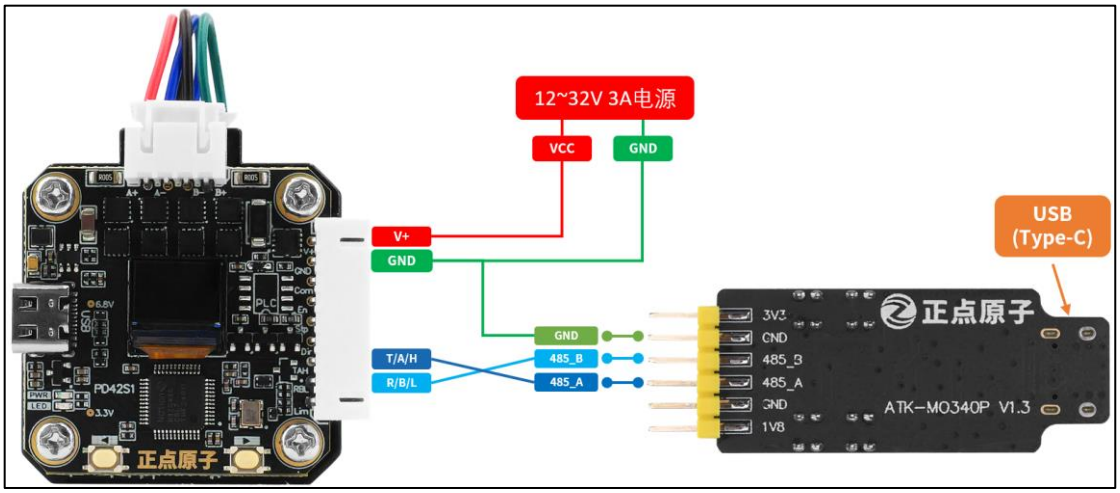


图 2.2.4.2 PDxxS1 驱动器与 ATK-MO340P 模块接线示意图

串口 RS485 多机通讯控制接线示意图：

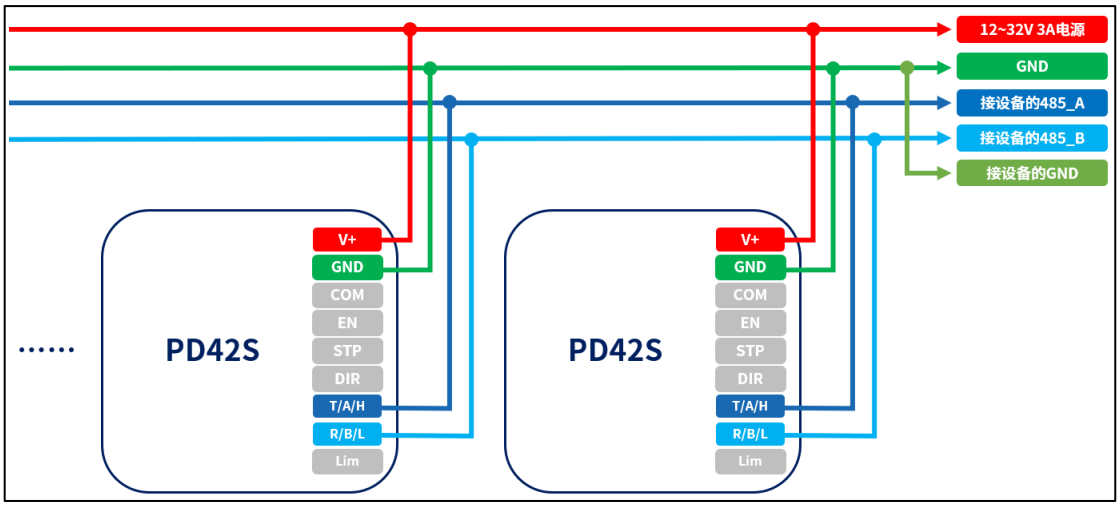


图 2.2.4.3 串口 RS485 多机通讯控制接线示意图

注意：PDxxS1 驱动板上已经集成 RS485 终端电阻 120Ω。多机通信时只需保留两个 120Ω 的终端电阻！

2.2.5 CAN 通讯控制接线

CAN 单机通讯控制接线示意图：

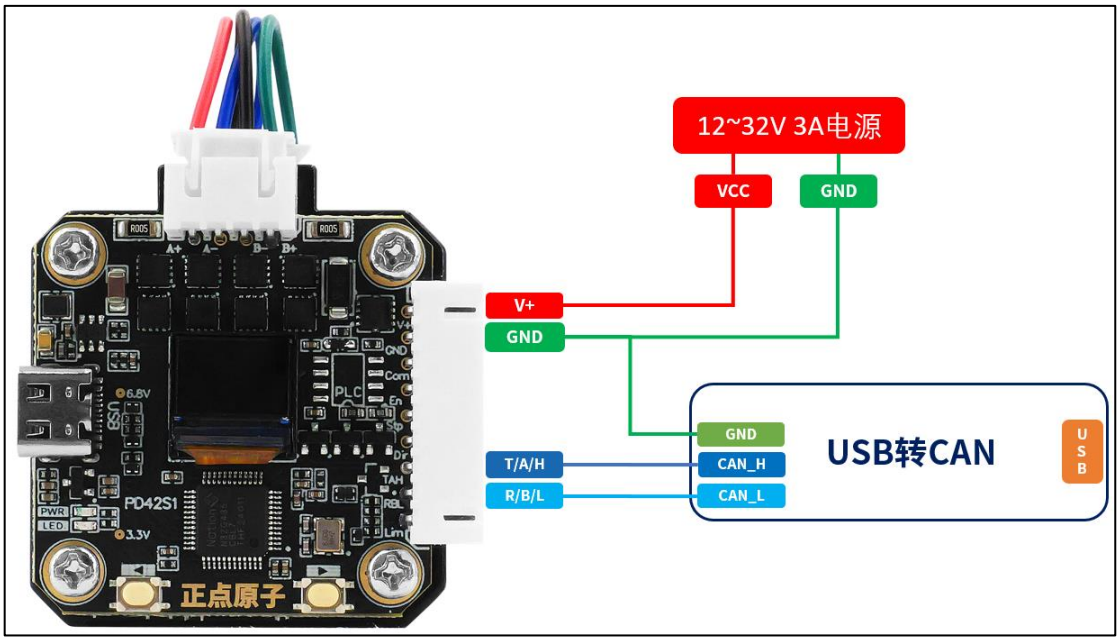


图 2.2.5.1 CAN 通讯控制接线示意图

注意：PDxxS1 驱动器要和 USB 转 CAN 设备共地，两设备的 CAN_H 相互连接，CAN_L 也相互连接。

CAN 多机通讯控制接线示意图：

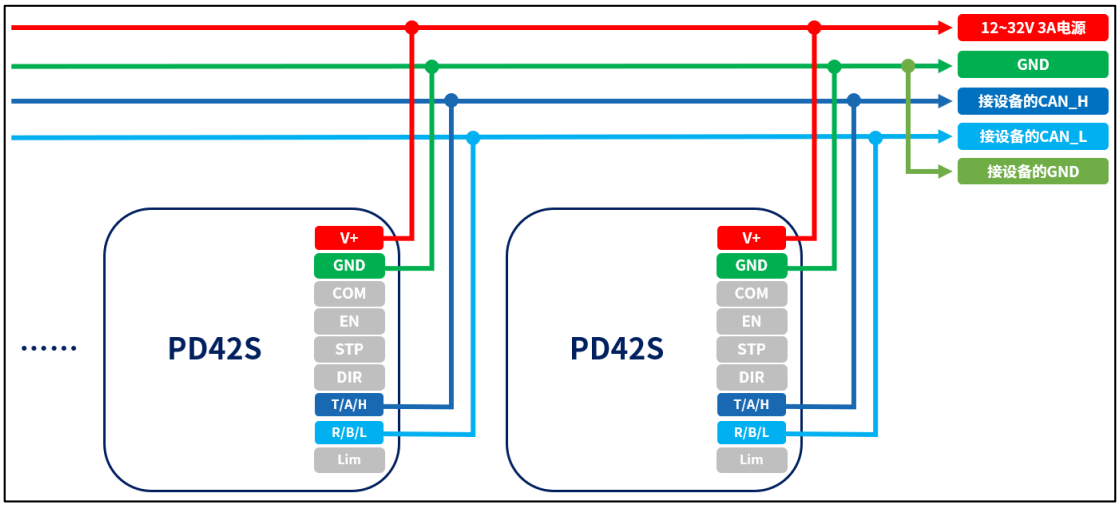


图 2.2.5.2 CAN 多机通讯控制接线示意图

注意：PDxxS1 驱动板上已经集成 CAN 终端电阻 120Ω。多机通信时只需保留两个 120Ω 的终端电阻！

2.2.6 脉冲模式控制接线

脉冲模式控制接线示意图：

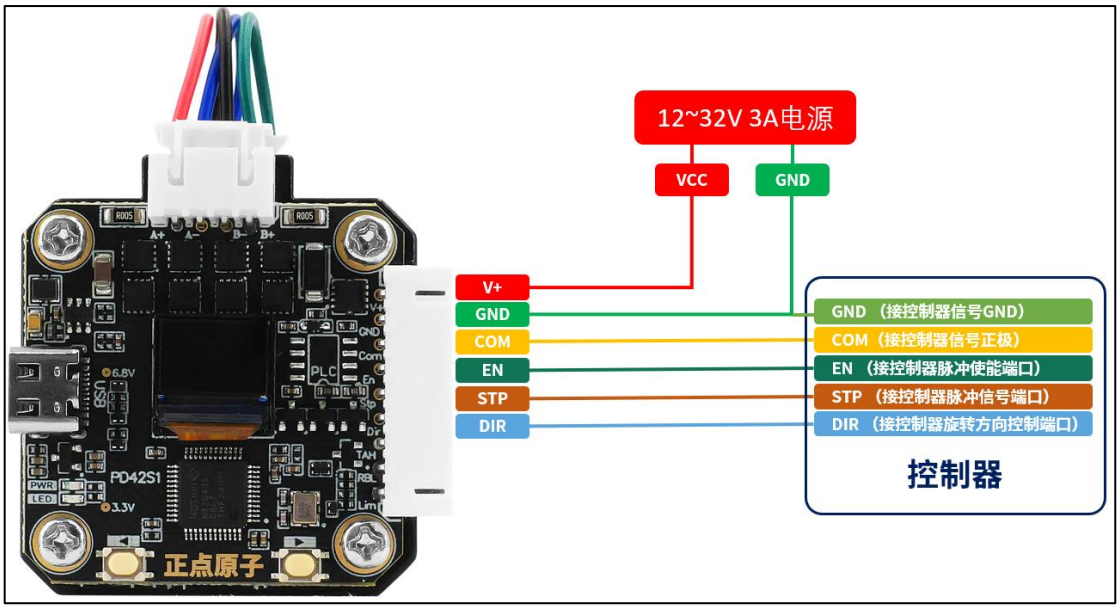


图 2.2.6.1 脉冲模式控制接线示意图

COM：公共端输入引脚，不同版本有较大区别（请注意！）：

1）PLC 版本：支持共阳 / 共阴输入

共阳：COM 接电压范围：3.3V～24V。

共阴：COM 接 GND

2）光耦版本：仅支持共阳输入，输入电压范围 3.3V～5V。

注意：如果（EN/STP/DIR）信号的高电平为 3.3V，COM 必须接 3.3V。

如果（EN/STP/DIR）信号的高电平为 5.0V，COM 必须接 5.0V。

以此类推……

2.2.7 左右限位开关接线

左限位回零模式的限位开关接线示意图**（注意：低电平触发，且 Dir 输入输出电压范围：0~COM V）**：

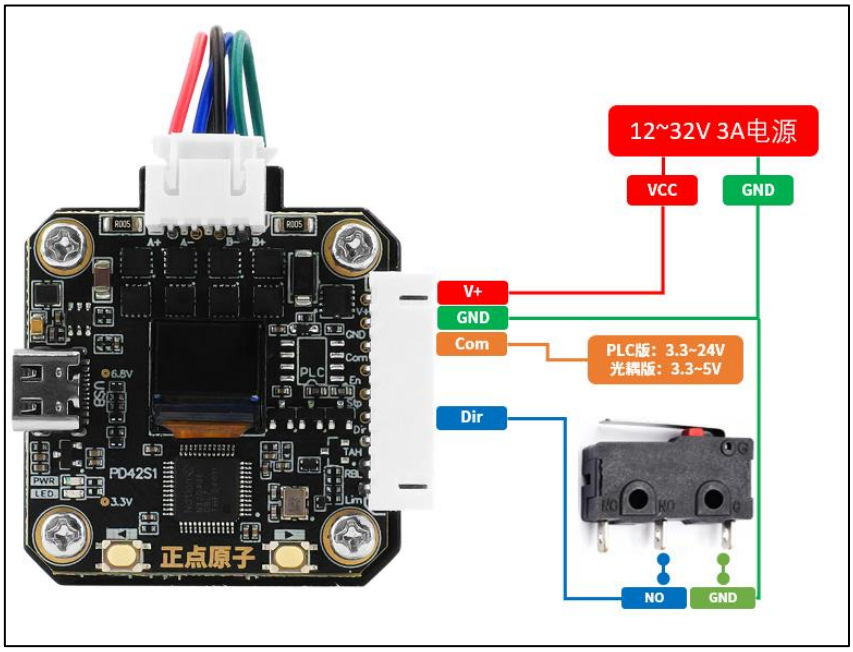


图 2.2.7.1 左限位开关接线示意图

右限位回零模式的限位开关接线示意图**（注意：低电平触发，且 EN 输入输出电压范围：0~COM V）**：

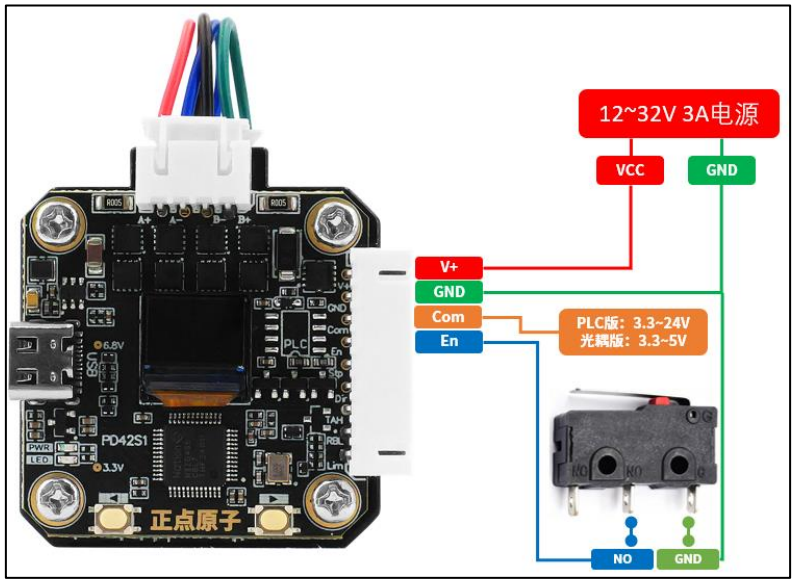


图 2.2.7.2 右限位开关接线示意图

2.2.8 IO 启停接线

IO 启停模式接线示意图**（注意：Dir 输入电压范围：0~COM V）**：

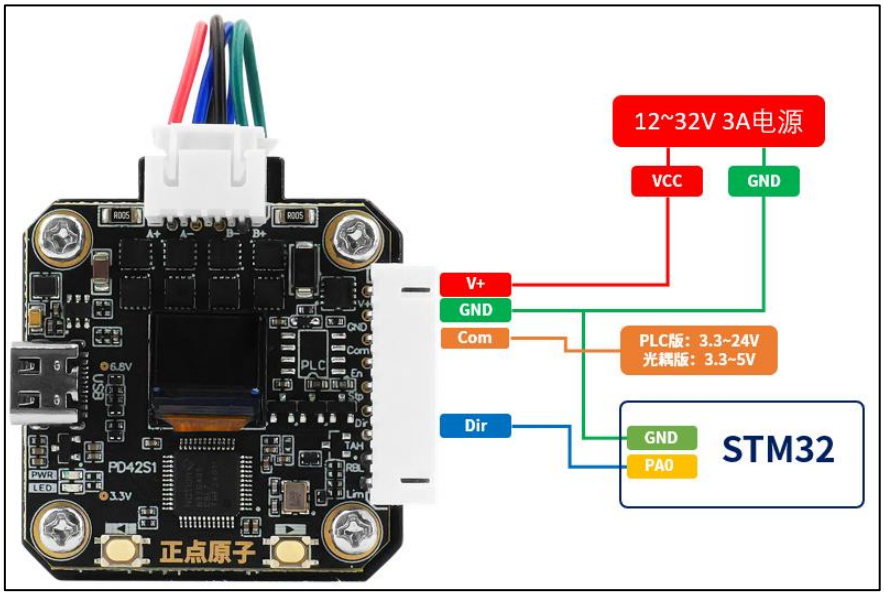


图 2.2.8.1 IO 启停接线示意图

说明：IO 启停模式是通过高低电平控制电机是否旋转的，可通过发送指令设置高电平或者低电平旋转。这里用 STM32 举例。

2.3 运动模式简介

本小节先给大家简单介绍一下 PD42S1 的运动模式，让大家先有个大概的了解，方便找到适合自己项目使用的模式。

①力矩模式

通过 FOC（磁场定向控制）算法实现电机力矩输出控制。 在此模式下，控制量为电流（Iq 分量），最大可达 3000 mA。 需要注意的是： 由于采用 FOC 闭环控制，系统并非简单恒流输出，而是在有负载作用时，才会自动提升电流以满足目标力矩要求。此模式适用于力控实验、压力装配、张力控制等应用场景。

②速度模式

通过设定目标转速，系统实时调节电流与电压矢量，实现平滑的速度闭环控制。速度范围：0.1 RPM ~ 6000 RPM。支持梯形加减速，可有效避免启动抖动和突变。适用于恒速驱动、输送、旋转平台等应用。

③绝对位置模式

以编码器的绝对角度为控制目标进行位置控制。单圈分辨率为 51200，例如：51200 表示 1 圈，102400 表示 2 圈，以此类推。注意：由于使用单圈编码器，掉电后多圈累计位置会丢失，仅保留单圈位置。支持梯形加减速，实现平稳定位与精确控制。

④相对位置模式

以当前电机位置为基准，通过设定偏移量实现相对运动。每圈对应 51200。同样支持梯形加减速，可实现柔性移动、往复运动等控制策略。

⑤IO 启停模式

通过外部 IO 信号实现启停控制。系统可掉电保存运行参数（包括速度、加速度、方向等），上电后自动恢复运行状态。适用于简化系统通信的场合，例如按钮启停控制、自动化执行流程。

⑥脉冲模式

由外部输入脉冲信号控制电机运行：STEP 控制步进脉冲，DIR 控制方向，EN 控制使能状态。该模式兼容标准运动控制卡。支持脉冲插补控制，实现高速、高精度同步运动。

⑦脉宽位置模式

通过输入 PWM 信号的高电平宽度来控制电机位置。可配置最短与最长脉宽对应的位置范围，系统根据输入脉宽按比例输出目标位置。常用于模拟舵机控制、外部比例输入等应用。

⑧脉宽速度模式

PWM 高电平脉宽对应电机目标速度。用户可设置脉宽与速度的线性映射关系，实现模拟量式速度控制。 适用于简易调速旋钮、比例控制接口等系统。

⑨脉宽力矩模式

利用 PWM 信号脉宽控制输出力矩。可设置最小/最大脉宽及对应的力矩范围，系统按比例输出目标力矩。适用于力控实验、张力控制等对外部模拟信号兼容的场合。

⑩回零模式

支持多种回零策略：

1）有限位 / 无限位回零

2）单圈 / 多圈回零

3）就近回零

系统可通过限位信号或电流阈值检测判断零点，实现快速、可靠的回零操作。回零完成后，可进入任意其他模式运行。

⑪开环速度模式

在无编码器反馈的情况下，通过控制电压矢量实现转速控制。系统根据给定速度计算电角频率并输出对应矢量，控制简单直观，速度范围：0~400RPM，支持梯形加减速。适合对速度精度要求不高的低成本应用场景。

⑫开环绝对位置模式

在开环条件下，通过设定目标位置实现绝对角度定位。控制系统根据步进角度累计实现开环定位。适用于无需闭环反馈但需固定角度输出的场合。

⑬开环相对位置模式

以当前电机角度为参考点，执行相对偏移控制。 适用于低速、轻载、简易定位等非闭环应用。

⑭开环脉冲模式

由外部 STEP/DIR/EN 信号直接控制电机运行，无需编码器反馈。响应快速、控制简洁，适用于成本敏感型或传统步进控制系统。

2.4 产品尺寸

驱动板（不带注塑外壳）尺寸图：

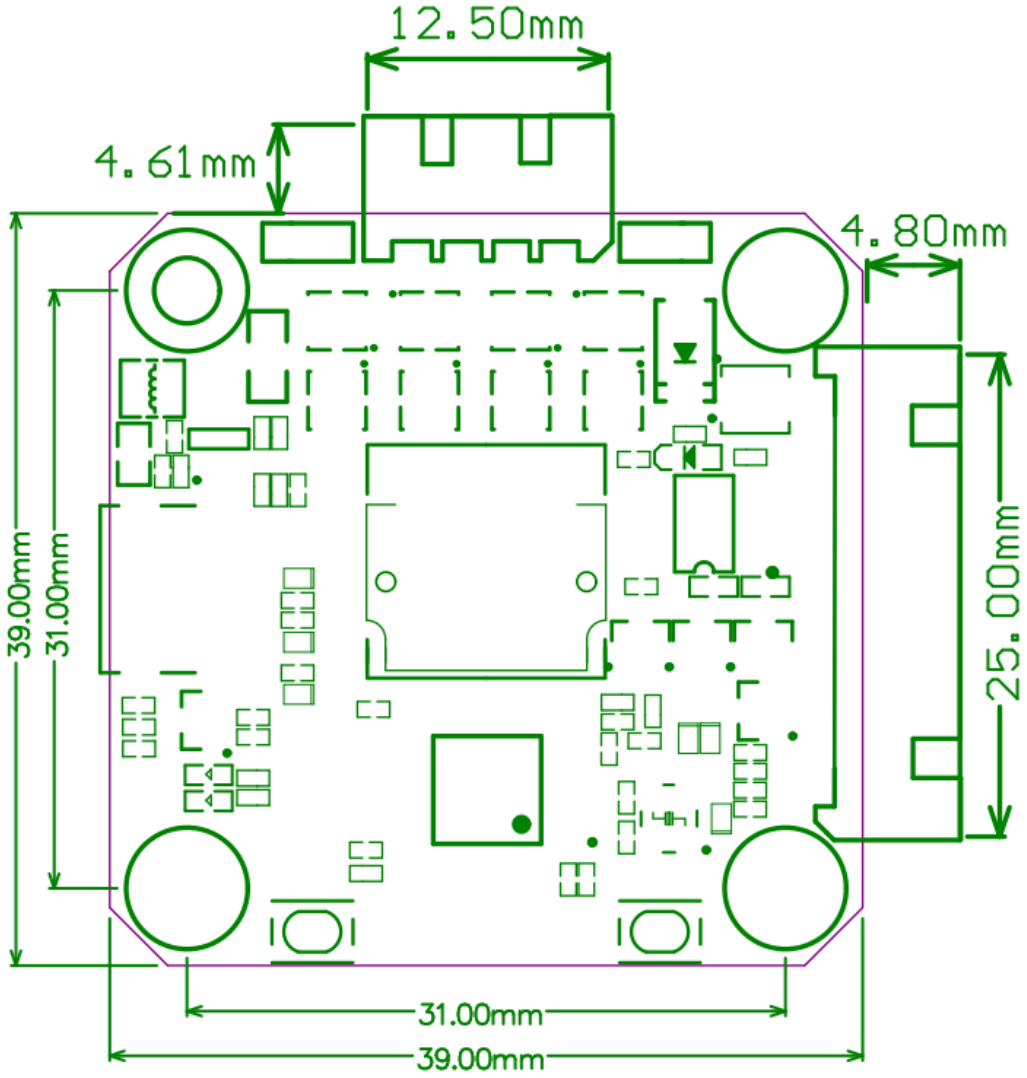


图 2.5.1 驱动板（不带注塑外壳）尺寸图

整机带注塑外壳俯视尺寸图：

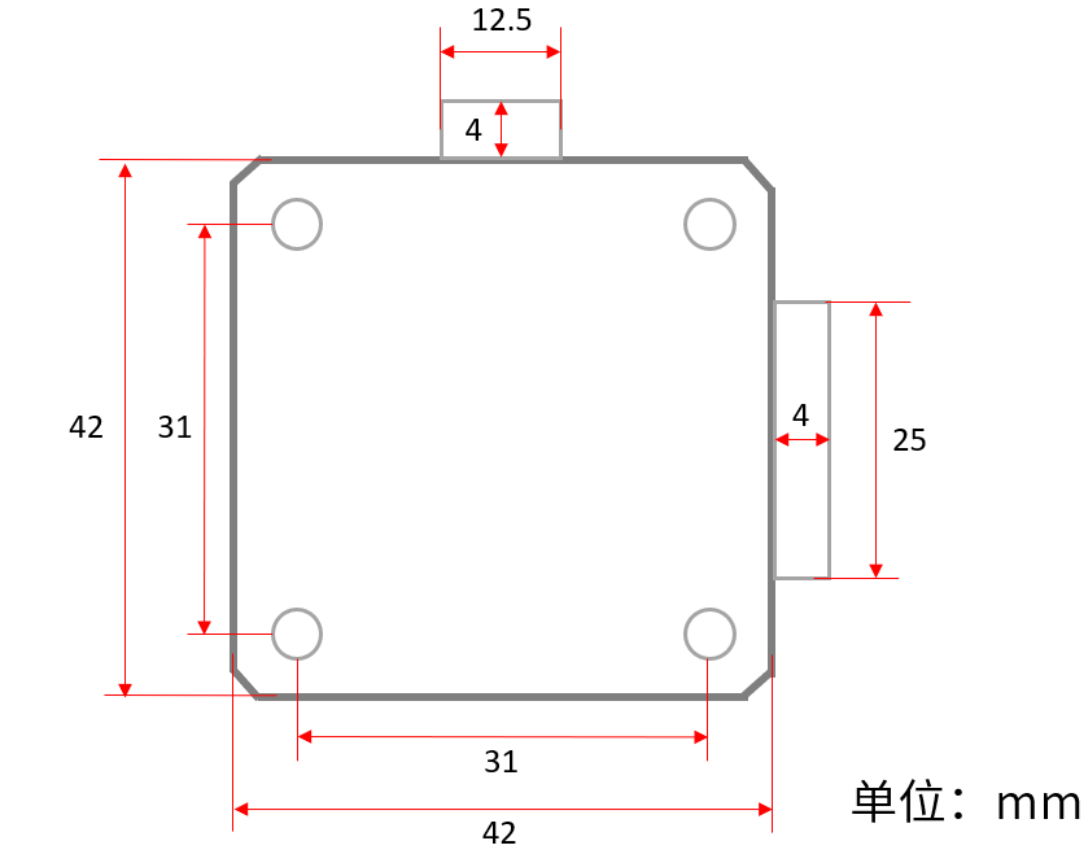


图 2.5.2 整机带注塑外壳俯视尺寸图

整机带注塑外壳右视尺寸图：

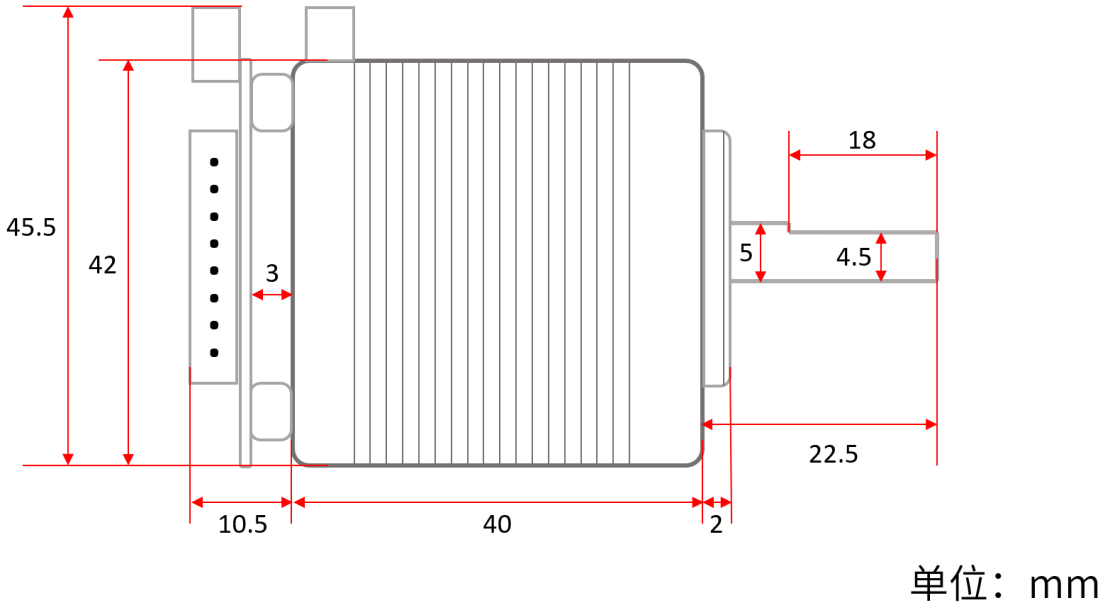


图 2.5.3 整机带注塑外壳右视尺寸图

2.5 产品安装

安装前需准备的硬件清单如图 2.5.1 所示：

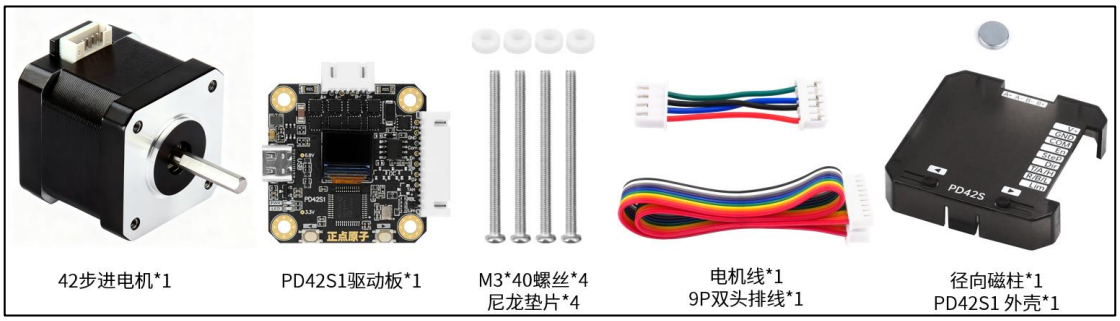


图 2.5.1 硬件清单图

下面介绍一下安装步骤：

1，安装磁柱：磁柱默认都是已经安装在 42 步进电机的磁柱孔内，安装前可以检查一下是否掉了或者歪了？一般不容易掉的，如果掉了歪了，请按照图 2.5.2 正确安装好。



图 2.5.2 安装磁柱

注意：

1）如果电机需要剧烈运动的，可以考虑用 3M 胶水、超能胶水等进行固定，尽量不选择 502 等低粘稠状的胶水，此类胶水容易渗透到电机内部，造成电机的损坏。

2）建议选用本店的 42 磁柱步进电机，PID 参数都是调试得比较好用的。

2，放入垫片：拧出螺丝，放入垫片，如图 2.5.3 所示：



图 2.5.3 放入垫片

3，安装驱动板：平稳放好模块，打上螺丝，安装好电机线，如图 2.5.4 所示：

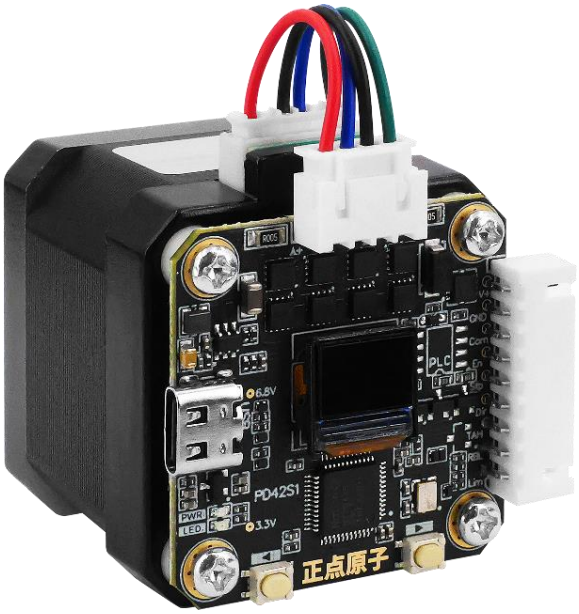


图 2.5.4 安装驱动板

注意：电机线是防呆接口的，但是如果自己在其它地方购买了 42 步进电机的话，注意看一下线序是否和我们的接口兼容，线序参考 2.2.1 小节。

4，安装外壳：检查没问题就可以安装外壳，如图 2.5.5 所示：

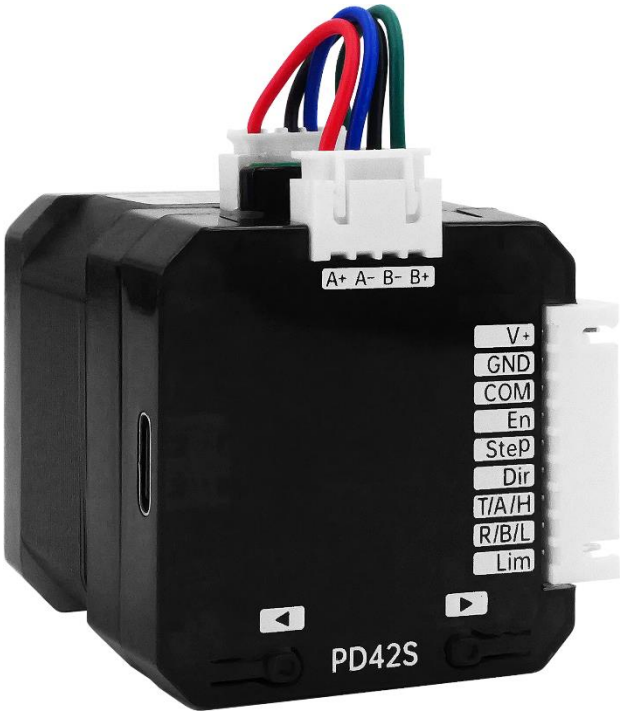


图 2.5.5 安装外壳

3，菜单说明

3.1 参数显示说明

参数显示共有 3 个界面，可通过短按◀键和▶键进行切换，分别是基本参数界面、电机参数界面和工作模式参数界面。

1，**基本参数界面**显示的参数包括：电源、电流、速度，如下图 3.1.1 所示：



图 3.1.1 基本参数界面

① 电源，即电源电压，单位：V。

② 电流，即相电流，代表 FOC 中的 Iq 分量，单位：mA。

③ 速度，即电机旋转速度，单位：RPM。

2，**电机参数界面**显示的参数包括：磁链、电阻、电感，如下图 3.1.2 所示：



图 3.1.2 电机参数界面

① 磁链，即电机磁链，单位：mWb。

② 电阻，即电机相电阻，单位：Ω。

③ 电感，即电机相电感，单位：mH。

3，**工作模式参数界面**根据不同运行模式显示的参数也有所不同，例如：**在脉冲模式下**，显示的参数包括：脉冲和细分，如下图 3.1.3 所示：



图 3.1.3 脉冲模式参数界面

① 脉冲，即累计输入脉冲数。

② 细分，即电机细分。

在通信位置模式下，界面显示的参数包括目标位置与实际位置。由于屏幕显示空间有限，位置以“r（圈）”为单位表示，每 51200 计数对应 1 圈，图略。

① 目标，即目标位置。

② 实际，即实际位置。

在通信速度模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标速度。

② 实际，即实际速度。

在通信力矩模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标电流。

② 实际，即实际电流。

在脉宽位置模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标位置。

② 实际，即实际位置。

在脉宽速度模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标速度。

② 实际，即实际速度。

在脉宽力矩模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标电流。

② 实际，即实际电流。

在回零模式下，显示的参数包括：零点和实际，图略。

① 零点，即零点位置。

② 实际，即实际位置。

在开环速度模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标速度。

② 实际，即实际速度。

在开环位置模式下，界面显示的参数包括目标位置与实际位置。由于屏幕显示空间有限，位置以“r（圈）”为单位表示，每 51200 计数对应 1 圈，图略。

① 目标，即目标位置。

② 实际，即实际位置。

在开环脉冲模式下，显示的参数包括：脉冲和细分，图略。

① 脉冲，即累计输入脉冲数。

② 细分，即电机细分。

在 IO 启停模式下，显示的参数包括：目标和实际，图略。

① 目标，即目标速度。

② 实际，即实际速度。

注意：在位置控制相关模式下，每 51200 个计数对应电机旋转一整圈。数值的正负符号用于区分旋转方向，正值表示正向旋转，负值表示反向旋转。

3.2 菜单设置说明

在 3.1 小节任意的参数显示界面中，**可通过长按 ▶键进入菜单设置界面**，如图 3.2.1 所示：

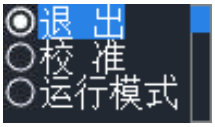


图 3.2.1 菜单设置界面

菜单设置界面从上到下包括：退出、校准、运行模式、使能设置、方向设置、细分设置、ID 地址、通信速率、MODBUS、屏幕亮度、自动熄屏、语言选择、恢复出厂、版本信息。

在菜单设置界面中，可通过短按 ◀键/▶键，上下切换菜单选项；可通过长按 ◀键，返回上一级；可通过长按 ▶键，进入下一级或者确定选择。

①退出：即退出菜单设置，回到参数显示界面。

②校准：即校准编码器，**注意校准编码器前，请确保电机是空载状态!!!**

③运行模式：即电机的工作模式，有 12 种工作模式，分别是：通信位置、通信速度、通信力矩、脉冲模式、脉宽位置、脉宽速度、脉宽力矩、回零模式、开环速度、开环位置、开环脉冲、IO 启停。

④使能设置：这里指的是设置 EN 引脚有效电平（仅用于脉冲模式），有三种情况：EN 引脚保持有效、EN 引脚高电平使能、EN 引脚低电平使能。

⑤方向设置：即设置电机旋转方向（仅用于脉冲模式），有两种情况：默认方向、翻转方向。

⑥细分设置：即设置电机细分，可设置：1、2、4、8、16、32、64、128、256，其它值请用指令设置。

⑦ID 地址：即设置电机 ID 地址，可设置：0～10，其它值请用指令设置。

⑧通信速率：即设置串口速率和 CAN 速率，串口速率可设置：4800、9600、115200（默认）、256000、921600，其它值请用指令设置。

⑨MODBUS：即设置是否使用 MODBUS 协议，可设置：失能（默认）、使能。失能即使用正点原子自定义通信协议，使能即使用 MODBUS-RTU 协议（CAN 版本不支持）。

⑩屏幕亮度：即设置 LCD 屏背光亮度，可设置：20%、40%、60%、80%、100%。

⑪自动熄屏：指按键无操作一定时间后自动熄屏，可设置：关闭、开启。

⑫语言选择：即系统文字语言，可设置：简体中文、English、繁体中文。

⑬恢复出厂：执行该操作后，系统将恢复出厂默认参数，编码器会自动重新校准，并重新识别电机参数。此功能通常用于更换电机时。操作前请确保电机处于空载状态，以避免校准误差或设备损坏。

⑭版本信息：即查看软硬件版本信息。

4，自定义协议指令使用说明

注意事项：

1. 协议使能方法

需要失能 modbus 协议，方可使用，下面两个方法效果一样：

- 方法 1：屏幕菜单→MODBUS→失能
- 方法 2：使用“设置 modbus 指令”，设置使用正点原子自定义协议

2. 协议通用性说明

本协议适用于以下三种通信接口版本的设备：

- 串口 TTL 版本
- RS485 版本
- CAN 版本

其中，协议的指令格式与数据内容在三种版本中保持一致，仅在物理层和部分收发规则上存在差异。

3. CAN 版本的特殊要求

在 CAN 总线上使用本协议时，需要注意以下几点：

- **仅支持扩展帧 (Extended Frame)**。
- **报文 ID 规范**：ID 必须符合 0x100x 格式，其中最后的 x 表示任意 16 进制数。例如：0x1001、0x100A、0x100F 均为合法 ID。
- 除上述限制外，协议内容（指令、数据格式、校验规则等）与 串口 TTL/RS485 版本保持完全一致。

4. 串口 TTL 与 RS485 默认通信参数

- 波特率 115200、
- 停止位 1、
- 数据位 8、
- 检验位 None

5. CAN 默认通信参数

- 波特率：500 Kbps

6. 从机地址

- 出厂默认从机地址为 01H
- 文档中示例均以 01H 作为从机地址。

功能码表：

指令类型	功能码	功能说明
系统指令(0x00 ~ 0x0F)	0x01	校准编码器
	0x02	复位重启
	0x03	恢复出厂设置
	0x04	参数保存
读参数指令 (0x20 ~ 0x3F)	0x20	读取软硬件版本
	0x21	读取电机磁链
	0x22	读取相电阻和相电感
	0x23	读取相电流
	0x24	读取总线电压
	0x25	读取电流环 PID 参数
	0x26	读取速度环 PID 参数
	0x27	读取位置环 PID 参数
	0x28	读取输入累计脉冲值
	0x29	读取电机实时转速
	0x2A	读取电机实时位置
	0x2B	读取电机位置误差
	0x2C	读取电机运行状态
	0x2D	读取堵转标志
	0x2E	读取堵转电流
	0x2F	读取使能状态
	0x30	读取到位标志
	0x31	读取系统参数
	0x32	读取驱动参数
设置参数指令 (0x60 ~ 0x7F)	0x60	设置从机地址
	0x61	设置分组地址
	0x62	设置工作模式
	0x63	设置位置环 PID 参数
	0x64	设置位置环最大力矩限制
	0x65	设置细分
	0x66	设置目标电流
	0x67	设置串口波特率
	0x68	设置 CAN 波特率
	0x69	设置 Modbus 协议
	0x6A	设置堵转保护功能
	0x6B	设置堵转电流
	0x6C	设置发送 CAN_ID
	0x6D	设置 DIR 正转电平
	0x6E	设置 EN 脚有效电平
	0x6F	设置指令回响

	0x70	设置按键锁定
	0x71	设置自动熄屏
	0x72	设置 IO 启停电平
	0x73	设置速度环 PID 参数
限位回零指令（0x90 ~ 0x9F）	0x90	设置左限位原点位置
	0x91	设置有无限位回零
	0x92	触发回零
	0x93	强制中断并退出回零操作
	0x94	读取回零参数
	0x95	修改原点回零超时时间
	0x96	读取回零状态
	0x97	设置上电自动回零
	0x98	设置右限位原点位置
	0x99	设置左右限位开关状态
运动控制指令（0xE0 ~ 0xFF）	0xE0	开环速度模式控制
	0xE1	开环绝对位置模式控制
	0xE2	开环相对位置模式控制
	0xE3	开环脉冲模式控制
	0xE4	IO 启停模式
	0xF0	力矩模式控制
	0xF1	速度模式控制
	0xF2	绝对位置模式控制
	0xF3	相对位置模式控制
	0xF4	脉冲模式控制
	0xF5	脉宽位置模式控制
	0xF6	脉宽力矩模式控制
	0xF7	脉宽速度模式控制
	0xF8	将当前位置角度清零
	0xF9	解除堵转保护
	0xFA	电机使能控制
	0xFB	清除状态（堵转、刹车、失能）
	0xFC	立即停止（刹车）

4.1 指令格式说明

指令格式，如表 4.1.1 所示：

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
0xC5	addr	code		CHECKSUM	0x5C
上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
0xC5	addr	code		CHECKSUM	0x5C

表 4.1.1 串行数据格式说明

1，帧头为 0xC5，帧尾为 0x5C，都是固定的

2，从机地址范围：0~255，其中 0 为广播地址，1~255 为从机地址（默认地址为 1）

3，功能码范围：0~255，用于选择执行不同的指令

4，指令数据：指令发送或者应答的真正数据，不同功能码，长度可能不同

5，校验码：采用的校验方式是 8 位校验和（CHECKSUM 8bit）。

校验码计算方法如下：

CHECKSUM = (帧头 + 地址 + 功能码 + 数据区所有字节) & 0xFF

例如 指令“C5 01 80 02 CHECKSUM 5C”

CHECKSUM = (0xC5 + 0x01 + 0x80 + 0x02) & 0xFF = 0x148 & 0xFF = 0x48

6，**时序要求（单条指令内）**：主机发送指令时，整帧数据（从 0xC5 到 0x5C）必须 **连续输出**，字节之间不允许超过 1 个字节周期的延时，否则可能导致从机接收失败。

7，**时序要求（指令间隔）**：两条完整指令之间应保持 **至少 2ms 的时间间隔**，以保证从机有足够时间完成上一条指令的解析与处理。

注意：

- **使用广播地址或分组地址发送命令，从机均不应答。**
- **主机发送的从机地址，如果总线上没有匹配的从机，也不应答。**
- **如果不想从机应答，可以使用 0x6F 指令回响功能码进行设置。**

为了方便大家熟悉和查找相关指令，我们把指令分为了四类：系统类指令、参数类指令、运动类指令、回零类指令。详细的指令描述请看下文。

4.2 系统类指令说明

4.2.1 校准编码器指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	01H	无	C7H	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	01H	Byte1	Byte2	CHECKSUM	5CH

注意：校准编码器前，请确保电机是空载状态!!!

1，从机地址：01H（后续都用 01H 从机地址来举例，将不再赘述）

2，功能码：01H，对应校准编码器功能

3，指令数据：

◆ **下行帧：01H 功能码无指令数据**

◆ **上行帧：有 2 个字节指令数据（Byte1：表示操作结果，Byte2：表示校准结果）**

下面重点介绍一下上行帧的 Byte1，Byte1 是上行帧指令数据部分的第一个字节，固定用来表示指令操作结果。指令操作结果有以下几种可能：

0x01，表示指令操作成功

0xE1，表示帧长度不足（小于最小帧长度）

0xE2，表示帧头错误（非 0xC5）

0xE3，表示帧尾错误（非 0x5C）

0xE4，表示校验和错误

0xE5，表示不支持的功能码

0xE6，表示数据不合法

可以看到只有收到 0x01 才表示指令操作成功，其它值都表示错误。错误码都是 0xE 开头，其中‘E’寓意‘Error’，方便理解。

Byte1（指令操作结果）只在这里进行介绍，其它指令都一样的，后续就不再赘述！

Byte2：表示校准结果，有以下几种结果：

0：空闲

1：校准中

2：校准失败

3：校准成功

◆ **举个例子：**

主机发送 C5 01 01 C7 5C

从机空闲，应答 C5 01 01 01 00 C8 5C

从机校准中，应答 C5 01 01 01 01 C9 5C

从机校准失败，应答 C5 01 01 01 02 CA 5C

从机校准成功，应答 C5 01 01 01 03 CB 5C

4.2.2 复位重启指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	02H	无	C8H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	02H	Byte1	CHECKSUM	5CH

该指令用于复位重启驱动板，不会修改配置参数。

◆ **举个例子：**

主机发送 C5 01 02 C8 5C

从机操作成功，应答 C5 01 02 01 C9 5C

4.2.3 恢复出厂设置指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	03H	无	C9H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	03H	Byte1	CHECKSUM	5CH

该指令用于恢复出厂设置，把配置参数修改为出厂默认值。

◆ **举个例子：**

主机发送 C5 01 03 C9 5C

从机操作成功，应答 C5 01 03 01 CA 5C

4.2.4 参数保存指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	04H	无	CAH	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	04H	Byte1	CHECKSUM	5CH

该指令用于保存所有掉电不丢失的配置参数。

◆ **举个例子：**

主机发送 C5 01 04 CA 5C

从机操作成功，应答 C5 01 04 01 CB 5C

4.3 参数类指令说明

4.3.1 读取软硬件版本指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	20H	无	E6H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	20H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于读取驱动器软硬件版本号。

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示固件版本号

Byte3：表示硬件版本号

◆ **举个例子：**

主机发送 C5 01 20 E6 5C

从机操作成功，应答 C5 01 20 01 0A 0A 07 5C

上行帧的指令数据：

Byte2：0A(即十进制 10)，表示固件版本号为 V1.0

Byte3：0A(即十进制 10)，表示硬件版本号为 V1.0

4.3.2 读取电机磁链指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	21H	无	E7H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	21H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于读取电机磁链。（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示电机磁链参数（单位 mWb）

◆ **举个例子：**

主机发送 C5 01 21 E7 5C

从机操作成功，应答 C5 01 21 01 40 97 0A 3D 06 5C

上行帧的指令数据：

Byte2~ Byte5：上行帧的指令数据我们做了处理，采用**大端序输出**。即 0x40970A3D 表示电机磁链参数为 4.72mWb。

整型数据怎么能读成浮点型数据？刚接触到这个的读者可能会觉得很奇怪。这其实在嵌入式开发中是非常常见且重要的技巧，下面给大家介绍一下。

◆ **STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南（后续将不再赘述）**

核心概念：

联合体（union）：所有成员共享同一块内存空间（大小为最大成员尺寸）。写入一个成员，会覆盖整个内存区域；读取另一个成员，会以新类型的规则重新解释这段内存中的原始二进制数据。这不是类型转换，而是数据解释方式的切换。

STM32 内存模型（小端系统）

在小端模式中，多字节数据的低位字节存储在低地址，高位字节存储在高地址。

下面定义一个联合体：

```
typedef union
{
    float       f_val;          /* 单精度浮点数 (4 字节) */
    uint32_t    u32_val;        /* 32 位无符号整数 (4 字节) */
    uint8_t     bytes[4];       /* 字节数组，用于观察内存布局 */
} converter_t;
```

假设 f_val = 4.72，其在内存中存储的十六进制数表示为 0x40970A3D。

在小端系统中，内存布局如下：

内存地址	存储的数据 Hex	对应 bytes[]索引
低地址	3DH	bytes[0]
↓	0AH	bytes[1]

↓	97H	bytes[2]
高地址	40H	bytes[3]

用 u32_val 读取这 4 个字节时，会按地址从低到高组合成 0x3D0A9740 吗？不会！CPU 知道这是一个 32 位整数，它会直接访问整个 4 字节字地址空间，得到正确的值 0x40970A3D。字节序的影响主要体现在字节级别的操作（如通过 bytes[]数组访问或串口发送单个字节时的访问）。例如用 bytes[0]读取时得到 3DH，…用 bytes[3]读取时得到 40H。我们提供的 STM32 例程就是字节数组的方式，方便串口发送数据。

0x40970A3D 用浮点数变量 f_val 读取的结果，代码实现如下：

```
int main()
{
    converter_t u;                      /* 定义联合体变量 */
    u.u32_val = 0x40970A3D;             /* 将十六进制值 0x40970A3D 写入联合体 */

    printf("Hex: 0x%08X\r\n ", u.u32_val);  /* 打印 u32_val 的值 */
    printf("Float: %.6f\r\n ", u.f_val);    /* 打印 f_val 浮点数小数点后 6 位 */
    return 0;
}
```

打印结果：

```
Hex: 0x40970A3D
Float: 4.720000
```

该代码在例程 00 联合体 float 和 uint32_t 类型互换读取方法，大家可以直接使用！！！

总结：联合体提供了同一块内存的多种视图。写入一种视图，读取另一种视图，实现了对原始二进制数据的不同解释，从而在浮点数和整数的二进制表示之间建立桥梁，方便我们进行这种巧妙的转换。

注意事项：在本套自定义中，指令数据采用大端序输出。在小端系统中（如：STM32），如果想自行解析 float 转 uint8_t[4]的数据，那么就要切换成小端序。方法在我们提供的 STM32 例程 process_frame.c 中有，这里贴出来：

```
/* 将读取到的数据转成浮点数 */
typedef union
{
    float f;
    uint8_t b[4];
} float_union_t;

/**
 * @brief       将 4 个字节的数据转换为 float 类型
 * @param       buf 指向包含 4 字节数据的缓冲区（使用大端格式：高字节在前）
 * @return      转换后的 float 浮点数值
 */
float bytes_to_float(uint8_t *buf)
{
    float_union_t u;
    /* 将字节逐一写入联合体中的字节数组（大端顺序，所以这里要反过来传值，重新变成小端） */
    u.b[0] = buf[3];    /* 高字节 */
```

```
    u.b[1] = buf[2];
    u.b[2] = buf[1];
    u.b[3] = buf[0];    /* 低字节 */

    /* 返回组合后的浮点数 */
    return u.f;
}
```

指令数据为什么采用大端序输出呢？答：符合人们从左到右的阅读习惯，方便大家阅读。

4.3.3 读取相电阻和相电感指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	22H	无	E8H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	22H	Byte1	…	Byte9	CHECKSUM	5CH

该指令用于读取电机相电阻和相电感(含 MOS 内阻)。（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示电机相电阻参数（单位Ω）

Byte6~ Byte9：表示电机相电感参数（单位 mH）

◆ **举个例子：**

主机发送 C5 01 22 E8 5C

从机操作成功，应答 C5 01 22 01 3F DC D7 D3 40 88 02 D4 4C 5C

上行帧的指令数据：

Byte2~ Byte5：0x3FDCD7D3 表示电机相电阻参数为 1.725Ω

Byte2~ Byte5：0x408802D4 表示电机相电感参数为 4.250mH

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

4.3.4 读取相电流指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	23H	无	E9H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	23H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于读取电机相电流。（类型：int16_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte3：表示电机相电流（单位 mA）

◆ **举个例子：**

主机发送 C5 01 23 E9 5C
从机操作成功，应答 C5 01 23 01 00 6C 56 5C
上行帧的指令数据：
Byte2~ Byte3：0x006C 表示电机相电流为 108mA

4.3.5 读取总线电压指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	24H	无	EAH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	24H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于读取驱动器总线电压。（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte5：表示驱动器总线电压（单位 V）

◆ **举个例子：**

主机发送 C5 01 24 EA 5C
从机操作成功，应答 C5 01 24 01 41 BF C7 C0 72 5C
上行帧的指令数据：
Byte2~ Byte5：0x41BFC7C0 表示驱动器总线电压为 23.97V
4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

4.3.6 读取电流环 PID 参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	25H	无	EBH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	25H	Byte1	…	Byte17	CHECKSUM	5CH

该指令用于读取电流环 PID 参数。（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte5：表示 Q 轴 PI 参数 P
Byte6~ Byte9：表示 Q 轴 PI 参数 I
Byte10~ Byte13：表示 D 轴 PI 参数 P
Byte14~ Byte17：表示 D 轴 PI 参数 I

◆ **举个例子：**

主机发送 C5 01 25 EB 5C
从机操作成功，应答 C5 01 25 01 37 27 C5 AC 35 86 37 BD 37 27 C5 AC 35 86 37 BD E8 5C

上行帧的指令数据：

Byte2~ Byte5： 0x3727C5AC 表示 Q 轴 PI 参数 p 为 0.00001

Byte6~ Byte9：0x358637BD 表示 Q 轴 PI 参数 I 为 0.000001

Byte10~ Byte13：0x3727C5AC 表示 D 轴 PI 参数 P 为 0.00001

Byte14~ Byte17：0x358637BD 表示 D 轴 PI 参数 I 为 0.000001

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

4.3.7 读取速度环 PID 参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	26H	无	ECH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	26H	Byte1	…	Byte13	CHECKSUM	5CH

该指令用于读取速度环 PID 参数。（类型：uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示 PID 参数 P

Byte6~ Byte9：表示 PID 参数 I

Byte10~ Byte13：表示 PID 参数 D

◆ **举个例子：**

主机发送 C5 01 26 EC 5C

从机操作成功，应答 C5 01 26 01 00 00 00 03 00 00 00 04 00 00 00 00 F4 5C

上行帧的指令数据：

Byte2~ Byte5：0x00000004 表示 Q 轴 PI 参数 p 为 3

Byte6~ Byte9：0x00000004 表示 Q 轴 PI 参数 I 为 4

Byte10~ Byte13：0x00000000 表示 D 轴 PI 参数 P 为 0

4.3.8 读取位置环 PID 参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	27H	无	EDH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	27H	Byte1	…	Byte13	CHECKSUM	5CH

该指令用于读取位置环 PID 参数。（类型：uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示 PID 参数 P

Byte6~ Byte9：表示 PID 参数 I

Byte10~ Byte13：表示 PID 参数 D

◆ **举个例子：**

主机发送 C5 01 27 ED 5C

从机操作成功，应答 C5 01 27 01 00 00 01 2C 00 00 00 1E 00 00 01 7C B6 5C

上行帧的指令数据：

Byte2~ Byte5：0x0000012C 表示 Q 轴 PI 参数 p 为 300

Byte6~ Byte9：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte10~ Byte13：0x0000017C 表示 D 轴 PI 参数 P 为 380

4.3.9 读取输入累计脉冲数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	28H	无	EEH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	28H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于读取输入累计脉冲数。（类型：uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示输入累计脉冲数

◆ **举个例子：**

主机发送 C5 01 28 EE 5C

从机操作成功，应答 C5 01 28 01 00 00 C8 00 EF 5C

上行帧的指令数据：

Byte2~ Byte5：0x0000C800 表示输入累计脉冲数为 51200

4.3.10 读取电机实时转速指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	29H	无	EFH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	29H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于读取电机实时转速。（类型：int16_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte3：表示电机实时转速（单位 RPM）

◆ **举个例子：**

主机发送 C5 01 29 EF 5C

从机操作成功，应答 C5 01 29 01 00 1F 0F 5C

上行帧的指令数据：

Byte2~ Byte3：0x001F 表示电机实时转速为 31RPM

4.3.11 读取电机实时位置指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2AH	无	F0H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	2AH	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于读取电机实时位置。（类型：int32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示电机实时位置（51200 代表一圈）

◆ **举个例子：**

主机发送 C5 01 2A F0 5C

从机操作成功，应答 C5 01 2A 01 00 07 D0 00 C8 5C

上行帧的指令数据：

Byte2~ Byte5：0x0007D000 表示电机实时位置为 512000

4.3.12 读取电机位置误差指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2BH	无	F1H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	2BH	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于读取电机位置误差。（类型：int32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：表示电机位置误差（51200 代表一圈）

◆ **举个例子：**

主机发送 C5 01 2B F1 5C

从机操作成功，应答 C5 01 2B 01 00 00 00 01 F3 5C

上行帧的指令数据：

Byte2~ Byte5：0x00000001 表示电机位置误差为 1

4.3.13 读取电机运行状态指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2CH	无	F2H	5CH

上行帧(上位机<--驱动板)

帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	2CH	Byte1	Byte2	CHECKSUM	5CH

该指令用于读取电机运行状态。（类型：uint8_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示电机运行状态，有以下可能：

0：停止状态
1：任务完成状态
2：正在运行
3：过载状态
4：堵转状态
5：欠压状态

◆ **举个例子：**

主机发送 C5 01 2C F2 5C

从机操作成功，应答 C5 01 2C 01 01 F4 5C

上行帧的指令数据 Byte2：0x01 表示电机运行状态为任务完成状态

4.3.14 读取电机堵转标志指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2DH	无	F3H	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	2DH	Byte1	Byte2	CHECKSUM	5CH

该指令用于读取电机堵转标志。（类型：uint8_t）

◆ 上行帧的指令数据：

Byte1：表示指令操作结果

Byte2：表示电机堵转标志，有以下可能：

0：未堵转　1：堵转

◆ 举个例子：

主机发送 C5 01 2D F3 5C

从机操作成功，应答 C5 01 2D 01 00 F4 5C

上行帧的指令数据 Byte2：0x00 表示电机未堵转

4.3.15 读取堵转电流指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2EH	无	F4H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	2EH	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于读取堵转电流。（类型：int16_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte3：表示堵转电流（单位 mA）

◆ **举个例子：**

主机发送 C5 01 2E F4 5C

从机操作成功，应答 C5 01 2E 01 05 DC D6 5C

上行帧的指令数据：

Byte2~ Byte3：0x05DC 表示堵转电流为 1500mA

4.3.16 读取电机使能状态指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	2FH	无	F5H	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	2FH	Byte1	Byte2	CHECKSUM	5CH

该指令用于读取电机使能状态。（类型：uint8_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示电机使能状态，有以下可能：

0：使能电机　　1：失能电机

◆ **举个例子：**

主机发送 C5 01 2F F5 5C

从机操作成功，应答 C5 01 2F 01 00 F6 5C

上行帧的指令数据：

Byte2：0x00 表示失能电机

4.3.17 读取电机到位标志指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	30H	无	F6H	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	30H	Byte1	Byte2	CHECKSUM	5CH

该指令用于读取电机到位标志。（类型：uint8_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示电机到位标志，有以下可能：

0：未到位

1：到位

◆ **举个例子：**

主机发送 C5 01 30 F6 5C

从机操作成功，应答 C5 01 30 01 01 F8 5C

上行帧的指令数据：

Byte2：0x01 表示电机到位

4.3.18 读取系统参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	31H	无	F7H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	31H	Byte1	…	Byte41	CHECKSUM	5CH

该指令用于读取系统参数，这部分参数都是只读属性。

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：电机总线电压（单位 V）（float → uint8_t[4]）

Byte6~ Byte7：读取相电流（单位 mA）（类型 int16_t）

Byte8~ Byte11：电机磁链参数（单位 mWb）（float → uint8_t[4]）

Byte12~ Byte15：电机相电阻参数（单位Ω）（float → uint8_t[4]）

Byte16~ Byte19：电机相电感参数（单位 mH）（float → uint8_t[4]）

Byte20~ Byte21：读取实时转速（单位 RPM）（类型 int16_t）

Byte22~ Byte25：读取目标位置（51200 代表一圈）（类型 int32_t）

Byte26~ Byte29：读取实时位置（51200 代表一圈）（类型 int32_t）

Byte30~ Byte33：读取位置误差（51200 代表一圈）（类型 int32_t）

Byte34~ Byte37：读取脉冲数（类型 uint32_t）

Byte38：电机使能状态

0：电机使能

1：电机失能

Byte39：电机到位标志，有以下可能：

0：未到位

1：到位

Byte40：电机堵转标志，有以下可能：

0：未堵转

1：堵转

Byte41：地址模式，有以下可能：

0：从机地址模式

1：分组地址模式

◆ **举个例子：**

主机发送 C5 01 31 F7 5C

从机操作成功，应答 C5 01 31 01 41 C0 00 00 00 0F 40 88 5D 39 3F ED EB 28 40 8D 27 10 00 00 00 07 D0 00 00 07 D0 00 00 00 00 00 00 00 7D 02 00 01 00 D7 5C

该指令一次性读取系统参数，便于观察。用我们提供的 STM32 例程可以一次性在串口助手打印指令数据解析结果，如下所示：

Byte2~ Byte5：电机总线电压 24.0V
Byte6~ Byte7：读取相电流 15mA
Byte8~ Byte11：电机磁链 4.26mWb
Byte12~ Byte15：电机相电阻 1.86Ω
Byte16~ Byte19：电机相电感 4.41mH
Byte20~ Byte21：读取实时转速 0RPM
Byte22~ Byte25：读取目标位置（51200 代表一圈）512000
Byte26~ Byte29：读取实时位置（51200 代表一圈）512000
Byte30~ Byte33：读取位置误差（51200 代表一圈）0
Byte34~ Byte37：读取脉冲数 32002
Byte38：0：电机使能
Byte39：1：到位
Byte40：0：未堵转
Byte41：0：从机地址模式
4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

4.3.19 读取驱动参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	32H	无	F8H	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	32H	Byte1	…	Byte32	CHECKSUM	5CH

该指令用于读取全部配置参数。

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2：电机工作模式，有以下可能：（类型 uint8_t）
0：通信位置模式
1：通信速度模式
2：通信力矩模式
3：脉冲模式
4：脉宽位置模式
5：脉宽速度模式
6：脉宽力矩模式
7：回零模式
Byte3：指令回响状态，有以下可能：
0：正常回响
1：不回响
Byte4~ Byte7 串口波特率（单位 bps）（类型 uint32_t）
Byte8~ Byte9：CAN 波特率（单位 Kbps）（类型 uint16_t）
Byte10：DIR 正转电平，有以下可能：

0：高电平正转
1：低电平正转

Byte11：EN 脚有效电平，有以下可能：
0：EN 引脚低电平使能
1：EN 引脚高电平使能
2：EN 引脚保持有效

Byte12~ Byte13：设置的细分（类型 uint16_t）
Byte14~ Byte15：位置环最大力矩（单位：mA）（类型 int32_t）
Byte16~ Byte19：位置环 PID 参数 P（类型 uint32_t）
Byte20~ Byte23：位置环 PID 参数 I（类型 uint32_t）
Byte24~ Byte27：位置环 PID 参数 D（类型 uint32_t）
Byte28~ Byte29：堵转电流（单位：mA）（类型 int16_t）
Byte30：电机堵转保护功能标志
0：关闭堵转保护功能
1：开启堵转保护功能
Byte31：按键上锁状态，有以下可能：
0：解锁
1：上锁
Byte32~ Byte35：速度环 PID 参数 P（类型 uint32_t）
Byte36~ Byte39：速度环 PID 参数 I（类型 uint32_t）
Byte40~ Byte43：速度环 PID 参数 D（类型 uint32_t）
Byte44：自动熄屏状态，有以下可能：
0：关闭自动熄屏
1：开启自动熄屏
Byte45：IO 启停电平，有以下可能：
0：低电平启动
1：高电平启动

◆ **举个例子：**

主机发送 C5 01 32 F8 5C

从机操作成功，应答 C5 01 32 01 03 00 00 01 C2 00 01 F4 00 01 00 10 0B B8 00 00 01 2C 00 00 00 1E 00 00 01 7C 05 DC 01 00 00 32 5C

Byte2：3：脉冲模式
Byte3：0：正常回响
Byte4~ Byte7：串口波特率为：115200
Byte8~ Byte9：CAN 速率为：500K
Byte10：0：高电平正转
Byte11：1：EN 引脚高电平使能
Byte12~ Byte13：细分为：16
Byte14~ Byte15：位置环最大力矩：3000
Byte16~ Byte19：PID 参数 P 为 300
Byte20~ Byte23：PID 参数 I 为 30
Byte24~ Byte27：PID 参数 D 为 380
Byte28~ Byte29：堵转电流 1500 mA

Byte30：1：开启堵转保护功能
Byte31：0：按键解锁
Byte32：0：关闭自动熄屏
4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

4.3.20 设置从机地址指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	60H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	60H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置从机地址。（**注意：总线上不能设置两个一样的从机地址，应答会冲突**）

◆ **下行帧的指令数据：**

Byte1：新的从机地址（范围：1~255）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2：新的从机地址

◆ **举个例子：**

主机发送 C5 01 60 02 28 5C
从机操作成功，应答 C5 01 60 01 02 29 5C
下行帧的指令数据 Byte1：新的从机地址设置为 2
上行帧的指令数据 Byte2：新的从机地址成功设置为 2
注意：下次再想跟这个从机通信，注意从机地址要改成 2 了！！！

4.3.21 设置分组地址指令

有了设置从机地址指令，为什么还要设置分组地址指令呢？电机驱动器被设置为相同的分组地址可实现多机同步通讯和控制。而从机地址是不允许被设置成一样的。下面来介绍一下它们的区别。

从机地址和分组地址的区别：在电机驱动器中，从机地址和分组地址是共用一套地址的，只有模式的区别。它们可以被视为是两种模式，当使用 60H 功能码设置从机地址时，该从机就被切换到从机地址模式；当使用 61H 功能码设置分组地址时，该从机就被切换到分组地址模式。

从机地址模式下：总线上的电机不可以设置为相同的从机地址，因为从机收到指令后都会**执行指令，并应答主机**，会出现总线应答冲突等问题。（可以用 6FH 功能码设置从机指令不回响。）

分组地址模式下：总线上的电机可以设置为相同的分组地址，因为被设置为分组地址的从机收到指令都**只执行指令，不作应答**。

如果想分组地址模式下，也有从机应答，怎么办？答：将其中一个从机地址的值设置等于分组地址，此过程要用 60H 功能码来设置。

顺便介绍一下广播地址，如果下行帧的从机地址是 0，表示**广播地址**，那么总线上所有的从机都被同步通讯和控制。**发广播地址指令，所有从机都只执行指令，不作应答，避免应答冲突。**广播地址和分组地址都可以实现同步通讯和控制，**广播地址是所有从机都进行的，**

而分组地址是部分被设置为相同分组地址的从机进行的。

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	61H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	61H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置分组地址。（**总线上可设置相同的分组地址，进行同步通讯和控制**）

◆ **下行帧的指令数据：**

Byte1：新的分组地址（范围：1~255）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：新的分组地址

◆ **举个例子：**

主机发送 C5 01 61 02 29 5C

从机操作成功，应答 C5 01 61 01 02 2A 5C

下行帧的指令数据：

Byte1：新的分组地址设置为 2

上行帧的指令数据：

Byte2：新的分组地址成功设置为 2

注意：下次再想跟这个从机通信，注意从机地址要改成 2 了!!!

◆ **假设有 6 个电机，地址设置如下：**

电机序号	广播地址	从机地址	分组地址
电机 1	0	1	
电机 2	0		1
电机 3	0		1
电机 4	0		2
电机 5	0		2
电机 6	0		2

从机地址模式和分组地址模式是互斥的，同一时刻只能二选一!!!

主机发送 C5 00 F3 00 64 07 D0 00 07 D0 00 CA 5C，电机 1~6 都转 10 圈，都不作应答

主机发送 C5 01 F3 00 64 07 D0 00 07 D0 00 CB 5C，电机 1~3 都转 10 圈，只有电机 1 应答

主机发送 C5 02 F3 00 64 07 D0 00 07 D0 00 CC 5C，电机 4~6 都转 10 圈，都不作应答

4.3.22 设置工作模式指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	62H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	62H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置工作模式。

◆ **下行帧的指令数据：**

Byte1：新的工作模式，有以下可能：

0x00：通信位置模式
0x01：通信速度模式
0x02：通信力矩模式
0x03：脉冲模式
0x04：脉宽位置模式
0x05：脉宽速度模式
0x06：脉宽力矩模式
0x07：回零模式

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2：新的工作模式

◆ **举个例子：**

主机发送 C5 01 62 00 28 5C
从机操作成功，应答 C5 01 62 01 00 29 5C
下行帧的指令数据 Byte1：新的工作模式为 0，即通信位置模式
上行帧的指令数据 Byte2：新的工作模式为 0，即通信位置模式

4.3.23 设置位置环 PID 参数指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	63H	Byte1	…	Byte12	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	63H	Byte1	…	Byte13	CHECKSUM	5CH

该指令用于设置位置环 PID 参数。

◆ **下行帧的指令数据：**

Byte1~ Byte4：PID 参数 P（类型 uint32_t）
Byte5~ Byte8：PID 参数 I（类型 uint32_t）
Byte9~ Byte12：PID 参数 D（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte5：PID 参数 P（类型 uint32_t）
Byte6~ Byte9：PID 参数 I（类型 uint32_t）
Byte10~ Byte13：PID 参数 D（类型 uint32_t）

◆ **举个例子：**

主机发送 C5 01 63 00 00 01 2C 00 00 00 1E 00 00 01 7C F1 5C
从机操作成功，应答 C5 01 63 01 00 00 01 2C 00 00 00 1E 00 00 01 7C F2 5C
下行帧的指令数据：
Byte1~ Byte4：0x0000012C 表示 Q 轴 PI 参数 p 为 300
Byte5~ Byte8：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte9~ Byte12：0x0000017C 表示 D 轴 PI 参数 P 为 380
上行帧的指令数据：
Byte2~ Byte5：0x0000012C 表示 Q 轴 PI 参数 p 为 300
Byte6~ Byte9：0x0000001E 表示 Q 轴 PI 参数 I 为 30
Byte10~ Byte13：0x0000017C 表示 D 轴 PI 参数 P 为 380

4.3.24 设置速度/位置环最大力矩限制指令

下行帧(上位机-->驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	64H	Byte1	Byte2	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	64H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于设置速度/位置环最大力矩限制（范围：0~3000）（单位：mA）。

◆ **下行帧的指令数据：**

Byte1~ Byte2：速度/位置环最大力矩电流限制

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte3：速度/位置环最大力矩电流限制

◆ **举个例子：**

主机发送 C5 01 64 05 DC 0B 5C
从机操作成功，应答 C5 01 64 01 05 DC 0C 5C
下行帧的指令数据 Byte1~ Byte2：速度/位置环最大力矩电流限制为 1500mA
上行帧的指令数据 Byte2~ Byte3：速度/位置环最大力矩电流限制为 1500mA

4.3.25 设置细分指令

下行帧(上位机-->驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	65H	Byte1	Byte2	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	65H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于设置细分（范围：1~256）。

◆ **下行帧的指令数据：**

Byte1~ Byte2：细分

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte3：细分

◆ **举个例子：**

主机发送 C5 01 65 00 10 3B 5C
从机操作成功，应答 C5 01 65 01 00 10 3C 5C

下行帧的指令数据 Byte1~ Byte2：细分为 16
上行帧的指令数据 Byte2~ Byte3：细分为 16

4.3.26 设置目标电流指令

下行帧(上位机-->驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	66H	Byte1	Byte2	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	66H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于设置目标电流（范围：-3000~3000）（单位：mA）。

◆ **下行帧的指令数据：**

Byte1~ Byte2：目标电流

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte3：目标电流

◆ **举个例子：**

主机发送 C5 01 66 0B B8 EF 5C
从机操作成功，应答 C5 01 66 01 0B B8 F0 5C
下行帧的指令数据 Byte1~ Byte2：目标电流为 3000mA
上行帧的指令数据 Byte2~ Byte3：目标电流为 3000mA

4.3.27 设置串口波特率指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	67H	Byte1	…	Byte4	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	67H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于设置串口波特率（范围：0~3000000）（默认：115200）（单位 bps）。

◆ **下行帧的指令数据：**

Byte1~ Byte4：串口波特率

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2~ Byte5：串口波特率

◆ **举个例子：**

主机发送 C5 01 67 00 01 C2 00 F0 5C
从机操作成功，应答 C5 01 67 01 00 01 C2 00 F1 5C
下行帧的指令数据 Byte1~ Byte4：设置串口波特率为 115200 bps
上行帧的指令数据 Byte2~ Byte5：设置串口波特率为 115200 bps
注意：设置了串口波特率，上位也必须设置一样的波特率，方可通信！！！

4.3.28 设置 CAN 波特率指令

下行帧(上位机-->驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	68H	Byte1	Byte2	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	68H	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于设置 CAN 波特率（范围：125、250、500、1000）（单位 kbps）。

◆ **下行帧的指令数据：**

Byte1~ Byte2：CAN 波特率

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte3：CAN 波特率

◆ **举个例子：**

主机发送 C5 01 68 01 F4 23 5C

从机操作成功，应答 C5 01 68 01 01 F4 24 5C

下行帧的指令数据 Byte1~ Byte2：设置 CAN 波特率为 500 kbps

上行帧的指令数据 Byte2~ Byte3：设置 CAN 波特率为 500 kbps

注意：设置了 CAN 波特率，上位也必须设置一样的波特率，方可通信!!!

4.3.29 设置 moubus 指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	69H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	69H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置使用正点原子自定义协议（默认）还是 moubus-rtu 协议。

◆ **下行帧的指令数据：**

Byte1：0：使用正点原子自定义协议；1：使用 moubus-rtu 协议

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：使用正点原子自定义协议；1：使用 moubus-rtu 协议

◆ **举个例子：**

主机发送 C5 01 69 00 2F 5C

从机操作成功，应答 C5 01 69 01 00 30 5C

下行帧的指令数据 Byte1：0：使用正点原子自定义协议

上行帧的指令数据 Byte2：0：使用正点原子自定义协议

注意：如果设置为 1，那么正点原子自定义协议马上就不能通信了，需要通过屏幕菜单栏设置 modbus：失能；或者恢复出厂设置；又或者用 moubus-rtu 协议指令设置回来!!!

4.3.30 设置堵转保护功能指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	6AH	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	6AH	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置堵转保护功能。

◆ **下行帧的指令数据：**

Byte1：0：关闭堵转保护功能；1：开启堵转保护功能

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：关闭堵转保护功能；1：开启堵转保护功能

◆ **举个例子：**

主机发送 C5 01 6A 01 31 5C

从机操作成功，应答 C5 01 6A 01 01 32 5C

下行帧的指令数据 Byte1：1：开启堵转保护功能

上行帧的指令数据 Byte2：1：开启堵转保护功能

4.3.31 设置堵转电流指令

下行帧(上位机-->驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	6BH	Byte1	Byte2	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	6BH	Byte1	Byte2	Byte3	CHECKSUM	5CH

该指令用于设置堵转电流（范围：0~3000）（单位：mA）。

◆ **下行帧的指令数据：**

Byte1~ Byte2：堵转电流

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte3：堵转电流

◆ **举个例子：**

主机发送 C5 01 6B 07 D0 08 5C

从机操作成功，应答 C5 01 6B 01 07 D0 09 5C

下行帧的指令数据 Byte1~ Byte2：堵转电流 2000 mA

上行帧的指令数据 Byte2~ Byte3：堵转电流 2000 mA

4.3.32 设置发送 CAN_ID 指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾

C5H	01H	6CH	Byte1	…	Byte4	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	6CH	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于设置要发送的 CAN_ID（CAN 固定使用扩展帧，扩展帧 ID：29 位）。

◆ **下行帧的指令数据：**

Byte1~ Byte4：要发送的 CAN_ID

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：要发送的 CAN_ID

◆ **举个例子：**

主机发送 C5 01 6C 00 00 00 01 33 5C

从机操作成功，应答 C5 01 6C 01 00 00 00 01 34 5C

下行帧的指令数据 Byte1~ Byte4：要发送的 CAN_ID 为 0x00000001

上行帧的指令数据 Byte2~ Byte5：要发送的 CAN_ID 为 0x00000001

4.3.33 电机正反转 DIR 控制指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	6DH	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	6DH	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置 DIR 引脚控制电机正反转的有效电平。（仅用于脉冲模式）

◆ **下行帧的指令数据：**

Byte1：0：DIR 引脚高电平正转；1：DIR 引脚低电平正转

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：DIR 引脚高电平正转；1：DIR 引脚低电平正转

◆ **举个例子：**

主机发送 C5 01 6D 01 34 5C

从机操作成功，应答 C5 01 6D 01 01 35 5C

下行帧的指令数据 Byte1：1：DIR 引脚低电平正转

上行帧的指令数据 Byte2：1：DIR 引脚低电平正转

4.3.34 设置 EN 引脚有效电平指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	6EH	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)

帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	6EH	Byte1	Byte2	CHECKSUM	5CH

该指令设置 EN 引脚有效电平。（仅用于脉冲模式）

◆ **下行帧的指令数据：**

Byte1：0：EN 引脚低电平使能；1：EN 引脚高电平使能；2：EN 引脚保持有效

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：EN 引脚低电平使能；1：EN 引脚高电平使能；2：EN 引脚保持有效

◆ **举个例子：**

主机发送 C5 01 6E 01 35 5C

从机操作成功，应答 C5 01 6E 01 01 36 5C

下行帧的指令数据 Byte1：1：EN 引脚高电平使能

上行帧的指令数据 Byte2：1：EN 引脚高电平使能

4.3.35 指令回响指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	6FH	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	6FH	Byte1	Byte2	CHECKSUM	5CH

该指令用于配置从机接收到指令后是否回响（即应答）。

◆ **下行帧的指令数据：**

Byte1：从机接收到指令后是否回响，有以下可能：

0：指令回响

1：指令不回响

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：从机接收到指令后是否回响

◆ **举个例子：**

主机发送 C5 01 6F 01 36 5C

从机操作成功，应答 C5 01 6F 01 37 5C

下行帧的指令数据 Byte1：1：指令不回响

上行帧的指令数据 Byte2：1：指令不回响

4.3.36 设置按键锁定指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	70H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾

C5H	01H	70H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置按键锁定。

◆ **下行帧的指令数据：**

Byte1：0：按键解锁；1：按键上锁

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：按键解锁；1：按键上锁

◆ **举个例子：**

主机发送 C5 01 70 01 37 5C

从机操作成功，应答 C5 01 70 01 01 38 5C

下行帧的指令数据 Byte1：1：按键上锁

上行帧的指令数据 Byte2：1：按键上锁

注意：如果设置为 1，那么按键功能马上失效，需要通过屏幕菜单栏选择恢复出厂设置或者发送设置按键解锁指令，方可恢复按键功能!!!

4.3.37 设置自动熄屏指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	71H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	71H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置自动熄屏，长时间无操作会自动熄屏。

◆ **下行帧的指令数据：**

Byte1：0：关闭自动熄屏；1：开启自动熄屏

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：关闭自动熄屏；1：开启自动熄屏

◆ **举个例子：**

主机发送 C5 01 71 01 38 5C

从机操作成功，应答 C5 01 71 01 01 39 5C

下行帧的指令数据 Byte1：1：开启自动熄屏

上行帧的指令数据 Byte2：1：开启自动熄屏

注意：如果设置为 1，那么长时间无操作时会自动熄屏，可通过屏幕菜单栏自动熄屏或恢复出厂设置，也可发送关闭自动熄屏指令，恢复屏幕常亮!!!

4.3.38 设置 IO 启动指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	72H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)

帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	72H	Byte1	Byte2	CHECKSUM	5CH

该指令用于配置 IO 启停模式的启动电平。

◆ **下行帧的指令数据：**

Byte1：启动电平，有以下可能：

0：低电平启动

1：高电平启动

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：启动电平

◆ **举个例子：**

主机发送 C5 01 72 00 38 5C

从机操作成功，应答 C5 01 72 01 00 39 5C

下行帧的指令数据 Byte1：0：低电平启动

上行帧的指令数据 Byte2：0：低电平启动

4.3.39 设置速度环 PID 参数指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	73H	Byte1	…	Byte12	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	73H	Byte1	…	Byte13	CHECKSUM	5CH

该指令用于设置位置环 PID 参数。

◆ **下行帧的指令数据：**

Byte1~ Byte4：PID 参数 P（类型 uint32_t）

Byte5~ Byte8：PID 参数 I（类型 uint32_t）

Byte9~ Byte12：PID 参数 D（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：PID 参数 P（类型 uint32_t）

Byte6~ Byte9：PID 参数 I（类型 uint32_t）

Byte10~ Byte13：PID 参数 D（类型 uint32_t）

◆ **举个例子：**

主机发送 C5 01 73 00 00 01 2C 00 00 00 1E 00 00 01 7C 01 5C

从机操作成功，应答 C5 01 73 01 00 00 01 2C 00 00 00 1E 00 00 01 7C 02 5C

下行帧的指令数据：

Byte1~ Byte4：0x0000012C 表示 Q 轴 PI 参数 p 为 300

Byte5~ Byte8：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte9~ Byte12：0x0000017C 表示 D 轴 PI 参数 P 为 380

上行帧的指令数据：

Byte2~ Byte5：0x0000012C 表示 Q 轴 PI 参数 p 为 300

Byte6~ Byte9：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte10~ Byte13：0x0000017C 表示 D 轴 PI 参数 P 为 380

4.4 运动类指令说明

4.4.1 力矩模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F0H	Byte1	Byte2	Byte3	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F0H	Byte1	CHECKSUM	5CH

该指令用于力矩模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向，有以下可能：

0：正转

1：反转

Byte2~ Byte3：电流，取值范围：0~3000（单位：mA）（类型 uint16_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F0 00 03 E8 A1 5C

从机操作成功，应答 C5 01 F0 01 B7 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2~ Byte3：电流为 1000mA

上行帧的指令数据 Byte1：1，操作成功

4.4.2 速度模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F1H	Byte1	…	Byte6	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F1H	Byte1	CHECKSUM	5CH

该指令用于速度模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：

0：正转

1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~6000 RPM（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F1 00 64 44 16 00 00 75 5C

从机操作成功，应答 C5 01 F1 01 B8 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 600.0 RPM，用联合体 float→ uint8_t[4]读取为 0x44160000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

上行帧的指令数据 Byte1：1，操作成功

4.4.3 绝对位置模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F2H	Byte1	…	Byte8	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F2H	Byte1	CHECKSUM	5CH

该指令用于绝对位置模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~6000 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F2 00 64 07 D0 00 07 D0 00 CA 5C

从机操作成功，应答 C5 01 F2 01 B9 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 2000 RPM

Byte5~ Byte8：绝对位置，取值 512000

上行帧的指令数据 Byte1：1，操作成功

注意：在绝对位置模式控制下，当前位置角度值到设定的绝对位置值后，重复发送上述指令，电机将不会继续运动，除非把绝对位置值改大，或者将当前位置角度清零后，再发。

4.4.4 相对位置模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F3H	Byte1	…	Byte8	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F3H	Byte1	CHECKSUM	5CH

该指令用于相对位置模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~6000 RPM（类型 uint16_t）

Byte5~ Byte8：相对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F3 00 64 07 D0 00 07 D0 00 CB 5C

从机操作成功，应答 C5 01 F3 01 BA 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 2000 RPM

Byte5~ Byte8：相对位置，取值 512000（即转 10 圈）

上行帧的指令数据 Byte1：1，操作成功

注意：在相对位置模式控制下，每次设置新的相对位置，当前位置角度值会一直累加，可重复发送指令，让电机继续运动。

4.4.5 脉冲模式控制指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F4H	无	BCH	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F4H	Byte1	CHECKSUM	5CH

该指令用于让电机进入脉冲模式控制。说明：EN 引脚用于使能脉冲模式控制，STP 引脚用于输入脉冲，DIR 用于控制电机旋转方向，涉及到的功能码有 0x6D 和 0x6E。

◆ **举个例子：**

主机发送 C5 01 F4 BA 5C

从机操作成功，应答 C5 01 F4 01 BB 5C

上行帧的指令数据 Byte1：1，操作成功

进入脉冲模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程:05 脉冲模式控制(计算目标速度)和 06 脉冲模式控制(计算脉冲频率)。

◆ **脉冲模式控制简明介绍：**

1， 脉冲模式使能：EN 引脚和 0x6E 功能码指令共同控制

2， 脉冲模式电机旋转方向：DIR 引脚和 0x6D 功能码指令共同控制

3， 脉冲模式电机旋转速度：确定(1.8° 步进角)电机的情况下，由脉冲频率和细分控制

公式：RPM = (脉冲频率 * 60) / (每圈总步数)

其中，RPM 表示速度，单位：RPM，即圈每分钟。脉冲频率，单位：Hz。每圈总步数，即电机转一圈要多少个步数（一个脉冲对应一个微步）。在无细分的情况下，1.8° 步进角的步进电机每旋转一圈对应 200 步（即 360° /1.8° ）。

细分：一种驱动技术，通过控制电机线圈中的电流，将电机的一个完整物理步距（如 1.8°）分割成若干个更小的、电流连续的微步来运行。比如：对于 1.8°（200 步/圈）的电机，4 细分后，每个微步的步距角 = 1.8° / 4 = 0.45°，每圈总步数就是 360° / 0.45° = 800。

所以一般计算每圈总步数，都会先看细分设置了多少。例如我们的步进电机驱动器默认的细分设置为 16，那么每圈总步数 = （360° /1.8° ） * 16 = 3200 个脉冲/圈。

控制分辨率越高，运行越平滑。

4， 脉冲模式电机旋转角度：确定(1.8° 步进角)电机的情况下，由细分和脉冲数控制

知道细分概念，控制旋转角度很简单了。例如：想控制电机旋转 720° ，在 16 细分下，只需要输入 3200 * 2 = 6400 个脉冲即可。

4.4.6 脉宽位置模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F5H	Byte1	…	Byte12	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F5H	Byte1	CHECKSUM	5CH

该指令用于脉宽位置模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的位置角度。PWM 占空比和旋转的位置角度对应关系由用户自定义。

◆ **下行帧的指令数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的位置（注意 51200 为一圈）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的位置（注意 51200 为一圈）（类型 int32_t）

上行帧的指令数据：Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F5 09 C4 01 F4 00 00 64 00 00 00 00 00 E1 5C

从机操作成功，应答 C5 01 F5 01 BC 5C

下行帧的指令数据：

Byte1~ Byte2：高电平脉宽最长长度，取值：2500us

Byte3~ Byte4：高电平脉宽最短长度，取值：500us

Byte5~ Byte8：脉宽最长长度对应的位置，取值：25600（旋转半圈）

Byte9~ Byte12：脉宽最短长度对应的位置，取值：0（未旋转）

上行帧的指令数据 Byte1：1，操作成功

◆ **该例子模仿 SG90 舵机来定义，PWM 的占空比和控制关系如下：**

PWM 的频率设置为 50Hz(即周期为 20ms)

高电平时间	占空比	角度
0.5ms	2.5%	0°
1.0ms	5.0%	45°

1.5ms	7.5%	90°
2.0ms	10.0%	135°
2.5ms	12.5%	180°

由 PWM 的占空比和控制关系可以知道：

当角度为 180° 时，对应高电平脉宽最长长度，取值为：2.5ms（即 2500us）

当角度为 0° 时，对应高电平脉宽最短长度，取值为：0.5ms（即 500us）

由于驱动器的位置值 51200 为一圈，所以脉宽最长长度对应的位置 180° ，对应的取值为：25600（旋转半圈）。脉宽最短长度对应的位置 0° ，对应的取值：0（未旋转）。

进入脉宽位置模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：07 脉宽位置模式。

4.4.7 脉宽力矩模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F6H	Byte1	…	Byte12	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F6H	Byte1	CHECKSUM	5CH

该指令用于脉宽力矩模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的力矩。PWM 占空比和旋转的力矩对应关系由用户自定义。

◆ **下行帧的指令数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的电流（取值范围：0~3000mA）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的电流（取值范围：0~3000mA）（类型 int32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F6 0D AC 01 F4 00 00 0B B8 00 00 00 00 2D 5C

从机操作成功，应答 C5 01 F6 01 BD 5C

下行帧的指令数据：

Byte1~ Byte2：高电平脉宽最长长度，取值：3500us

Byte3~ Byte4：高电平脉宽最短长度，取值：500us

Byte5~ Byte8：脉宽最长长度对应的电流，取值：3000mA

Byte9~ Byte12：脉宽最短长度对应的电流，取值：0 mA

上行帧的指令数据 Byte1：1，操作成功

◆ **该例子自定义 PWM 的占空比和电流关系如下：**

PWM 的频率设置为 100Hz(即周期为 10ms)

高电平时间	占空比	电流
0.5ms	5%	0 mA
1.0ms	10%	500 mA
1.5ms	15%	1000 mA
2.0ms	20%	1500 mA

2.5ms	25%	2000 mA
3.0ms	30%	2500 mA
3.5ms	35%	3000 mA

由 PWM 的占空比和控制关系可以知道：

当电流为 3000 mA 时，对应高电平脉宽最长长度，取值为：3.5ms（即 3500us）

当角度为 0 mA 时，对应高电平脉宽最短长度，取值为：0.5ms（即 500us）

进入脉宽力矩模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：08 脉宽力矩模式控制。

4.4.8 脉宽速度模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	F7H	Byte1	…	Byte12	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F7H	Byte1	CHECKSUM	5CH

该指令用于脉宽速度模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的速度。PWM 占空比和旋转的速度对应关系由用户自定义。

◆ **下行帧的指令数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的速度（取值范围：0~6000 RPM）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的速度（取值范围：0~6000 RPM）（类型 int32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 F7 0C E4 01 2C 00 00 0B B8 00 00 00 00 9D 5C

从机操作成功，应答 C5 01 F7 01 BE 5C

下行帧的指令数据：

Byte1~ Byte2：高电平脉宽最长长度，取值：3300us

Byte3~ Byte4：高电平脉宽最短长度，取值：300us

Byte5~ Byte8：脉宽最长长度对应的速度，取值：3000 RPM

Byte9~ Byte12：脉宽最短长度对应的速度，取值：0 RPM

上行帧的指令数据：

Byte1：1，操作成功

◆ **该例子自定义 PWM 的占空比和速度关系如下：**

PWM 的频率设置为 100Hz(即周期为 10ms)

高电平时间	占空比	电流
0.3ms	3%	0 RPM
0.6ms	6%	300 RPM
0.9ms	9%	600 RPM
1.2ms	12%	900 RPM
1.5ms	15%	1200 RPM

1.8ms	18%	1500 RPM
2.1ms	21%	1800 RPM
2.4ms	24%	2100 RPM
2.7ms	27%	2400 RPM
3.0ms	30%	3000 RPM
3.3ms	33%	3000 RPM

由 PWM 的占空比和控制关系可以知道：

当速度为 3000 RPM 时，对应高电平脉宽最长长度，取值为：3.3ms（即 3300us）

当速度为 0 RPM 时，对应高电平脉宽最短长度，取值为：0.3ms（即 300us）

进入脉宽力矩模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：09 脉宽速度模式控制。

4.4.9 将当前的位置角度清零指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F8H	无	BEH	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F8H	Byte1	CHECKSUM	5CH

该指令用于将当前位置角度、位置误差和累计脉冲数清零。

◆ **举个例子：**

主机发送 C5 01 F8 BE 5C

从机操作成功，应答 C5 01 F8 01 BF 5C

4.4.10 解除堵转保护指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F9H	无	BFH	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	F9H	Byte1	CHECKSUM	5CH

该指令用于解除堵转保护（电机发生堵转后，可以使用该指令解除堵转保护）。

◆ **举个例子：**

主机发送 C5 01 F9 BF 5C

从机操作成功，应答 C5 01 F9 01 C0 5C

4.4.11 电机使能控制指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	FAH	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	FAH	Byte1	Byte2	CHECKSUM	5CH

该指令用于使能电机。

◆ **下行帧的指令数据：**

Byte1：0：使能电机；1：失能电机

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：0：使能电机；1：失能电机

◆ **举个例子：**

主机发送 C5 01 FA 00 C0 5C

从机操作成功，应答 C5 01 FA 01 00 C1 5C

下行帧的指令数据 Byte1：0：使能电机

上行帧的指令数据 Byte2：0：使能电机

4.4.12 清除状态（堵转、刹车，失能）指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	FBH	无	C1H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	FBH	Byte1	CHECKSUM	5CH

该指令可以清除状态（包括：堵转、刹车，失能）。

◆ **举个例子：**

主机发送 C5 01 FB C1 5C

从机操作成功，应答 C5 01 FB 01 C2 5C

4.4.13 立即停止（刹车）指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	FCH	无	C2H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	FCH	Byte1	CHECKSUM	5CH

该指令用于立即停止。**注意：刹停后，请及时清除电机状态，否则电机会严重发烫!!!**

◆ **举个例子：**

主机发送 C5 01 FC C2 5C

从机操作成功，应答 C5 01 FC 01 C3 5C

4.4.14 开环速度模式控制指令

下行帧(上位机-->驱动板)

帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	E0H	Byte1	…	Byte6	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E0H	Byte1	CHECKSUM	5CH

该指令用于速度模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：

0：正转

1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~400 RPM（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 E0 00 64 41 70 00 00 BB 5C

从机操作成功，应答 C5 01 E0 01 A7 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 15.0 RPM，用联合体 float→ uint8_t[4]读取为 0x41700000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

上行帧的指令数据 Byte1：1，操作成功

4.4.15 开环绝对位置模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	E1H	Byte1	…	Byte8	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E1H	Byte1	CHECKSUM	5CH

该指令用于绝对位置模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~400 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 E1 00 64 00 64 00 00 C8 00 37 5C

从机操作成功，应答 C5 01 E1 01 A8 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 100 RPM

Byte5~ Byte8：绝对位置，取值 51200

上行帧的指令数据 Byte1：1，操作成功

注意：在绝对位置模式控制下，当前位置角度值到设定的绝对位置值后，重复发送上述指令，电机将不会继续运动，除非把绝对位置值改大，或者将当前位置角度清零后，再发。

4.4.16 开环相对位置模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	E2H	Byte1	…	Byte8	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E2H	Byte1	CHECKSUM	5CH

该指令用于相对位置模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~400 RPM（类型 uint16_t）

Byte5~ Byte8：相对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 E2 00 64 00 64 00 00 C8 00 38 5C

从机操作成功，应答 C5 01 E2 01 A9 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 100 RPM

Byte5~ Byte8：相对位置，取值 51200

上行帧的指令数据 Byte1：1，操作成功

注意：在相对位置模式控制下，每次设置新的相对位置，当前位置角度值会一直累加，可重复发送指令，让电机继续运动。

4.4.17 开环脉冲模式控制指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E3H	无	A9H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E3H	Byte1	CHECKSUM	5CH

该指令用于让电机进入脉冲模式控制。说明：EN 引脚用于使能脉冲模式控制，STP 引脚用于输入脉冲，DIR 用于控制电机旋转方向，涉及到的功能码有 0x6D 和 0x6E。

◆ **举个例子：**

主机发送 C5 01 E3 A9 5C

从机操作成功，应答 C5 01 E3 01 AA 5C

上行帧的指令数据 Byte1：1，操作成功

进入脉冲模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：05 脉冲模式控制(计算目标速度)和 06 脉冲模式控制(计算脉冲频率)。

4.4.18 IO 启停模式控制指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	E4H	Byte1	…	Byte6	CHECKSUM	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	E4H	Byte1	CHECKSUM	5CH

该指令用于速度模式控制。

◆ **下行帧的指令数据：**

Byte1：电机旋转方向：

0：正转

1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（注意：0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~6000 RPM（float → uint8_t[4]）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

◆ **举个例子：**

主机发送 C5 01 E4 00 64 42 C8 00 00 18 5C

从机操作成功，应答 C5 01 E4 01 AB 5C

下行帧的指令数据：

Byte1：电机旋转方向：0：正转

Byte2：加减速度，取值 100

Byte3~ Byte4：速度，取值 50.0 RPM，用联合体 float→ uint8_t[4]读取为 0x42C80000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

上行帧的指令数据 Byte1：1，操作成功

4.5 回零类指令说明

4.5.1 设置左限位原点指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	90H	Byte1	…	Byte4	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	90H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于设置回零零点位置（类型 int32_t）。

◆ **下行帧的指令数据：**

Byte1~ Byte4：左限位原点位置

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：左限位原点位置

◆ **举个例子：**

主机发送 C5 01 90 00 00 C8 00 1E 5C

从机操作成功，应答 C5 01 90 01 00 00 C8 00 1F 5C

下行帧的指令数据 Byte1~ Byte4：设置回零零点位置为 51200

上行帧的指令数据 Byte2~ Byte5：回零零点位置为 51200

4.5.2 设置有无限位回零指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	91H	Byte1	…	Byte8	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	91H	Byte1	…	Byte9	CHECKSUM	5CH

该指令用于设置有无限位回零。

◆ **下行帧的指令数据：**

Byte1：使用无限位回零或有限位回零：

0：左无限位回零；

1：右无限位回零；

2：左有限位回零；

3：右有限位回零；

Byte2：电机旋转方向，0：正转；1：反转

Byte3~ Byte6：电机旋转速度范围：0~6000（单位 RPM）

Byte7~ Byte8：限位电流范围：0~3000（单位 mA），注意：限位电流，只对无限位回零有作用，用于检测是否到位

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2：使用无限位回零或有限位回零
Byte3：电机旋转方向
Byte4~ Byte7：电机旋转速度
Byte8~ Byte9：限位电流

◆ **举个例子：**

主机发送 C5 01 91 01 00 00 00 07 D0 0B B8 F2 5C
从机操作成功，应答 01 91 01 01 00 00 00 07 D0 0B B8 F3 5C
下行帧的指令数据：
Byte1：使用无限位回零或有限位回零，1：有限位回零
Byte2：电机旋转方向，0：正转
Byte3~ Byte6：电机旋转速度：2000 RPM
Byte7~ Byte8：限位电流：3000 mA
上行帧的指令数据：Byte1 为操作结果，其他和下行帧一样

4.5.3 触发回零指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	92H	Byte1	CHECKSUM	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	92H	Byte1	Byte2	CHECKSUM	5CH

该指令用于触发回零。

◆ **下行帧的指令数据：**

Byte1：回零模式：
0：单圈回零（按完整一圈去找到原点信号）
1：就近回零（电机从当前位置开始，朝最近的原点位置移动）
2：多圈回零（找到绝对 0 点）

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果
Byte2：回零模式

◆ **举个例子：**

主机发送 C5 01 92 01 59 5C
从机操作成功，应答 C5 01 92 01 01 5A 5C
下行帧的指令数据 Byte1：回零模式：1：就近回零
上行帧的指令数据 Byte2：回零模式：1：就近回零

4.5.4 强制中断并退出回零操作指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	93H	无	59H	5CH

上行帧(上位机<--驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	93H	Byte1	CHECKSUM	5CH

该指令用于强制中断并退出回零操作。

举个例子：

主机发送 C5 01 93 59 5C

从机操作成功，应答 C5 01 93 01 5A 5C

4.5.5 读取回零参数指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	94H	无	5AH	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	94H	Byte1	…	Byte18	CHECKSUM	5CH

该指令用于读取回零参数。

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示上电自动回零标志，有以下可能：

0：无需上电自动回零　　1：上电自动回零

Byte3：表示回零状态，有以下可能：

0：空闲态

1：正在运行

2：回零完成

3：回零失败未找到零点

Byte4~ Byte5：表示限位电流，范围：0~3000（单位：mA）（类型 int16_t）

Byte6~ Byte9：表示左限位原点，（类型 int32_t）

Byte10~ Byte13：表示超时时间（单位：ms）（类型 uint32_t）

Byte14~ Byte17：表示右限位原点，（类型 int32_t）

Byte18：表示左右限位开关状态：

◆ **举个例子：**

主机发送 C5 01 94 5A 5C

从机操作成功，应答 C5 01 94 01 00 00 01 2C 00 00 C8 00 00 00 27 10 00 00 C8 00 01 50 5C

上行帧的指令数据：

Byte2：00 表示上电自动回零标志，0：无需上电自动回零

Byte3：00 表示回零状态，0：空闲态

Byte4~ Byte5：01 2C 表示限位电流，300mA

Byte6~ Byte9：00 00 C8 00 表示左限位原点，51200

Byte10~ Byte13：00 00 27 10 表示超时时间，10000ms

Byte14~ Byte17：00 00 C8 00 表示右限位原点，51200

Byte18：01 表示开启左右限位

4.5.6 修改原点回零超时时间指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	95H	Byte1	…	Byte4	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	95H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于修改原点回零超时时间（单位：ms）（类型 uint32_t）。

◆ **下行帧的指令数据：**

Byte1~ Byte4：原点回零超时时间

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：原点回零超时时间

◆ **举个例子：**

主机发送 C5 01 95 00 00 13 88 F6 5C

从机操作成功，应答 C5 01 95 01 00 00 13 88 F7 5C

下行帧的指令数据：

Byte1~ Byte4：点回零超时时间为 5000ms

上行帧的指令数据：

Byte2~ Byte5：点回零超时时间为 5000ms

4.5.7 读取回零状态指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	96H	无	5CH	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	96H	Byte1	Byte2	CHECKSUM	5CH

该指令用于读取回零状态。

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示回零状态，有以下可能：

0：空闲态

1：正在运行

2：回零完成

3：回零失败未找到零点

◆ **举个例子：**

主机发送 C5 01 96 5C 5C

从机操作成功，应答 C5 01 96 01 00 5D 5C

上行帧的指令数据：Byte2：0x00 表示空闲态

4.5.8 设置上电自动回零指令

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	97H	Byte1	5EH	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	97H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置上电是否自动回零（**该零点对应左限位原点**）。

◆ **下行帧的指令数据：**

Byte1：是否上电自动回零，有以下可能：

0：上电不自动回零

1：上电自动回零

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示上电自动回零标志

◆ **举个例子：**

主机发送 C5 01 97 01 5E 5C

从机操作成功，应答 C5 01 97 01 01 5F 5C

下行帧的指令数据 Byte1：1：上电自动回零

上行帧的指令数据 Byte2：1：上电自动回零

4.5.9 设置右限位原点指令

下行帧(上位机-->驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	98H	Byte1	…	Byte4	CHECKSUM	5CH

上行帧(上位机<--驱动板)							
帧头	从机地址	功能码	指令数据			检验码	帧尾
C5H	01H	98H	Byte1	…	Byte5	CHECKSUM	5CH

该指令用于设置回零零点位置（类型 int32_t）。

◆ **下行帧的指令数据：**

Byte1~ Byte4：右限位原点位置

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2~ Byte5：右限位原点位置

◆ **举个例子：**

主机发送 C5 01 98 00 00 C8 00 26 5C

从机操作成功，应答 C5 01 98 01 00 00 C8 00 27 5C

下行帧的指令数据 Byte1~ Byte4：设置回零零点位置为 51200

上行帧的指令数据 Byte2~ Byte5：回零零点位置为 51200

4.5.10 设置左右限位开关状态

下行帧(上位机-->驱动板)					
帧头	从机地址	功能码	指令数据	检验码	帧尾
C5H	01H	99H	Byte1	5EH	5CH

上行帧(上位机<--驱动板)						
帧头	从机地址	功能码	指令数据		检验码	帧尾
C5H	01H	99H	Byte1	Byte2	CHECKSUM	5CH

该指令用于设置左右限位是否开启，开启后电机运动范围受限于左右限位原点。

◆ **下行帧的指令数据：**

Byte1：是否开启左右限位开关，有以下可能：

0：关闭左右限位

1：开启左右限位

◆ **上行帧的指令数据：**

Byte1：表示指令操作结果

Byte2：表示左右限位开关标志

◆ **举个例子：**

主机发送 C5 01 99 01 60 5C

从机操作成功，应答 C5 01 99 01 01 61 5C

下行帧的指令数据 Byte1：1：开启左右限位

上行帧的指令数据 Byte2：1：开启左右限位

5，Modbus-RTU 指令使用说明

注意事项：

1. 协议使能方法

需要使能 modbus 协议，方可使用，下面两个方法效果一样：

- 方法 1：屏幕菜单→MODBUS→使能
- 方法 2：使用“设置 modbus 指令”，设置使用 mudbus 协议

2. 协议通用性说明

本协议适用于以下两种通信接口版本的设备：

- 串口 TTL 版本
- RS485 版本

其中，协议的指令格式与数据内容在三种版本中保持一致，仅在物理层和部分收发规则上存在差异。

3. 串口 TTL 与 RS485 默认通信参数

- 波特率 115200、
- 停止位 1、
- 数据位 8、
- 检验位 None

4. 从机地址

- 出厂默认从机地址为 01H
- 文档中示例均以 01H 作为从机地址。

在 PD42S1 步进电机闭环驱动器中，会使用到 Modbus-rtu 协议的 04H、06H、10H 三个功能码，请看 5.1 指令格式说明。

功能码表：

指令类型	Modbus-rtu 协议功能码	Modbus-rtu 寄存器起始地址（实际对应自定义协议的功能码）	功能说明
系统指令(0x00 ~ 0x0F)	06H	0x01	校准编码器
		0x02	复位重启
		0x03	恢复出厂设置
		0x04	参数保存
读参数指令 (0x20 ~ 0x3F)	04H	0x20	读取软硬件版本
		0x21	读取电机磁链
		0x22	读取相电阻和相电感
		0x23	读取相电流
		0x24	读取总线电压
		0x25	读取电流环 PID 参数
		0x26	读取速度环 PID 参数
		0x27	读取位置环 PID 参数
		0x28	读取输入累计脉冲值
		0x29	读取电机实时转速
		0x2A	读取电机实时位置
		0x2B	读取电机位置误差
		0x2C	读取电机运行状态
		0x2D	读取堵转标志
		0x2E	读取堵转电流
		0x2F	读取使能状态
		0x30	读取到位标志
		0x31	读取系统参数
		0x32	读取驱动参数
设置参数指令 (0x60 ~ 0x7F)	06H	0x60	设置从机地址
		0x62	设置工作模式
	10H	0x63	设置位置环 PID 参数
	06H	0x64	设置位置环最大力矩限制
		0x65	设置细分
		0x66	设置目标电流
	10H	0x67	设置串口波特率
	06H	0x69	设置 Modbus 协议
		0x6A	设置堵转保护功能
		0x6B	设置堵转电流
		0x6D	设置 DIR 正转电平
		0x6E	设置 EN 脚有效电平

		0x70	设置按键锁定
		0x71	设置自动熄屏
		0x72	设置 IO 启动电平
限位回零指令 (0x90 ~ 0x9F)	10H	0x90	设置回零零点位置
		0x91	设置有无限位回零
	06H	0x92	触发回零
		0x93	强制中断并退出回零操作
	04H	0x94	读取回零参数
	10H	0x95	修改原点回零超时时间
	04H	0x96	读取回零状态
	06H	0x97	设置上电自动回零
运动控制指令 (0xF0 ~ 0xFF)	10H	0xE0	开环速度模式控制
		0xE1	开环绝对位置模式控制
		0xE2	开环相对位置模式控制
	06H	0xE3	开环脉冲模式控制
	10H	0xE4	IO 启停模式
	10H	0xF0	力矩模式控制
		0xF1	速度模式控制
		0xF2	绝对位置模式控制
		0xF3	相对位置模式控制
	06H	0xF4	脉冲模式控制
	10H	0xF5	脉宽位置模式控制
		0xF6	脉宽力矩模式控制
		0xF7	脉宽速度模式控制
	06H	0xF8	将当前位置角度清零
		0xF9	解除堵转保护
		0xFA	电机使能控制
		0xFB	清除状态（堵转、刹车、失能）
		0xFC	立即停止（刹车）

5.1 指令格式说明

Modbus-rtu 04H 功能码指令格式，如表 5.1.1 所示：

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
addr	04H	00H	00H	00H	00H	CRC16_H	CRC16_L
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据		寄存器 n 数据	CRC16 校验码	
			高字节	低字节	高/低字节	高字节	低字节
addr	04H	00H	00H	00H	…	CRC16_H	CRC16_L

表 5.1.1 Modbus-rtu 04H 功能码指令格式说明

Modbus-rtu 06H 功能码指令格式，如表 5.1.2 所示：

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
addr	06H	00H	00H	00H	00H	CRC16_H	CRC16_L
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
addr	06H	00H	00H	00H	00H	CRC16_H	CRC16_L

表 5.1.2 Modbus-rtu 06H 功能码指令格式说明

Modbus-rtu 10H 功能码指令格式，如表 5.1.3 所示：

下行帧(上位机-->驱动板)												
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1 数据		寄存器...n 数据		CRC16 校验码	
		高字节	低字节	高字节	低字节		高字节	低字节	高字节	低字节	高字节	低字节
addr	10H	00H	00H	00H	00H	00H	00H	00H	00H	00H	CRC16_H	CRC16_L
上行帧(上位机<--驱动板)												
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码						
		高字节	低字节	高字节	低字节	高字节	低字节					
addr	10H	00H	00H	00H	00H	CRC16_H	CRC16_L					

表 5.1.3 Modbus-rtu 10H 功能码指令格式说明

1，从机地址范围：0~255，其中 0 为广播地址，1~255 为从机地址（默认地址为 1）

2，功能码：用到的 Modbus-rtu 协议功能码有：04H、06H、10H

3，寄存器起始：对应自定义协议的功能码(高字节没用到，补 0 即可)

4，校验码：采用的校验方式是 CRC16。

5，**时序要求（单条指令内）**：主机发送指令时，整帧数据（从 0xC5 到 0x5C）必须 **连续输出**，字节之间不允许超过 1 个字节周期的延时，否则可能导致从机接收失败。

6，**时序要求（指令间隔）**：两条完整指令之间应保持 **至少 2ms 的时间间隔**，以保证从机有足够时间完成上一条指令的解析与处理。

注意：

- **使用广播地址发送命令，从机均不应答。**
- **主机发送的从机地址，如果总线上没有匹配的从机，也不应答。**

为了方便大家熟悉和查找相关指令，我们把指令分为了四类：系统类指令、参数类指令、运动类指令、回零类指令。详细的指令描述请看下文。

5.2 系统类指令说明

5.2.1 校准编码器指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	01H	00H	01H	19H	CAH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	01H	00H	01H	19H	CAH

该指令用于校准编码器。**注意：校准编码器前，请确保电机是空载状态!!!**

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 01 00 01 19 CA

上行帧数据为：01 06 00 01 00 01 19 CA

5.2.2 复位重启指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	02H	00H	01H	E9H	CAH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	02H	00H	01H	E9H	CAH

该指令用于复位重启驱动板，不会修改配置参数。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 02 00 01 E9 CA

上行帧数据为：01 06 00 02 00 01 E9 CA

5.2.3 恢复出厂设置指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	03H	00H	01H	B8H	0AH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	03H	00H	01H	B8H	0AH

该指令用于恢复出厂设置，把配置参数修改为出厂默认值。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 03 00 01 B8 0A

上行帧数据为：01 06 00 03 00 01 B8 0A

5.2.4 参数保存指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	04H	00H	01H	09H	CBH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	04H	00H	01H	09H	CBH

该指令用于保存所有掉电不丢失的配置参数。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 04 00 01 09 CB

上行帧数据为：01 06 00 04 00 01 09 CB

5.3 参数类指令说明

5.3.1 读取软硬件版本指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	20H	00H	01H	30H	00H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取驱动器软硬件版本号。

◆ **举个例子：**

下行帧数据为：01 04 00 20 00 01 30 00

上行帧数据为：01 04 02 0A 0A 3F 97

Byte1：0AH(即十进制 10)，表示固件版本号为 V1.0

Byte2：0AH(即十进制 10)，表示硬件版本号为 V1.0

5.3.2 读取电机磁链指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	21H	00H	02H	21H	C1H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据		寄存器 2 数据	CRC16 校验码	
			高/低字节		高/低字节	高字节	低字节
01H	04H	04H	Byte1~ Byte4			CRC16_H	CRC16_L

该指令用于读取电机磁链（单位 mWb）。（float → uint8_t[4]）

◆ **举个例子：**

下行帧数据为：01 04 00 21 00 02 21 C1

上行帧数据为：01 04 04 40 97 0A 3D 98 D9

Byte1~ Byte4：寄存器数据我们做了处理，采用**大端序输出**。即 0x40970A3D 表示电机磁链参数为 4.72mWb。

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.3 读取相电阻和相电感指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	22H	00H	04H	51H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1~4 数据			CRC16 校验码	
			1~4 组高/低字节			高字节	低字节
01H	04H	08H	Byte1~ Byte8			CRC16_H	CRC16_L

该指令用于读取电机相电阻和相电感(含 MOS 内阻)。（float → uint8_t[4]）

Byte1~ Byte4：表示电机相电阻参数（单位Ω）

Byte5~ Byte8：表示电机相电感参数（单位 mH）

◆ **举个例子：**

下行帧数据为：01 04 00 22 00 04 51 C3

上行帧数据为：01 04 08 3F DC D7 D3 40 88 02 D4 79 B4

Byte1~ Byte4：0x3FDCD7D3 表示电机相电阻参数为 1.725Ω

Byte5~ Byte8：0x408802D4 表示电机相电感参数为 4.250mH

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.4 读取相电流指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	23H	00H	01H	C0H	00H

上行帧(上位机<--驱动板)						
从机地址	功能码	字节数	寄存器 1 数据		CRC16 校验码	
			高字节	低字节	高字节	低字节
01H	04H	02H	Byte1	Byte2	CRC16_H	CRC16_L

该指令用于读取电机相电流（单位：mA）。（类型：int16_t）

◆ **举个例子：**

下行帧数据为：01 04 00 23 00 01 C0 00

上行帧数据为：01 04 02 00 6C B9 1D

Byte1~ Byte2：0x006C 表示电机相电流为 108mA

5.3.5 读取总线电压指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	24H	00H	02H	31H	C0H

上行帧(上位机<--驱动板)						
从机地址	功能码	字节数	寄存器 1 数据	寄存器 2 数据	CRC16 校验码	
			高/低字节	高/低字节	高字节	低字节
01H	04H	04H	Byte1~ Byte4		CRC16_H	CRC16_L

该指令用于读取驱动器总线电压（单位：V）。（float → uint8_t[4]）

◆ **举个例子：**

下行帧数据为：01 04 00 24 00 02 31 C0

上行帧数据为：01 04 04 41 BF C7 C0 8C 3C

Byte1~ Byte4：0x41BFC7C0 表示驱动器总线电压为 23.97V

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.6 读取电流环 PID 参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	25H	00H	08H	E0H	07H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1~8 数据			CRC16 校验码	
			1~8 组高/低字节			高字节	低字节
01H	04H	10H	Byte1~ Byte16			CRC16_H	CRC16_L

该指令用于读取电流环 PID 参数。（float → uint8_t[4]）

Byte1~ Byte4：表示 Q 轴 PI 参数 P

Byte5~ Byte8：表示 Q 轴 PI 参数 I

Byte9~ Byte12：表示 D 轴 PI 参数 P

Byte13~ Byte16：表示 D 轴 PI 参数 I

◆ **举个例子：**

下行帧数据为：01 04 00 25 00 08 E0 07

上行帧数据为：01 04 10 37 27 C5 AC 35 86 37 BD 37 27 C5 AC 35 86 37 BD BB 15

Byte1~ Byte4： 0x3727C5AC 表示 Q 轴 PI 参数 p 为 0.00001

Byte5~ Byte8：0x358637BD 表示 Q 轴 PI 参数 I 为 0.000001

Byte9~ Byte12：0x3727C5AC 表示 D 轴 PI 参数 P 为 0.00001

Byte13~ Byte16：0x358637BD 表示 D 轴 PI 参数 I 为 0.000001

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.7 读取速度环 PID 参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	26H	00H	06H	91H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1~6 数据			CRC16 校验码	
			1~6 组高/低字节			高字节	低字节
01H	04H	0CH	Byte1~ Byte12			CRC16_H	CRC16_L

该指令用于读取速度环 PID 参数。（类型：uint32_t）

Byte1~ Byte4：表示 PID 参数 P

Byte5~ Byte8：表示 PID 参数 I

Byte9~ Byte12：表示 PID 参数 D

◆ **举个例子：**

下行帧数据为：01 04 00 26 00 06 91 C3

上行帧数据为：01 04 0C 00 00 00 03 00 00 00 04 00 00 00 00 70 87

Byte1~ Byte4：0x00000004 表示 Q 轴 PI 参数 p 为 3

Byte5~ Byte8：0x00000004 表示 Q 轴 PI 参数 I 为 4

Byte9~ Byte12：0x00000000 表示 D 轴 PI 参数 P 为 0

5.3.8 读取位置环 PID 参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	27H	00H	06H	C0H	03H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1~6 数据			CRC16 校验码	
			1~6 组高/低字节			高字节	低字节
01H	04H	0CH	Byte1~ Byte12			CRC16_H	CRC16_L

该指令用于读取位置环 PID 参数。（类型：uint32_t）

Byte1~ Byte4：表示 PID 参数 P

Byte5~ Byte8：表示 PID 参数 I

Byte9~ Byte12：表示 PID 参数 D

◆ **举个例子：**

下行帧数据为：01 04 00 27 00 06 C0 03

上行帧数据为：01 04 0C 00 00 01 2C 00 00 00 1E 00 00 01 7C A0 C0

Byte1~ Byte4：0x0000012C 表示 Q 轴 PI 参数 p 为 300

Byte5~ Byte8：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte9~ Byte12：0x0000017C 表示 D 轴 PI 参数 P 为 380

5.3.9 读取输入累计脉冲数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	28H	00H	02H	F1H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据		寄存器 2 数据	CRC16 校验码	
			高/低字节		高/低字节	高字节	低字节
01H	04H	04H	Byte1~ Byte4			CRC16_H	CRC16_L

该指令用于读取输入累计脉冲数。（类型：uint32_t）

◆ **举个例子：**

下行帧数据为：01 04 00 28 00 02 F1 C3

上行帧数据为：01 04 04 00 00 C8 00 AC 44

Byte1~ Byte4：0x0000C800 表示输入累计脉冲数为 51200

5.3.10 读取电机实时转速指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	29H	00H	01H	E0H	02H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取电机实时转速（单位：RPM）。（类型：int16_t）

◆ **举个例子：**

下行帧数据为：01 04 00 29 00 01 E0 02

上行帧数据为：01 04 02 00 1F F8 F8

Byte1~ Byte2：0x001F 表示电机实时转速为 31RPM

5.3.11 读取电机实时位置指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2AH	00H	02H	50H	03H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据		寄存器 2 数据	CRC16 校验码	
			高/低字节		高/低字节	高字节	低字节
01H	04H	04H	Byte1~ Byte4			CRC16_H	CRC16_L

该指令用于读取电机实时位置（51200 代表一圈）。（类型：int32_t）

◆ **举个例子：**

下行帧数据为：01 04 00 2A 00 02 50 03

上行帧数据为：01 04 04 00 07 D0 00 17 85

Byte1~ Byte4：0x0007D000 表示电机实时位置为 512000

4.3.12 读取电机位置误差指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2BH	00H	02H	01H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据		寄存器 2 数据	CRC16 校验码	
			高/低字节		高/低字节	高字节	低字节
01H	04H	04H	Byte1~ Byte4			CRC16_H	CRC16_L

该指令用于读取电机位置误差（51200 代表一圈）。（类型：int32_t）

◆ **举个例子：**

下行帧数据为：01 04 00 2B 00 02 01 C3

上行帧数据为：01 04 04 00 00 00 00 FB 84

Byte1~ Byte4：0x00000000 表示电机实时位置为 0

5.3.13 读取电机运行状态指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2CH	00H	01H	F0H	03H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取电机运行状态。（类型：uint8_t）

Byte1：没用到，补 0 即可

Byte2：表示电机运行状态，有以下可能：

0：停止状态

1：任务完成状态

2：正在运行

3：过载状态

4：堵转状态

5：欠压状态

◆ **举个例子：**

下行帧数据为：01 04 00 2C 00 01 F0 03

上行帧数据为：01 04 02 00 01 78 F0

Byte2：0x01 表示电机在任务完成状态

5.3.14 读取电机堵转标志指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2DH	00H	01H	A1H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取电机堵转标志。（类型：uint8_t）

Byte1：没用到，补 0 即可

Byte2：表示电机堵转标志，有以下可能：

0：未堵转　　1：堵转

◆ **举个例子：**

下行帧数据为：01 04 00 2D 00 01 A1 C3

上行帧数据为：01 04 02 00 00 B9 30

Byte2：0x00 表示电机未堵转

5.3.15 读取堵转电流指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2EH	00H	01H	51H	C3H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取堵转电流（单位：mA）。（类型：int16_t）

◆ **举个例子：**

下行帧数据为：01 04 00 2E 00 01 51 C3

上行帧数据为：01 04 02 05 DC BB F9

Byte1~ Byte2：0x05DC 表示堵转电流 1500mA

5.3.16 读取电机使能状态指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	2FH	00H	01H	00H	03H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节		低字节	高字节	低字节
01H	04H	02H	Byte1		Byte2	CRC16_H	CRC16_L

该指令用于读取电机使能状态。（类型：uint8_t）

Byte1：没用到，补 0

Byte2：表示电机使能状态，有以下可能：

0：使能电机

1：失能电机

◆ **举个例子：**

下行帧数据为：01 04 00 2F 00 01 00 03

上行帧数据为：01 04 02 00 00 B9 30

Byte2：0x00 表示失能电机

5.3.17 读取电机到位标志指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	30H	00H	01H	31H	C5H

上行帧(上位机<--驱动板)						
从机地址	功能码	字节数	寄存器 1 数据		CRC16 校验码	
			高字节	低字节	高字节	低字节
01H	04H	02H	Byte1	Byte2	CRC16_H	CRC16_L

该指令用于读取电机到位标志。（类型：uint8_t）

Byte1：没用到，补 0

Byte2：表示电机到位标志，有以下可能：

0：未到位

1：到位

◆ **举个例子：**

下行帧数据为：01 04 00 30 00 01 31 C5

上行帧数据为：01 04 02 00 01 78 F0

Byte2：0x01 表示电机到位

5.3.18 读取系统参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	31H	00H	14H	A1H	CAH

上行帧(上位机<--驱动板)					
从机地址	功能码	字节数	寄存器 1~20 数据	CRC16 校验码	
			1~20 组高/低字节	高字节	低字节
01H	04H	28H	Byte1~ Byte40	CRC16_H	CRC16_L

该指令用于读取系统参数，这部分参数都是只读属性。

◆ **上行帧的指令数据：**

Byte1~ Byte4：电机总线电压（单位 V）（float → uint8_t[4]）

Byte5~ Byte6：读取相电流（单位 mA）（类型 int16_t）

Byte7~ Byte10：电机磁链参数（单位 mWb）（float → uint8_t[4]）

Byte11~ Byte14：电机相电阻参数（单位Ω）（float → uint8_t[4]）

Byte15~ Byte18：电机相电感参数（单位 mH）（float → uint8_t[4]）

Byte19~ Byte20：读取实时转速（单位 RPM）（类型 int16_t）

Byte21~ Byte24：读取目标位置（51200 代表一圈）（类型 int32_t）

Byte25~ Byte28：读取实时位置（51200 代表一圈）（类型 int32_t）

Byte29~ Byte32：读取位置误差（51200 代表一圈）（类型 int32_t）

Byte33~ Byte36：读取脉冲数（类型 uint32_t）

Byte37：电机使能状态

0：电机使能

1：电机失能

Byte38：电机到位标志，有以下可能：

0：未到位

1：到位

Byte39：电机堵转标志，有以下可能：

0：未堵转

1：堵转

Byte40：地址模式，有以下可能：

0：从机地址模式

1：分组地址模式

◆ **举个例子：**

下行帧数据为：01 04 00 31 00 14 A1 CA

上行帧数据为：01 04 28 41 C0 00 00 00 0F 40 88 5D 39 3F ED EB 28 40 8D 27 10 00 00 00 07 D0 00 00 07 D0 00 00 00 00 00 00 00 7D 02 00 01 00 00 B6 0E

Byte1~ Byte4：电机总线电压 24.0V

Byte5~ Byte6：读取相电流 15mA

Byte7~ Byte10：电机磁链 4.26mWb

Byte11~ Byte14：电机相电阻 1.86Ω

Byte15~ Byte18：电机相电感 4.41mH

Byte19~ Byte20：读取实时转速 0RPM

Byte21~ Byte24：读取目标位置（51200 代表一圈）512000

Byte25~ Byte28：读取实时位置（51200 代表一圈）512000

Byte29~ Byte32：读取位置误差（51200 代表一圈）0

Byte33~ Byte36：读取脉冲数 32002

Byte37：0：电机使能

Byte38：1：到位

Byte39：0：未堵转

Byte40：0：从机地址模式

4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.19 读取驱动参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	32H	00H	10H	50H	09H

上行帧(上位机<--驱动板)					
从机地址	功能码	字节数	寄存器 1~16 数据	CRC16 校验码	
			1~16 组高/低字节	高字节	低字节
01H	04H	20H	Byte1~ Byte32	CRC16_H	CRC16_L

该指令用于读取全部配置参数。

◆ **上行帧的指令数据：**

Byte1：电机工作模式，有以下可能：（类型 uint8_t）

0：通信位置模式

1：通信速度模式

2：通信力矩模式

3：脉冲模式

4：脉宽位置模式

5：脉宽速度模式

6：脉宽力矩模式
7：回零模式

Byte2：指令回响状态，有以下可能：
0：正常回响
1：不回响

Byte3~ Byte6 串口波特率（单位 bps）（类型 uint32_t）
Byte7~ Byte8：CAN 波特率（单位 Kbps）（类型 uint16_t）
Byte9：DIR 正转电平，有以下可能：
0：高电平正转
1：低电平正转
Byte10：EN 脚有效电平，有以下可能：
0：EN 引脚低电平使能
1：EN 引脚高电平使能
2：EN 引脚保持有效
Byte11~ Byte12：设置的细分（类型 uint16_t）
Byte13~ Byte14：位置环最大力矩（单位：mA）（类型 int32_t）
Byte15~ Byte18：位置环 PID 参数 P（类型 uint32_t）
Byte19~ Byte22：位置环 PID 参数 I（类型 uint32_t）
Byte23~ Byte26：位置环 PID 参数 D（类型 uint32_t）
Byte27~ Byte28：堵转电流（单位：mA）（类型 int16_t）
Byte29：电机堵转保护功能标志
0：关闭堵转保护功能
1：开启堵转保护功能
Byte30：按键上锁状态，有以下可能：
0：解锁
1：上锁
Byte31~ Byte34：速度环 PID 参数 P（类型 uint32_t）
Byte35~ Byte38：速度环 PID 参数 I（类型 uint32_t）
Byte39~ Byte42：速度环 PID 参数 D（类型 uint32_t）
Byte43：自动熄屏状态，有以下可能：
0：关闭自动熄屏
1：开启自动熄屏
Byte44：IO 启停电平，有以下可能：
0：低电平启动
1：高电平启动

◆ **举个例子：**

下行帧数据为：01 04 00 32 00 10 50 09

上行帧数据为：01 04 20 03 00 00 01 C2 00 01 F4 00 01 00 10 0B B8 00 00 01 2C 00 00 00 1E 00 00 01 7C 05 DC 01 00 00 EB A3

Byte1：3：脉冲模式
Byte2：0：正常回响
Byte3~ Byte4：串口波特率为：115200
Byte7~ Byte8：CAN 速率为：500K
Byte9：0：高电平正转

Byte10：1：EN 引脚高电平使能
Byte11~ Byte12：细分为：16
Byte13~ Byte14：位置环最大力矩：3000
Byte15~ Byte18：PID 参数 P 为 300
Byte19~ Byte22：PID 参数 I 为 30
Byte23~ Byte26：PID 参数 D 为 380
Byte27~ Byte28：堵转电流 1500 mA
Byte29：1：开启堵转保护功能
Byte30：0：按键解锁
Byte31：保留（未用到补 0）
Byte32：0：关闭自动熄屏
4.3.2 小节有介绍：STM32（小端系统）联合体存取 float 和 uint32_t 数据简明指南。

5.3.20 设置从机地址指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	60H	00H	02H	08H	15H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	60H	00H	02H	08H	15H

该指令用于设置从机地址（范围：1~255）。**注意：总线上不能设置两个一样的从机地址，应答会冲突。**寄存器数据高字节没有用到，默认补 00H（后续不再赘述）。

◆ **举个例子：**

下行帧数据为：01 06 00 60 00 02 08 15
上行帧数据为：01 06 00 60 00 02 08 15
注意：下次再想跟这个从机通信，注意从机地址要改成 2 了！！！

5.3.21 设置工作模式指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	62H	00H	01H	E9H	D4H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	62H	00H	01H	E9H	D4H

该指令用于设置工作模式，有以下可能：

0x00：通信位置模式
0x01：通信速度模式
0x02：通信力矩模式

0x03：脉冲模式
0x04：脉宽位置模式
0x05：脉宽速度模式
0x06：脉宽力矩模式
0x07：回零模式

◆ **举个例子：**

下行帧数据为：01 06 00 62 00 01 E9 D4
上行帧数据为：01 06 00 62 00 01 E9 D4
新的工作模式为 0x01：通信速度模式

5.3.22 设置位置环 PID 参数指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~6 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~6 组高/低字节	高字节	低字节
01H	10H	00H	63H	00H	06H	0CH	Byte1~ Byte12	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	63H	00H	06H	B0H	15H

该指令用于设置位置环 PID 参数。

◆ **下行帧的寄存器 1~6 数据：**

Byte1~ Byte4：PID 参数 P（类型 uint32_t）
Byte5~ Byte8：PID 参数 I（类型 uint32_t）
Byte9~ Byte12：PID 参数 D（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 63 00 06 0C 00 00 01 2C 00 00 00 1E 00 00 01 7C 57 79
上行帧数据为：01 10 00 63 00 06 B0 15
设置的位置 PID 参数为：
Byte1~ Byte4：0x0000012C 表示 Q 轴 PI 参数 p 为 300
Byte5~ Byte8：0x0000001E 表示 Q 轴 PI 参数 I 为 30
Byte9~ Byte12：0x0000017C 表示 D 轴 PI 参数 P 为 380

5.3.23 设置速度/位置环最大力矩限制指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	64H	05H	DCH	CAH	DCH

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	64H	05H	DCH	CAH	DCH

该指令用于设置速度/位置环最大力矩限制（范围：0~3000）（单位：mA）。

◆ **举个例子：**

下行帧数据为：01 06 00 64 05 DC CA DC

上行帧数据为：01 06 00 64 05 DC CA DC

设置位置环最大力矩限制为 0x05DC（即 1500mA）

5.3.24 设置细分指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	65H	00H	10H	98H	19H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	65H	00H	10H	98H	19H

该指令用于设置细分（范围：1~256）。

◆ **举个例子：**

下行帧数据为：01 06 00 65 00 10 98 19

上行帧数据为：01 06 00 65 00 10 98 19

设置细分为 16

5.3.25 设置目标电流指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	66H	05H	DCH	6BH	1CH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	66H	05H	DCH	6BH	1CH

该指令用于设置目标电流（范围：-3000~3000）（单位：mA）。

◆ **举个例子：**

下行帧数据为：01 06 00 66 05 DC 6B 1C

上行帧数据为：01 06 00 66 05 DC 6B 1C

设置目标电流为 0x05DC（即 1500mA）

5.3.26 设置串口波特率指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~2 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~2 组高/低字节	高字节	低字节
01H	10H	00H	67H	00H	02H	04H	Byte1~ Byte4	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	67H	00H	02H	F0H	17H

该指令用于设置串口波特率（范围：0~3000000）（默认：115200）（单位 bps）。

◆ **下行帧的寄存器 1~2 数据：**

Byte1~ Byte4：串口波特率

◆ **举个例子：**

下行帧数据为：01 10 00 67 00 02 04 00 01 C2 00 B4 C1

上行帧数据为：01 10 00 67 00 02 F0 17

设置串口波特率为 115200 bps

5.3.27 设置 moubus 指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	69H	00H	01H	98H	16H

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	69H	00H	01H	98H	16H

该指令用于设置使用正点原子自定义协议（默认）还是 moubus-rtu 协议，有以下可能：

0：使用正点原子自定义协议　　1：使用 moubus-rtu 协议

◆ **举个例子：**

下行帧数据为：01 06 00 69 00 01 98 16

上行帧数据为：01 06 00 69 00 01 98 16

寄存器数据为 1：使用 moubus-rtu 协议

注意：如果设置为 0，那么 moubus-rtu 协议马上就不能通信了，需要通过屏幕菜单栏设置 modbus：使能；或者恢复出厂设置；又或者用正点原子自定义协议指令设置回来!!!

5.3.28 设置堵转保护功能指令

下行帧(上位机-->驱动板)

从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6AH	00H	01H	68H	16H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6AH	00H	01H	68H	16H

该指令用于设置堵转保护功能，有以下可能：

0：关闭堵转保护功能

1：开启堵转保护功能

◆ **举个例子：**

下行帧数据为：01 06 00 6A 00 01 68 16

上行帧数据为：01 06 00 6A 00 01 68 16

寄存器数据为 1：开启堵转保护功能

5.3.29 设置堵转电流指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6BH	0BH	B8H	FFH	54H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6BH	0BH	B8H	FFH	54H

该指令用于设置堵转电流（范围：0~3000）（单位：mA）。

◆ **举个例子：**

下行帧数据为：01 06 00 6B 0B B8 FF 54

上行帧数据为：01 06 00 6B 0B B8 FF 54

设置堵转电流为 3000mA

5.3.30 电机正反转 DIR 控制指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6DH	00H	01H	D9H	D7H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6DH	00H	01H	D9H	D7H

该指令用于设置 DIR 引脚控制电机正反转的有效电平(仅用于脉冲模式)，有以下可能：

0：DIR 引脚高电平正转

1：DIR 引脚低电平正转

◆ **举个例子：**

下行帧数据为：01 06 00 6D 00 01 D9 D7

上行帧数据为：01 06 00 6D 00 01 D9 D7

寄存器数据为 1：DIR 引脚低电平正转

5.3.31 设置 EN 引脚有效电平指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6EH	00H	01H	D9H	D7H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6DH	00H	01H	D9H	D7H

该指令设置 EN 引脚有效电平（仅用于脉冲模式），有以下可能：

0：EN 引脚低电平使能

1：EN 引脚高电平使能

2：EN 引脚保持有效

◆ **举个例子：**

下行帧数据为：01 06 00 6E 00 01 29 D7

上行帧数据为：01 06 00 6E 00 01 29 D7

寄存器数据为 1：EN 引脚高电平使能

5.3.32 设置按键锁定指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	70H	00H	01H	49H	D1H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	6DH	00H	01H	49H	D1H

该指令用于设置按键锁定，有以下可能：

0：按键解锁

1：按键上锁

◆ **举个例子：**

下行帧数据为：01 06 00 70 00 01 49 D1

上行帧数据为：01 06 00 70 00 01 49 D1

寄存器数据为 1：按键上锁

注意：如果设置为 1，那么按键功能马上失效，需要通过屏幕菜单栏选择恢复出厂设置

或者发送设置按键解锁指令，方可恢复按键功能!!!

5.3.33 设置自动熄屏指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	71H	00H	01H	18H	11H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	71H	00H	01H	18H	11H

该指令用于设置自动熄屏，长时间无操作会自动熄屏，有以下可能：

0：关闭自动熄屏

1：开启自动熄屏

◆ **举个例子：**

下行帧数据为：01 06 00 71 00 01 18 11

上行帧数据为：01 06 00 71 00 01 18 11

寄存器数据为 1：开启自动熄屏

注意：如果设置为 1，那么长时间无操作时会自动熄屏，可通过屏幕菜单栏自动熄屏或恢复出厂设置，也可发送关闭自动熄屏指令，恢复屏幕常亮!!!

5.3.34 设置 IO 启动电平

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	72H	00H	01H	E8H	11H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	72H	00H	01H	E8H	11H

该指令用于设置 IO 启动电平，有以下可能：

0：低电平启动

1：高电平启动

◆ **举个例子：**

下行帧数据为：01 06 00 72 00 01 E8 11

上行帧数据为：01 06 00 72 00 01 E8 11

寄存器数据为 1：高电平启动

5.3.35 设置速度环 PID 参数指令

下行帧(上位机-->驱动板)						
从机	功能	寄存器起始地址	寄存器数量	字节数	寄存器 1~6 数据	CRC16 校验码

地址	码	高字节	低字节	高字节	低字节		1~6 组高/低字节	高字节	低字节
01H	10H	00H	73H	00H	06H	0CH	Byte1~ Byte12	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	63H	00H	06H	B0H	15H

该指令用于设置位置环 PID 参数。

◆ **下行帧的寄存器 1~6 数据：**

Byte1~ Byte4：PID 参数 P（类型 uint32_t）

Byte5~ Byte8：PID 参数 I（类型 uint32_t）

Byte9~ Byte12：PID 参数 D（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 73 00 06 0C 00 00 01 2C 00 00 00 1E 00 00 01 7C 5A B5

上行帧数据为：01 10 00 73 00 06 B1 D0

设置的位置 PID 参数为：

Byte1~ Byte4：0x0000012C 表示 Q 轴 PI 参数 p 为 300

Byte5~ Byte8：0x0000001E 表示 Q 轴 PI 参数 I 为 30

Byte9~ Byte12：0x0000017C 表示 D 轴 PI 参数 P 为 380

5.4 运动类指令说明

5.4.1 力矩模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~2 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~2 组高/低字节	高字节	低字节
01H	10H	00H	F0H	00H	02H	04H	Byte1~ Byte4	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F0H	00H	02H	41H	FBH

该指令用于力矩模式控制。

◆ **下行帧的寄存器 1~2 数据：**

Byte1：没用到，默认为 0

Byte2：电机旋转方向：0：正转； 1：反转

Byte3~ Byte4：电流，取值范围：0~3000（单位：mA）（类型 uint16_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F0 00 02 04 00 00 03 E8 FC 55

上行帧数据为：01 10 00 F0 00 02 41 FB

Byte2：0：正转

Byte3~ Byte4：电流为 1000mA

5.4.2 速度模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~3 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~3 组高/低字节	高字节	低字节
01H	10H	00H	F1H	00H	03H	06H	Byte1~ Byte6	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节

01H	10H	00H	F1H	00H	03H	D1H	FBH

该指令用于速度模式控制。

◆ **下行帧的寄存器 1~3 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~6000 RPM（float → uint8_t[4]）

◆ **举个例子：**

下行帧数据为：01 10 00 F1 00 03 06 00 64 44 16 00 00 36 7A

上行帧数据为：01 10 00 F1 00 03 D1 FB

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 600.0 RPM，用联合体 float→ uint8_t[4]读取为 0x44160000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

5.4.3 绝对位置模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~4 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~4 组高/低字节	高字节	低字节
01H	10H	00H	F2H	00H	04H	08H	Byte1~ Byte8	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F2H	00H	04H	60H	39H

该指令用于绝对位置模式控制。

◆ **下行帧的寄存器 1~4 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~6000 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F2 00 04 08 00 64 07 D0 00 07 D0 00 05 1E

上行帧数据为：01 10 00 F2 00 04 60 39

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 2000RPM

Byte5~ Byte8：绝对位置为 512000

注意：在绝对位置模式控制下，当前位置角度值到设定的绝对位置值后，重复发送上述指令，电机将不会继续运动，除非把绝对位置值改大，或者将当前位置角度清零后，再发。

5.4.4 相对位置模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~4 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~4 组高/低字节	高字节	低字节
01H	10H	00H	F3H	00H	04H	08H	Byte1~ Byte8	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F3H	00H	04H	31H	F9H

该指令用于相对位置模式控制。

◆ **下行帧的寄存器 1~4 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~6000 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F3 00 04 08 00 64 07 D0 00 07 D0 00 F8 DD

上行帧数据为：01 10 00 F3 00 04 31 F9

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 2000RPM

Byte5~ Byte8：绝对位置为 512000

注意：在相对位置模式控制下，每次设置新的相对位置，当前位置角度值会一直累加，可重复发送指令，让电机继续运动。

5.4.5 脉冲模式控制指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F4H	00H	01H	09H	F8H

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F4H	00H	01H	09H	F8H

该指令用于让电机进入脉冲模式控制。说明：EN 引脚用于使能脉冲模式控制，STP 引脚用于输入脉冲，DIR 用于控制电机旋转方向，涉及到的指令有：设置 DIR 正转电平和设置 EN 脚有效电平。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 F4 00 01 09 F8

上行帧数据为：01 06 00 F4 00 01 09 F8

进入脉冲模式后，具体的接线可以查看第 2 章的接线方法。**脉冲模式控制简明介绍**，请参考 **4.4.5 小节**。

5.4.6 脉宽位置模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~6 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~6 组高/低字节	高字节	低字节
01H	10H	00H	F5H	00H	06H	0CH	Byte1~ Byte12	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F5H	00H	06H	50H	39H

该指令用于脉宽位置模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的位置角度。PWM 占空比和旋转的位置角度对应关系由用户自定义。

◆ **下行帧的寄存器 1~6 数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的位置（注意 51200 为一圈）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的位置（注意 51200 为一圈）（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F5 00 06 0C 09 C4 01 F4 00 00 64 00 00 00 00 00 00 17 CA

上行帧数据为：01 10 00 F5 00 06 50 39

Byte1~ Byte2：高电平脉宽最长长度，取值：2500us

Byte3~ Byte4：高电平脉宽最短长度，取值：500us

Byte5~ Byte8：脉宽最长长度对应的位置，取值：25600（旋转半圈）

Byte9~ Byte12：脉宽最短长度对应的位置，取值：0（未旋转）

◆ **该例子模仿 SG90 舵机来定义，PWM 的占空比和控制关系如下：**

PWM 的频率设置为 50Hz(即周期为 20ms)

高电平时间	占空比	角度
0.5ms	2.5%	0°
1.0ms	5.0%	45°
1.5ms	7.5%	90°
2.0ms	10.0%	135°
2.5ms	12.5%	180°

由 PWM 的占空比和控制关系可以知道：

当角度为 180° 时，对应高电平脉宽最长长度，取值为：2.5ms（即 2500us）

当角度为 0°时，对应高电平脉宽最短长度，取值为：0.5ms（即 500us）

由于驱动器的位置值 51200 为一圈，所以脉宽最长长度对应的位置 180°，对应的取值为：25600（旋转半圈）。脉宽最短长度对应的位置 0°，对应的取值：0（未旋转）。

进入脉宽位置模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：07 脉宽位置模式。

5.4.7 脉宽力矩模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~6 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~6 组高/低字节	高字节	低字节
01H	10H	00H	F6H	00H	06H	0CH	Byte1~ Byte12	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F6H	00H	06H	A0H	39H

该指令用于脉宽力矩模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的力矩。PWM 占空比和旋转的力矩对应关系由用户自定义。

◆ **下行帧的寄存器 1~6 数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的电流（取值范围：0~3000mA）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的电流（取值范围：0~3000mA）（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F6 00 06 0C 0D AC 01 F4 00 00 0B B8 00 00 00 00 DB D1

上行帧数据为：01 10 00 F6 00 06 A0 39

Byte1~ Byte2：高电平脉宽最长长度，取值：3500us

Byte3~ Byte4：高电平脉宽最短长度，取值：500us

Byte5~ Byte8：脉宽最长长度对应的电流，取值：3000mA

Byte9~ Byte12：脉宽最短长度对应的电流，取值：0 mA

◆ **该例子自定义 PWM 的占空比和电流关系如下：**

PWM 的频率设置为 100Hz(即周期为 10ms)

高电平时间	占空比	电流
0.5ms	5%	0 mA
1.0ms	10%	500 mA
1.5ms	15%	1000 mA
2.0ms	20%	1500 mA
2.5ms	25%	2000 mA
3.0ms	30%	2500 mA
3.5ms	35%	3000 mA

由 PWM 的占空比和控制关系可以知道：

当电流为 3000 mA 时，对应高电平脉宽最长长度，取值为：3.5ms（即 3500us）

当角度为 0 mA 时，对应高电平脉宽最短长度，取值为：0.5ms（即 500us）

进入脉宽力矩模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：08 脉宽力矩模式控制。

5.4.8 脉宽速度模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~6 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~6 组高/低字节	高字节	低字节
01H	10H	00H	F7H	00H	06H	0CH	Byte1~ Byte12	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	F7H	00H	06H	F1H	F9H

该指令用于脉宽速度模式控制，也就是通过控制 STP 引脚输入的 PWM 脉宽（即占空比），来控制电机旋转的速度。PWM 占空比和旋转的速度对应关系由用户自定义。

◆ **下行帧的寄存器 1~6 数据：**

Byte1~ Byte2：高电平脉宽最长长度，取值范围：0~50000us（类型 uint16_t）

Byte3~ Byte4：高电平脉宽最短长度，取值取值：0~50000us（类型 uint16_t）

Byte5~ Byte8：脉宽最长长度对应的速度（取值范围：0~6000RPM）（类型 int32_t）

Byte9~ Byte12：脉宽最短长度对应的速度（取值范围：0~6000RPM）（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 F7 00 06 0C 0C E4 01 2C 00 00 0B B8 00 00 00 00 62 26

上行帧数据为：01 10 00 F7 00 06 F1 F9

Byte1~ Byte2：高电平脉宽最长长度，取值：3300us

Byte3~ Byte4：高电平脉宽最短长度，取值：300us

Byte5~ Byte8：脉宽最长长度对应的速度，取值：6000 RPM

Byte9~ Byte12：脉宽最短长度对应的速度，取值：0 RPM

◆ **该例子自定义 PWM 的占空比和速度关系如下：**

PWM 的频率设置为 100Hz(即周期为 10ms)

高电平时间	占空比	电流
0.3ms	3%	0 RPM
0.6ms	6%	300 RPM
0.9ms	9%	600 RPM
1.2ms	12%	900 RPM
1.5ms	15%	1200 RPM
1.8ms	18%	1500 RPM
2.1ms	21%	1800 RPM

2.4ms	24%	2100 RPM
2.7ms	27%	2400 RPM
3.0ms	30%	3000 RPM
3.3ms	33%	3000 RPM

由 PWM 的占空比和控制关系可以知道：

当速度为 3000 RPM 时，对应高电平脉宽最长长度，取值为：3.3ms（即 3300us）

当速度为 0 RPM 时，对应高电平脉宽最短长度，取值为：0.3ms（即 300us）

进入脉宽力矩模式后，具体的接线可以查看第 2 章的接线方法，具体的控制方法可以参考我们提供的 STM32 例程：09 脉宽速度模式控制。

5.4.9 将当前的位置角度清零指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F8H	00H	01H	C9H	FBH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F8H	00H	01H	C9H	FBH

该指令用于将当前位置角度、位置误差和累计脉冲数清零。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 F8 00 01 C9 FB

上行帧数据为：01 06 00 F8 00 01 C9 FB

5.4.10 解除堵转保护指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F9H	00H	01H	98H	3BH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	F9H	00H	01H	98H	3BH

该指令用于解除堵转保护（电机发生堵转后，可以使用该指令解除堵转保护）。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 F9 00 01 98 3B

上行帧数据为：01 06 00 F9 00 01 98 3B

5.4.11 电机使能控制指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FAH	00H	00H	A9H	FBH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FAH	00H	00H	A9H	FBH

该指令用于使能电机，有以下可能：

0：使能电机；1：失能电机

◆ **举个例子：**

下行帧数据为：01 06 00 FA 00 00 A9 FB

上行帧数据为：01 06 00 FA 00 00 A9 FB

0：使能电机

5.4.12 清除状态（堵转、刹车，失能）指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FBH	00H	01H	39H	FBH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FBH	00H	01H	39H	FBH

该指令可以清除状态（包括：堵转、刹车，失能）。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 FB 00 01 39 FB

上行帧数据为：01 06 00 FB 00 01 39 FB

5.4.13 立即停止（刹车）指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FCH	00H	01H	88H	3AH
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	FCH	00H	01H	88H	3AH

该指令可以立即停止（刹车）。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 FC 00 01 88 3A

上行帧数据为：01 06 00 FC 00 01 88 3A

注意：刹停后，请及时清除电机状态，否则电机会严重发烫。

5.4.14 开环速度模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~3 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~3 组高/低字节	高字节	低字节
01H	10H	00H	E0H	00H	03H	06H	Byte1~ Byte6	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	E0H	00H	03H	81H	FEH

该指令用于速度模式控制。

◆ **下行帧的寄存器 1~3 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~400 RPM（float → uint8_t[4]）

◆ **举个例子：**

下行帧数据为：01 10 00 E0 00 03 06 00 64 41 70 00 00 86 F9

上行帧数据为：01 10 00 E0 00 03 81 FE

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 15.0 RPM，用联合体 float→ uint8_t[4]读取为 0x41700000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

5.4.15 开环绝对位置模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~4 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~4 组高/低字节	高字节	低字节
01H	10H	00H	E1H	00H	04H	08H	Byte1~ Byte8	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	E1H	00H	04H	91H	FCH

该指令用于绝对位置模式控制。

◆ **下行帧的寄存器 1~4 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~400 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 E1 00 04 08 00 64 00 64 00 00 C8 00 CA 89

上行帧数据为：01 10 00 E1 00 04 91 FC

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 100RPM

Byte5~ Byte8：绝对位置为 51200

注意：在绝对位置模式控制下，当前位置角度值到设定的绝对位置值后，重复发送上述指令，电机将不会继续运动，除非把绝对位置值改大，或者将当前位置角度清零后，再发。

5.4.16 开环相对位置模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~4 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~4 组高/低字节	高字节	低字节
01H	10H	00H	E2H	00H	04H	08H	Byte1~ Byte8	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	E2H	00H	04H	61H	FCH

该指令用于相对位置模式控制。

◆ **下行帧的寄存器 1~4 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte4：速度，取值范围 0~400 RPM（类型 uint16_t）

Byte5~ Byte8：绝对位置（注意 51200 为一圈）（类型 uint32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 E2 00 04 08 00 64 00 64 00 00 C8 00 CE 8D

上行帧数据为：01 10 00 E2 00 04 61 FC

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 100RPM

Byte5~ Byte8：绝对位置为 51200

注意：在相对位置模式控制下，每次设置新的相对位置，当前位置角度值会一直累加，可重复发送指令，让电机继续运动。

5.4.17 开环脉冲模式控制指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	E3H	00H	01H	B9H	FCH

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	E3H	00H	01H	B9H	FCH

该指令用于让电机进入脉冲模式控制。说明：EN 引脚用于使能脉冲模式控制，STP 引脚用于输入脉冲，DIR 用于控制电机旋转方向，涉及到的指令有：设置 DIR 正转电平和设置 EN 脚有效电平。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 E3 00 01 B9 FC

上行帧数据为：01 06 00 E3 00 01 B9 FC

进入脉冲模式后，具体的接线可以查看第 2 章的接线方法。**脉冲模式控制简明介绍**，请参考 **4.4.5 小节**。

5.4.18 IO 启停模式控制指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~3 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~3 组高/低字节	高字节	低字节
01H	10H	00H	E4H	00H	03H	06H	Byte1~ Byte6	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节

01H	10H	00H	E4H	00H	03H	C0H	3FH

该指令用于速度模式控制。

◆ **下行帧的寄存器 1~3 数据：**

Byte1：电机旋转方向：0：正转；1：反转

Byte2：加减速度，取值范围 0~200，数值越大加减速度越大（0 表示直接启动）

Byte3~ Byte6：速度，取值范围 0.1~400 RPM（float → uint8_t[4]）

◆ **举个例子：**

下行帧数据为：01 10 00 E4 00 03 06 00 64 42 C8 00 00 47 4D

上行帧数据为：01 10 00 E4 00 03 C0 3F

Byte1：0：正转

Byte2：加减速度为 100

Byte3~ Byte4：速度为 100.0 RPM，用联合体 float→ uint8_t[4]读取为 0x42C80000

方法可参考：例程 00 联合体 float 和 uint32_t 类型互换读取方法

5.5 回零类指令说明

5.5.1 设置左限位原点位置指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~2 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~2 组高/低字节	高字节	低字节
01H	10H	00H	90H	00H	02H	04H	Byte1~ Byte4	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	90H	00H	02H	41H	E5H

该指令用于设置回零零点位置。

◆ **下行帧的寄存器 1~2 数据：**

Byte1~ Byte4：回零零点位置（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 90 00 02 04 00 00 C8 00 AD 03

上行帧数据为：01 10 00 90 00 02 41 E5

Byte1~ Byte4：设置回零零点位置为 51200

5.5.2 设置有无限位回零指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~4 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~4 组高/低字节	高字节	低字节
01H	10H	00H	91H	00H	04H	08H	Byte1~ Byte8	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	91H	00H	04H	90H	27H

该指令用于设置有无限位回零。

◆ **下行帧的寄存器 1~4 数据：**

Byte1：使用无限位回零或有限位回零，

0：左无限位回零

1：右无限位回零

2：左有限位回零

3：右有限位回零

Byte2：电机旋转方向，0：正转；1：反转

Byte3~ Byte6：电机旋转速度：0~6000（单位 RPM）

Byte7~ Byte8：限位电流：0~3000（单位 mA），注意：限位电流，只对无限位回零有作用，用于检测是否到位

◆ **举个例子：**

下行帧数据为：01 10 00 91 00 04 08 01 00 00 00 07 D0 0B B8 4E 3B

上行帧数据为：01 10 00 91 00 04 90 27

Byte1：使用无限位回零或有限位回零，1：有限位回零

Byte2：电机旋转方向，0：正转

Byte3~ Byte6：电机旋转速度：2000 RPM

Byte7~ Byte8：限位电流：3000 mA

5.5.3 触发回零指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	92H	00H	01H	E9H	E7H

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	92H	00H	01H	E9H	E7H

该指令用于触发回零，回零模式有以下可能：

0：单圈回零（按完整一圈去找到原点信号）

1：就近回零（电机从当前位置开始，朝最近的原点位置移动）

2：多圈回零（找到绝对 0 点）

◆ **举个例子：**

下行帧数据为：01 06 00 92 00 01 E9 E7

上行帧数据为：01 06 00 92 00 01 E9 E7

回零模式：1：就近回零

5.5.4 强制中断并退出回零操作指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	93H	00H	01H	B8H	27H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	93H	00H	01H	B8H	27H

该指令用于强制中断并退出回零操作。

寄存器数据没有用到，默认给 01H。

◆ **举个例子：**

下行帧数据为：01 06 00 93 00 01 B8 27

上行帧数据为：01 06 00 93 00 01 B8 27

5.5.5 读取回零参数指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	94H	00H	09H	71H	E0H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1~9 数据			CRC16 校验码	
			1~9 组高/低字节			高字节	低字节
01H	04H	12H	Byte1~ Byte18			CRC16_H	CRC16_L

该指令用于读取回零参数。

Byte1：表示上电自动回零标志，有以下可能：

0：无需上电自动回零

1：上电自动回零

Byte2：表示回零状态，有以下可能：

0：空闲态

1：正在运行

2：回零完成

3：回零失败未找到零点

Byte3~ Byte4：表示限位电流，范围：0~3000（单位：mA）（类型 int16_t）

Byte5~ Byte8：表示左限位原点，（类型 int32_t）
Byte9~ Byte12：表示超时时间（单位：ms）（类型 uint32_t）
Byte13~ Byte16：表示右限位原点，（类型 int32_t）
Byte17：没用到，补 0 即可
Byte18：表示左右限位开关状态，有以下可能：
0：左右限位关闭　　1：左右限位开启

◆ **举个例子：**

下行帧数据为：01 04 00 94 00 09 71 E0
上行帧数据为：01 04 12 00 00 01 2C 00 00 C8 00 00 00 27 10 00 00 C8 00 00 01 4B 93
Byte1：0x0 表示上电自动回零标志，0：无需上电自动回零
Byte2：0x0 表示回零状态，0：空闲态
Byte3~ Byte4：0x012C 表示限位电流，300mA
Byte5~ Byte8：0x0000C8 00 表示左限位原点，51200
Byte9~ Byte12：0x00002710 表示超时时间，10000ms
Byte13~ Byte16：0x0000C8 00 表示右限位原点，51200
Byte17：0x0 无意义
Byte18：0x1 表示左右限位开启

5.5.6 修改原点回零超时时间指令

下行帧(上位机-->驱动板)									
从机地址	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~2 数据	CRC16 校验码	
		高字节	低字节	高字节	低字节		1~2 组高/低字节	高字节	低字节
01H	10H	00H	95H	00H	02H	04H	Byte1~ Byte4	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	95H	00H	02H	51H	E4H

该指令用于修改原点回零超时时间（单位：ms）。

◆ **下行帧的寄存器 1~2 数据：**

Byte1~ Byte4：原点回零超时时间（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 95 00 02 04 00 00 13 88 37 AA
上行帧数据为：01 10 00 95 00 02 51 E4
点回零超时时间为 5000ms

5.5.7 读取回零状态指令

下行帧(上位机-->驱动板)

从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	04H	00H	96H	00H	01H	D1H	E6H
上行帧(上位机<--驱动板)							
从机地址	功能码	字节数	寄存器 1 数据			CRC16 校验码	
			高字节	低字节		高字节	低字节
01H	04H	02H	Byte1	Byte2		CRC16_H	CRC16_L

该指令用于读取回零状态。

Byte1：没用到，补 0 即可

Byte2：表示回零状态，有以下可能：

0：空闲态

1：正在运行

2：回零完成

3：回零失败未找到零点

◆ **举个例子：**

下行帧数据为：01 04 00 96 00 01 D1 E6

上行帧数据为：01 04 02 00 00 B9 30

Byte2：0x00 表示空闲态

5.5.8 设置上电自动回零指令

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	97H	00H	01H	F9H	E6H
上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	97H	00H	01H	F9H	E6H

该指令用于设置上电是否自动回零，有以下可能：

0：上电不自动回零

1：上电自动回零

◆ **举个例子：**

下行帧数据为：01 06 00 97 00 01 F9 E6

上行帧数据为：01 06 00 97 00 01 F9 E6

是否自动回零：1：上电自动回零

5.5.9 设置右限位原点位置指令

下行帧(上位机-->驱动板)									
从机地	功能码	寄存器起始地址		寄存器数量		字节数	寄存器 1~2 数据	CRC16 校验码	
		高字	低字	高字	低字		1~2 组高/低字节	高字节	低字节

址		节	节	节	节				
01H	10H	00H	98H	00H	02H	04H	Byte1~ Byte4	CRC16_H	CRC16_L

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数量		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	10H	00H	98H	00H	02H	C0H	27H

该指令用于设置右限位原点位置。

◆ **下行帧的寄存器 1~2 数据：**

Byte1~ Byte4：右限位原点位置（类型 int32_t）

◆ **举个例子：**

下行帧数据为：01 10 00 98 00 02 04 00 00 C8 00 AC A5

上行帧数据为：01 10 00 98 00 02 C0 27

Byte1~ Byte4：设置右限位原点位置为 51200

5.5.10 设置左右限位开关状态

下行帧(上位机-->驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	99H	00H	01H	98H	25H

上行帧(上位机<--驱动板)							
从机地址	功能码	寄存器起始地址		寄存器数据		CRC16 校验码	
		高字节	低字节	高字节	低字节	高字节	低字节
01H	06H	00H	99H	00H	01H	98H	25H

该指令用于设置左右限位的开关状态，有以下可能：

0：关闭左右限位

1：开启左右限位

◆ **举个例子：**

下行帧数据为：01 06 00 99 00 01 98 25

上行帧数据为：01 06 00 99 00 01 98 25

左右限位的开关状态：1：开启左右限位，注意开启后位置运动距离受限于左右限位原点

6，上位机使用说明

6.1 USB/TTL/RS485 通信上位机使用说明

6.1.1 ATK_PDXX.exe

ATK_PDxx.exe 是正点原子为 ATK_PDxx 步进电机闭环驱动器推出的串口上位机，支持正点原子自定义协议和 MODBUS-RTU 协议，使用起来好用方便，主界面如图 6.1.1.1 所示：



图 6.1.1.1 主界面

可以看到，该上位机整体比较简约、模块化布局，一目了然。



图 6.1.1.2 主界面介绍

按照第 2 章接线方法，连接好硬件后，可以通过图 6.1.1.2 的步骤①~⑥，可以连接上 ATK_PDxx.exe，方便后续通讯。

⑦连接成功后，该框会显示当前的协议类型，可以通过下方按钮设置协议类型。

⑧该框用于设置驱动器的从机地址和分组地址。关于从机地址和分组地址的区别在 4.3.21 小节介绍到。

⑨该框内主要涉及系统类指令，详细的介绍请回顾第 4 章内容。

⑩该框用于系统参数读取和显示，这些参数都是只读，不可修改的。

⑪该框用于读取和设置驱动参数，这些参数都是可读可写的。

接下来，给大家介绍一下运动控制界面，如图 6.1.1.3 所示：



图 6.1.1.3 运动控制界面

①打开运动控制界面

②该框是电机运动用到的一些指令，有：使能电机、失能电机、立即停止、清除电机状态、清除当前位置、保存参数。注意：使用立即停止指令，会使电机严重发热。解决办法：可以使用失能电机指令代替，或者使用立即停止指令后，修改运动参数，再清除电机状态。

③ 该框涉及运动模式的参数设置和运动控制，控制方法可参考前面的章节。

接下来是固件升级界面，如图 6.1.1.4 所示：

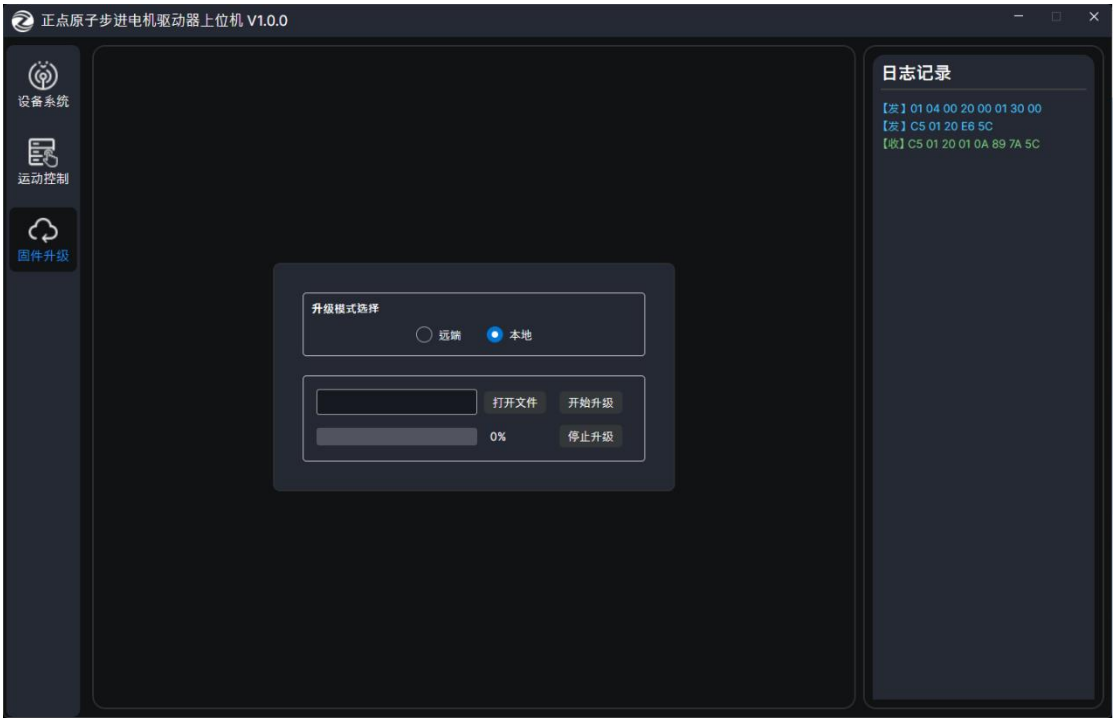


图 6.1.1.4 固件升级界面

固件升级支持远端和本地升级。下面我们会来详细介绍固件升级的操作步骤。

固件升级操作流程

固件升级准备：

1，USB-Type-C 线一根

2，PD42S1_Vx.x.atk 固件文件（远端升级则不需要）

3，ATK_PDXX.exe 上位机

4，Win10 系统以上电脑一台

固件升级方法：

1，PD42S1 先断电，然后**长按 ▶键**，再用 USB-Type-C 线的一端连接电脑 USB，Type-C 端给 PD42S1 上电，如图 6.1.1.5 所示。

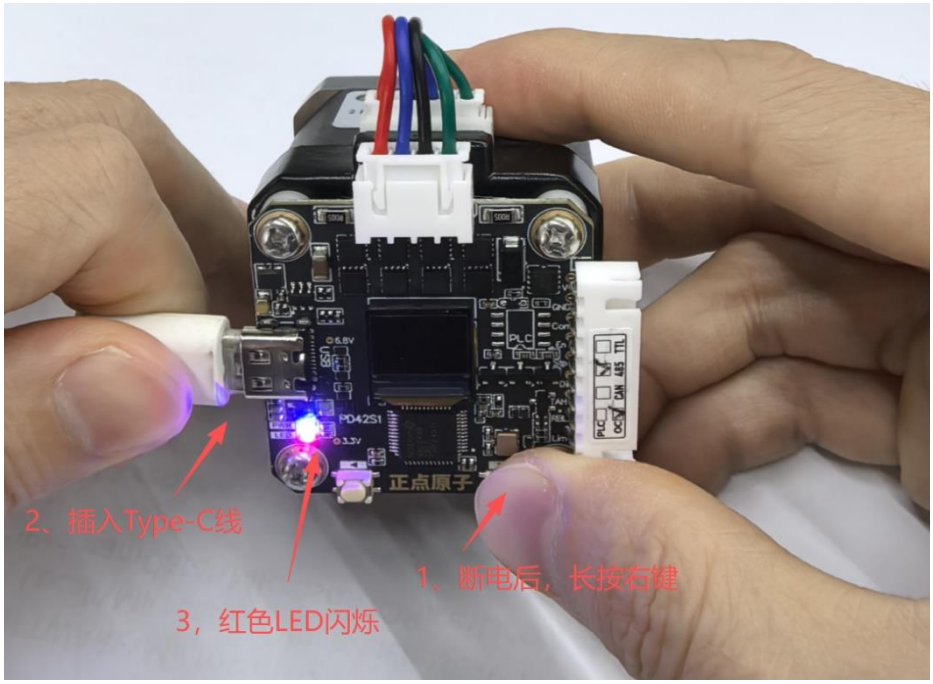


图 6.1.1.5 上电操作步骤

2，看到 PD42S1 板载的红色 LED 灯闪烁，代表成功进入固件升级模式，此时可松开按键。

3，打开 ATK_PDXX.exe 上位机连接好对应的 COM 端口，进入固件升级界面，按照图 6.1.1.6 步骤操作，开始升级固件。



图 6.1.1.6 上位机操作步骤

4，等待升级完成即可。最后就可以拔掉 Type-C 线，正常使用了。

6.1.2 ATK_XCOM.exe

ATK_XCOM.exe 是正点原子推出的通用款串口调试助手，相信大家都比较熟悉了。在发送指令控制步进电机闭环驱动器的使用中，按照第 2 章接线方法，连接好硬件后，需要进行如图 6.1.2.1 的设置，就可以测试正点原子自定义协议和 Modbus-RTU 协议的指令了。

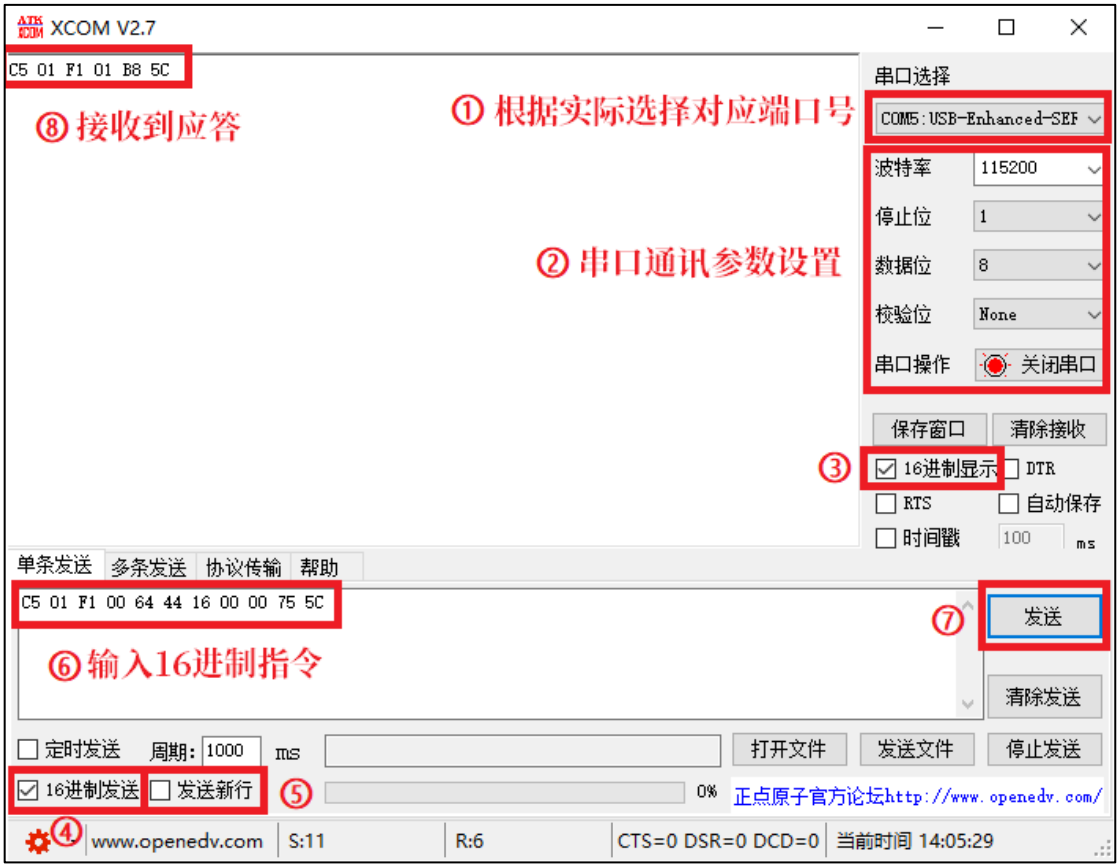


图 6.1.2.1 XCOM 设置

6.2 CAN 通信上位机使用说明

6.2.1 Cangaroo.exe

前文介绍的上位机程序均为 **串口通信类型**，仅支持 **TTL / RS485 接口**，不支持 **CAN 通信版本**。对于需要通过 **CAN 总线** 与驱动器进行通信的用户，可使用 **Cangaroo.exe** 作为调试与分析工具。该软件无需安装驱动，操作简便，兼容性强，是目前广受欢迎的 CAN 通信分析工具之一。

Cangaroo.exe 使用起来简单方便。按照第 2 章接线方法，连接好硬件后，打开 Cangaroo.exe 软件后，点击测量→开始测量，如图 6.2.1.1 所示：



图 6.2.1.1 开始测量

接着，选择接口、设置好比特率（默认为 500 Kbps）、点击 OK 即可，如图 6.2.1.2 所示：

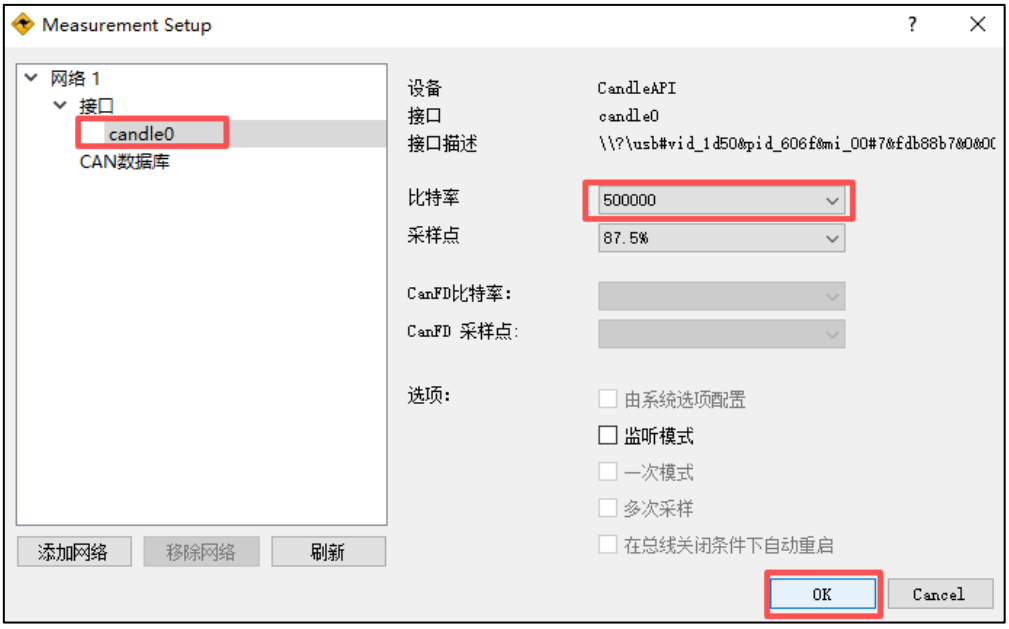


图 6.2.1.2 配置参数

下面就可以填写发送的数据帧格式和数据了，然后点击发送，就可以接收到从机的应答，如图 6.2.1.3 所示：



图 6.2.1.3 发送数据帧

注意：

1，CAN 通讯帧格式固定位扩展帧

2，ID 必须符合 0x100x 格式，其中最后的 x 表示任意 16 进制数。例如：0x1001、0x100A、0x100F 均为合法 ID。

3，CAN 通讯和串口 TTL /RS485 通讯发送的数据内容一样。

4，驱动板不支持 FDCAN，所以每条指令最多可以发送 8 个字节，对于多于 8 个字节数据的指令，目前上位机无法测试，可以通过我们提供的 CAN 通讯例程测试。

7，保修条款

一、服务期限

广州市星翼电子科技有限公司承诺产品的良好品质，产品自售出之日起 7 天内正常使用情况下机器出现故障，客户可以选择退货（不影响二次销售才可以退货）或者保修（非人为损坏）。

二、购买日期以销售商开出的购机发票或收据日期为准，网购可以以交易截图为准。

三、对于以下的情况，不提供免费保修服务

（1）一切人为损坏、私自改装、维修造成机器损坏的。

（2）不按电气参数规定使用，导致机器损坏的。

8，其他

1、 购买地址：

天猫：https://zhengdianyuanzi.tmall.com

淘宝：https://openedv.taobao.com

2、 资料下载

模块资料下载地址：http://www.openedv.com/docs/tool/USB/index.html

3、 技术支持

公司网址：www.alientek.com

技术论坛：http://www.openedv.com/forum.php

在线教学：www.yuanzige.com

B 站视频：https://space.bilibili.com/394620890

传真：020-36773971

电话：020-38271790







正点原子